

Kubernetes - Modul Advanced

Agenda

Tag 1 - Networking & Security

1. Vorbereitung
 - [kubectl Verbindung mit namespace einrichten](#)
 - [Das Tool kubectl - Spickzettel](#)
2. Cluster startklar machen: CNI installieren
 - [Übung: CNI-Provider Calico installieren](#)
3. MetalLB als Load-Balancer (Bare-Metal)
 - [Kubernetes Load Balancer - metalb](#)
 - [Feste IP beziehen](#)
4. Kubernetes-Networking-Grundlagen
 - [Networking Internal Overview](#)
 - [Cluster-CIDR, POD-CIDR und Service-CIDR](#)
 - [Wann wird die PodIP vergeben?](#)
 - [CNI - Wie funktioniert das unter der Haube](#)
 - [Überblick CNI-Provider](#)
 - [Weg vom Pod zum Host -> veth / calicoctl get wip](#)
5. Network Policies
 - [Einfache Übung NetworkPolicy \(Standard\)](#)
 - [Warum Calico-Policies statt Standard-NetworkPolicy?](#)
 - [Calico-Policies - Grundlagen \(Ordering, Implicit Deny, API-Version\)](#)
 - [Erweiterte Policies mit Calico - Übung](#)
6. RBAC & Identity
 - [Least Privileges mit RBAC](#)
 - [Wie funktioniert RBAC?](#)
 - [Wo spielt RBAC eine Rolle?](#)
 - [kubectl - Berechtigungen prüfen mit can-i](#)
 - [ServiceAccounts: kubectl im Pod - default ServiceAccount](#)
 - [ServiceAccounts: Automount - ja oder nein?](#)
 - [Praktische Übung: User mit Zertifikat anlegen \(kubeconfig\)](#)
 - [Praktische Übung RBAC \(ab Kubernetes 1.25\)](#)
 - [Praktische Übung: RBAC-Hygiene - Label-Konvention prüfen und Nutzung im Audit-Log nachweisen](#)
7. Secrets Management mit HashiCorp Vault
 - [Vault-Architektur einfach erklärt](#)
 - [HashiCorp Vault als Password-Safe \(Overview\)](#)
 - [Übung: MariaDB-Deployment mit HashiCorp Vault über den Vault Secrets Operator \(VSO\)](#)
 - [Übung: MariaDB-Deployment mit dem Vault Agent Injector](#)
8. Workload-Skalierung
 - [Autoscaling Pods/Deployments - Grundlagen](#)
 - [Übung: Horizontal Pod Autoscaler \(HPA\)](#)
 - [Übung: HPA mit eigener Metrik \(KEDA + eigener Prometheus\)](#)

Tag 2 - Observability, Service Mesh & GitOps

1. Monitoring mit Prometheus
 - [Prometheus Monitoring Server \(Overview\)](#)
 - [Prometheus/Grafana-Stack installieren mit helm \(Traefik + Letsencrypt\)](#)
 - [Übung: nginx mit ServiceMonitor und Exporter \(Sidecar\)](#)
2. Logging-Stack: EFK (Elasticsearch/Fluentd/Kibana)
 - [EFK-Stack: Aufbau, Fluentd vs. Fluent Bit, DaemonSet vs. Sidecar](#)
3. Alternative: Splunk-Integration
 - [Theorie: Kubernetes mit Splunk verbinden](#)
 - [Funktionsübersicht: Splunk-Menuepunkte und Kubernetes-Relevanz](#)
 - [Log-Forwarder an externen Splunk-Server anbinden](#)
 - [Abstuerzenden Pod über Splunk debuggen \(CrashLoopBackOff\)](#)
 - [CrashLoopBackOff-Alert einrichten \(optional\)](#)
 - [Optional: Splunk im Cluster betreiben \(Splunk Operator\)](#)
4. Troubleshooting
 - [Debugging von Pods \(Logs, Events, typische Fehlerbilder\)](#)
 - [kubectl debug - Ephemeral Container](#)

- [Host/Node erforschen mit kubectl debug \(z.B. CNI\)](#)
- [ClusterIP debuggen](#)

5. Service Mesh - Istio & Envoy verstehen

- [Einführung in Istio & Service-Mesh-Architekturen](#)
- [Warum ein Service Mesh?](#)
- [Herausforderungen & Vorteile](#)
- [Architektur & Komponenten von Istio](#)
- [Istio Proxy-Konzepte \(Envoy als Sidecar\)](#)
- [Vergleich mit Linkerd, Cilium, Consul](#)

6. Service Mesh - Praktischer Aufbau im Cluster (Sidecar-Modus)

- [Istio-Installation mit istioctl \(demo-Profil\)](#)
- [istioctl Cheatsheet zum Debuggen](#)
- [Übung: Sidecar-Injection](#)
- [Demo-App bookinfo installieren](#)
- [Übung: Header-basiertes Routing](#)
- [Übung: Traffic-Shifting / Load-Balancing](#)
- [Debugging mit debug/run_pod](#)

7. Service Mesh - Praktischer Aufbau mit Ambient-Mode (Gateway API statt Sidecar)

- [Istio-Installation mit istioctl \(Ambient-Profil\)](#)
- [istioctl Cheatsheet zum Debuggen](#)
- [Übung: Workload ins Ambient-Mesh aufnehmen \(statt Sidecar-Injection\)](#)
- [Demo-App bookinfo installieren \(Ambient/Waypoint\)](#)
- [Übung: Header-basiertes Routing \(Gateway API HTTPRoute\)](#)
- [Übung: Traffic-Shifting \(Gateway API HTTPRoute\)](#)
- [Debugging mit debug/run_pod \(Ambient: ztunnel + Waypoint\)](#)

8. GitOps mit Flux

- [ArgoCD vs. Flux CD im Ueberblick](#)
- [Flux Ueberblick - Controller, CRDs und Ablauf](#)
- [Flux Installation und GitOps-Sync mit dem Flux Operator](#)
- [HelmRepository - Helm Chart Repositories verwalten](#)
- [HelmRelease - Helm Charts deklarativ ausrollen](#)
- [OCI-Helm-Chart verwenden](#)
- [Eigenes Helm Chart aus Git-Repository ausrollen](#)
- [Übung: Flux-Operator Web-UI mit Ingress und HTTPS absichern](#)

9. Abschluss

- Best Practices & Hands-on Labs
- Fehler vermeiden, Debugging meistern

Backlog

1. Autoscaling für fpm-php (Gedankenexperimente, nicht sinnvoll)

Vorbereitung

kubectl Verbindung mit namespace einrichten

config einrichten

```
cd
mkdir -p .kube
cd .kube
cp -a /tmp/config config
ls -la
## das bekommt ihr aus Eurem Cluster Management Tool
```

```
kubectl cluster-info
```

Arbeitsbereich konfigurieren

```
kubectl create ns jochen
kubectl get ns
kubectl config set-context --current --namespace jochen
kubectl get pods
```

Das Tool kubectl - Spickzettel

Allgemein

```
## Zeige Information über das Cluster
kubectl cluster-info

## Welche api-resources gibt es ?
kubectl api-resources

## Hilfe zu object und eigenschaften bekommen
kubectl explain pod
kubectl explain pod.metadata
kubectl explain pod.metadata.name
```

Arbeiten mit manifesten

```
kubectl apply -f nginx-replicaset.yml
## Wie ist aktuell die hinterlegte config im system
kubectl get -o yaml -f nginx-replicaset.yml

## Änderung in nginx-replicaset.yml z.B. replicas: 4
## dry-run - was wird geändert
kubectl diff -f nginx-replicaset.yml

## anwenden
kubectl apply -f nginx-replicaset.yml

## Alle Objekte aus manifest löschen
kubectl delete -f nginx-replicaset.yml
```

Ausgabeformate

```
## Ausgabe kann in verschiedenen Formaten erfolgen
kubectl get pods -o wide # weitere informationen
## im json format
kubectl get pods -o json

## gilt natürluch auch für andere kommandos
kubectl get deploy -o json
kubectl get deploy -o yaml

## get a specific value from the complete json - tree
kubectl get node k8s-nue-jo-ff1p1 -o=jsonpath='{.metadata.labels}'
```

Zu den Pods

```
## Start einen pod // BESSER: direkt manifest verwenden
## kubectl run podname --image=imagename
kubectl run nginx --image=nginx
```

```
## Pods anzeigen
kubectl get pods
kubectl get pod

## Format weitere Information
kubectl get pod -o wide
## Zeige labels der Pods
kubectl get pods --show-labels

## Pods aus allen Namespaces anzeigen
kubectl get pods -A

## Zeige pods mit einem bestimmten label
kubectl get pods -l app=nginx

## Status eines Pods anzeigen
kubectl describe pod nginx

## Pod löschen
kubectl delete pod nginx

## Kommando in pod ausführen
kubectl exec -it nginx -- bash
## direkt in den 1. Pod des Deployments wechseln
kubectl exec -it deployment/name-des-deployments -- bash
```

Logs ausgeben

```
kubectl logs podname
## -n = namespace
## | less -> seitenweise Ausgabe
kubectl -n ingress logs nginx-ingress-nginx-controller-7bc7c7776d-jpj5h | less
```

Arbeiten mit namespaces

```
## Welche namespaces auf dem System
kubectl get ns
kubectl get namespaces
## Standardmäßig wird immer der default namespace verwendet
## wenn man kommandos aufruft
kubectl get deployments

## Möchte ich z.B. deployment vom kube-system (installation) aufrufen,
## kann ich den namespace angeben
kubectl get deployments --namespace=kube-system
kubectl get deployments -n kube-system

## wir wollen unseren default namespace ändern
kubectl config set-context --current --namespace <dein-namespace>
```

Referenz

- <https://kubernetes.io/de/docs/reference/kubectl/cheatsheet/>

Cluster startklar machen: CNI installieren

Uebung: CNI-Provider Calico installieren

Hintergrund

- Dein Cluster wurde bewusst OHNE CNI-Plugin provisioniert (nur kubeadm init/join)
- Ohne CNI: Nodes bleiben `NotReady`, CoreDNS bleibt `Pending` - es koennen keine normalen Pods starten
- Genau das schauen wir uns erst an, dann beheben wir es

Schritt 1: Ausgangslage ansehen (kaputt by design)

```
kubectl get nodes
## STATUS: NotReady - warum?

kubectl describe node <dein-cp-node> | grep -A 3 "Ready "
## message: Network plugin returns error: cni plugin not initialized

kubectl -n kube-system get pods
## coredns: Pending (kein CNI -> kein Pod-Netz -> nicht schedulebar)
```

Schritt 2: Tigera-Operator installieren

- Wichtig: `kubectl create` (nicht `apply`) - die Manifeste sind zu gross fuer die apply-Annotation

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.32.2/manifests/tigera-operator.yaml
```

```
## Operator laeuft?  
kubectl -n tigera-operator get pods
```

Schritt 3: Calico-Konfiguration (Custom Resources) anlegen

- Die `Installation` -Resource sagt dem Operator, wie er Calico ausrollen soll
- Das `Default-Pod-Netz` darin (`192.168.0.0/16`) passt zu unserem `kubeadm-Setup` (`--pod-network-cidr=192.168.0.0/16`)

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.32.2/manifests/custom-resources.yaml
```

Schritt 4: Zuschauen, wie das Cluster "heile" wird

```
## 1-3 Minuten, bis alles laeuft  
kubectl -n calico-system get pods -w  
## Ctrl+C wenn calico-node auf allen Nodes Running ist
```

```
kubectl -n calico-system get pods  
kubectl get nodes  
## STATUS: Ready !
```

```
kubectl -n kube-system get pods  
## coredns: Running
```

Schritt 5: Funktionstest - jetzt starten auch normale Pods

```
kubectl run cni-test --image=nginx  
kubectl get pods -o wide  
## Running, mit IP aus 192.168.x.x
```

Aufraeumen

- Calico bleibt natuerlich installiert - nur den Test-Pod entfernen:

```
kubectl delete pod cni-test
```

Reference

- <https://docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart>

MetalLB als Load-Balancer (Bare-Metal)

Kubernetes Load Balancer - metallb

General

- Supports bgp and arp (l2 mode) - this exercise uses l2/arp
- Divided into controller (ipam), speaker (advertises the ip)

Installation Ways

- helm
- manifests

Step 1: install metallb

```
## We use L2 mode (arp), not bgp  
## The speaker is required in both modes - it is the component that  
## actually announces the IP on the network (controller only does IPAM)
```

```
helm repo add metallb https://metallb.github.io/metallb
```

```
## reset-values, always reset values on upgrade  
## Attention: 0.16.1 is buggy, operator fires a lot of api-calls to the kube-api-server -> do not use before fix  
helm upgrade --install metallb metallb/metallb --namespace=metallb-system --create-namespace --version 0.15.3 --reset-values
```

Step 2: addresspool

```
## find your node public ips first - "kubectl get nodes -o wide" does NOT  
## show them (no cloud-controller-manager here), EXTERNAL-IP stays <none>.  
## they are listed in ~/cluster-zugang.txt on client-bka.  
cat ~/cluster-zugang.txt
```

```
cd  
mkdir -p manifests  
cd manifests
```

```
mkdir lb
cd lb
nano 01-addresspool.yml
```

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: first-pool
  namespace: metallb-system
spec:
  addresses:
    # hier die ip-adressen Deiner 3 worker nodes eintragen
    - 157.230.113.124/32
```

```
kubectl apply -f .
```

Step 3: L2Advertisement

```
nano 02-advertisement.yml
```

```
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: example
  namespace: metallb-system
```

```
kubectl apply -f .
```

Step 4: Test do i get an external ip

```
nano 03-deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-nginx
spec:
  selector:
    matchLabels:
      run: web-nginx
  replicas: 3
  template:
    metadata:
      labels:
        run: web-nginx
    spec:
      containers:
        - name: cont-nginx
          image: nginx
          ports:
            - containerPort: 80
```

```
nano 04-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: svc-nginx
spec:
  type: LoadBalancer
  ports:
    - port: 80
      protocol: TCP
  selector:
    run: web-nginx
```

```
kubectl apply -f .
kubectl get pods
kubectl get svc
kubectl describe svc svc-nginx
```

```
## auf dem client
curl http://<ip aus get svc>
```

```
kubectl delete -f 03-deploy.yml -f 04-service.yml
```

Step 5: Referenz:

- <https://metallb.io/installation/#installation-with-helm>

Feste IP beziehen

Beispiel

```
cd manifests/lb
```

```
## bestehende 04-service.yml aus der metallb-Uebung anpassen (NICHT neu anlegen,  
## sonst kollidiert der Service-Name svc-nginx mit zwei Manifests im selben Ordner)  
nano 04-service.yml
```

```
apiVersion: v1  
kind: Service  
metadata:  
  name: svc-nginx  
  labels:  
    svc: nginx  
  annotations:  
    # spec.loadBalancerIP ist deprecated (seit 1.24) - MetallB nutzt stattdessen  
    # diese Annotation, eure ip aus dem pool nehmen  
    metallb.io/loadBalancerIPs: 167.99.130.85  
spec:  
  type: LoadBalancer  
  ports:  
  - port: 80  
    protocol: TCP  
  selector:  
    run: web-nginx
```

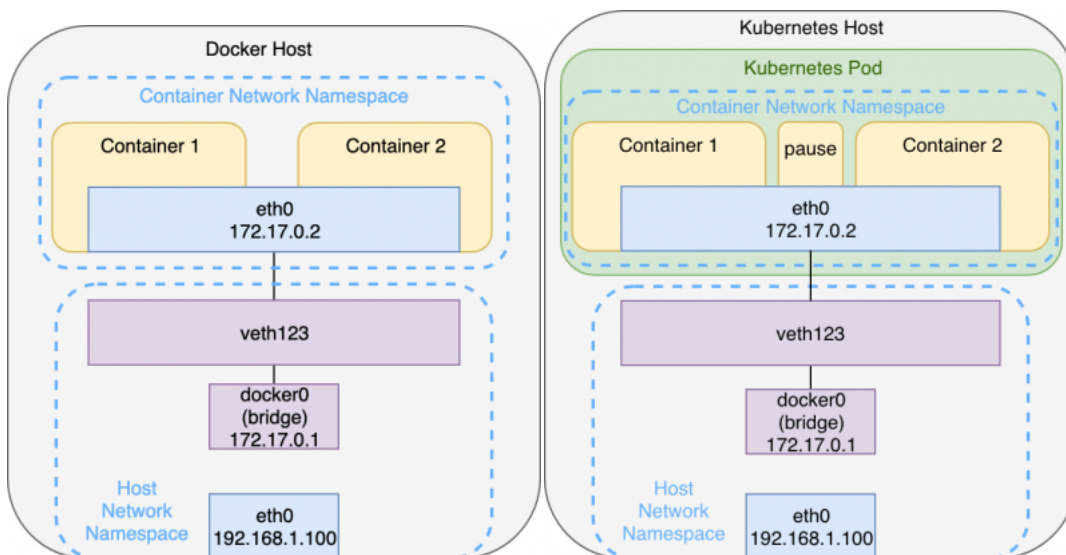
```
kubectl apply -f 04-service.yml  
## ist es die von oben ?  
kubectl get svc
```

Kubernetes-Networking-Grundlagen

Networking Internal Overview

Network Namespace for each pod

Overview



General

- Each pod will have its own network namespace
 - with routing, network devices
- Connection to default namespace to host is done through veth - Link to bridge on host network
 - similar like on docker to docker0

Each container is connected to the bridge via a veth-pair. This interface pair functions like a virtual point-to-point ethernet connection and connects the network namespaces of the containers with the network namespace of the host

- Every container is in the same Network Namespace, so they can communicate through localhost
 - Example with hashicorp/http-echo container 1 and busybox container 2

Pod-To-Pod Communication (across nodes)

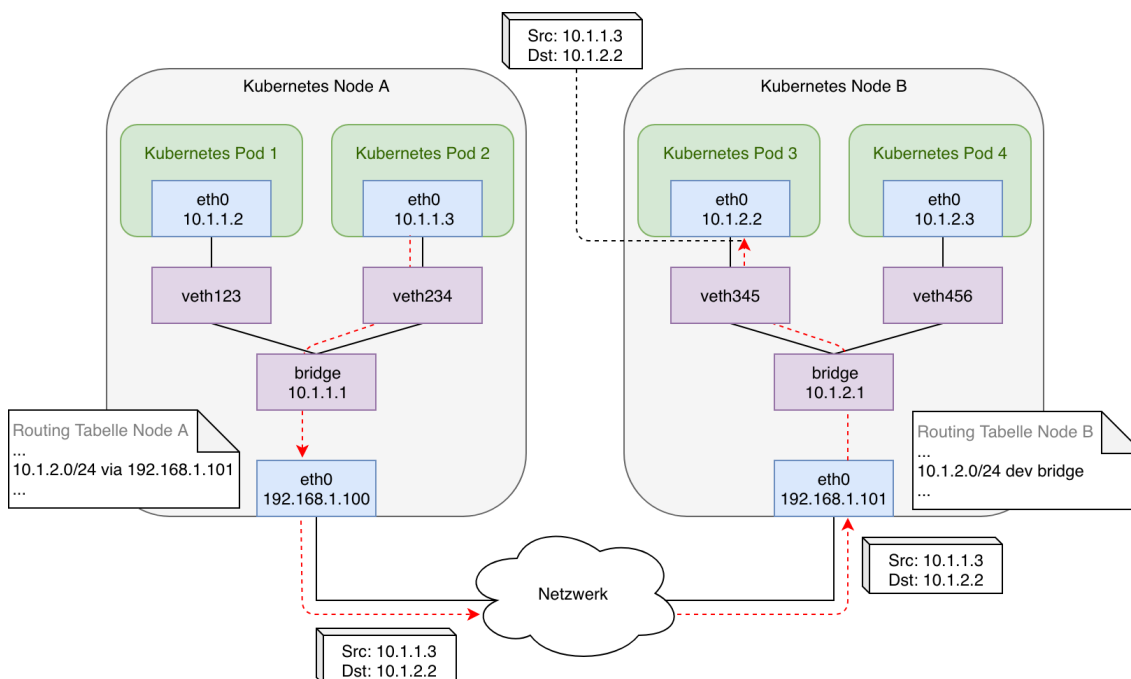
Prerequisites

- pods on a single node as well as pods on a topological remote can establish communication at all times
- Each pod receives a unique IP address, valid anywhere in the cluster. Kubernetes requires this address to not be subject to network address translation (NAT)
- Pods on the same node through virtual bridge (see image above)

General (what needs to be done) - and could be done manually

- local bridge networks of all nodes need to be connected
- there needs to be an IPAM (IP-Address Management) so addresses are only used once
- The need to be routes so, that each bridge can communicate with the bridge on the other network
- Plus: There needs to be a rule for incoming network
- Also: A tunnel needs to be set up to the outside world.

General - Pod-to-Pod Communication (across nodes) - what would need to be done



General - Pod-to-Pod Communication (side-note)

- This could of cause be done manually, but it is too complex
- So Kubernetes has created an Interface, which is well defined
 - The interface is called CNI (common network interface)
 - Funtionally is achieved through Network Plugin (which use this interface)
 - e.g. calico / cilium / weave net / flannel

CNI

- CNI only handles network connectivity of container and the cleanup of allocated resources (i.e. IP addresses) after containers have been deleted (garbage collection) and therefore is lightweight and quite easy to implement.
- There are some basic libraries within CNI which do some basic stuff.

Hidden Pause Container

What is for ?

- Holds the network - namespace for the pod
- Gets started first and falls asleep later
- Will still be there, when the other containers die

```
cd
mkdir -p manifests
cd manifests
mkdir pausetest
```



```
cd pausetest
nano 01-nginx.yml
```

```
## vi nginx-static.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pausetest
  labels:
    webserver: nginx
spec:
  containers:
  - name: web
    image: nginx
```

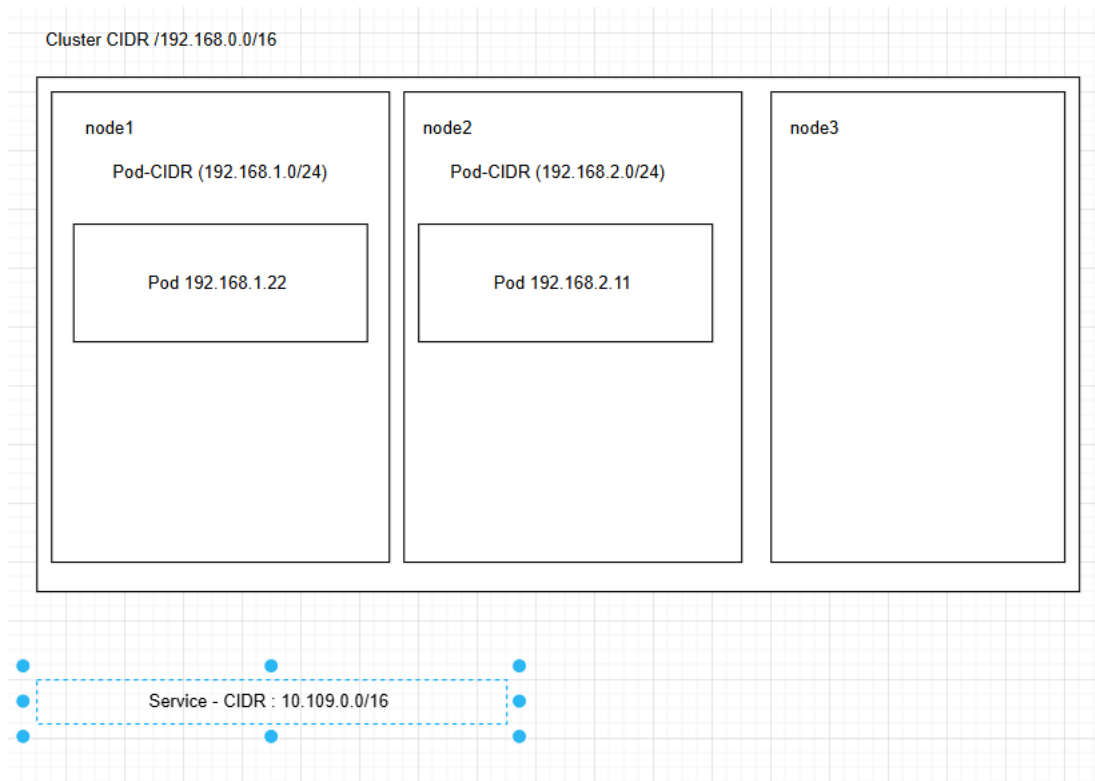
```
kubectl apply -f .
## als root auf dem worker node
ctr -n k8s.io c list | grep pause
```

References

- <https://www.inovex.de/de/blog/kubernetes-networking-part-1-en/>
- <https://www.inovex.de/de/blog/kubernetes-networking-2-calico-cilium-weavenet/>

Cluster-CIDR, POD-CIDR und Service-CIDR

Grafik



Cluster CIDR - IP-Bereich für das gesamte Kubernetes Cluster

```
## Netzbereich für mein gesamtes Cluster
10.244.0.0/16
```

POD-CIDR - Teilbereich aus der Cluster - CIDR pro Node

```
## Jede Node bekommt ein Teilnetz
Beispiel cilium

node 1 -> network.cilium.io/ipv4-pod-cidr: 10.244.0.0/25
node 2 -> network.cilium.io/ipv4-pod-cidr: 10.244.0.128/25
```

```
node 3 -> network.cilium.io/ipv4-pod-cidr: 10.244.1.128/25
node 4 -> network.cilium.io/ipv4-pod-cidr: 10.244.1.0/25
```

POD-IP

- Wird aus POD-CIDR des jeweiligen Nodes vergeben

```
## pod bekommt aus netzbereich POD-CIDR auf Node eine IP-Adresse zugewiesen
## CILIUUM CNI macht das z.B.
POD-CIDR: 10.244.1.128/25
-> POD - IP: 10.244.1.180
```

Service-CIDR

```
Netzbereich für IP-Adressen der Services
z.B. 10.109.0.0/16
```

Wann wird die PodIP vergeben?

Example (that does work)

```
## Show the pods that are running
kubectl get pods

## Synopsis (most simplistic example
## kubectl run NAME --image=IMAGE_EG_FROM_DOCKER
## example
kubectl run nginx --image=nginx:1.23

kubectl get pods
## on which node does it run ?
kubectl get pods -o wide
```

Example (that does not work)

```
kubectl run foo2 --image=foo2
## ImageErrPull - Image konnte nicht geladen werden
kubectl get pods
## Weitere status - info
kubectl describe pods foo2

## Auch der nicht laufende Pod
kubectl get pods -o wide
```

Ref:

- <https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#run>

CNI - Wie funktioniert das unter der Haube

Referenz:

- <https://isovalent.com/blog/post/demystifying-cni/>

Ablauf

- Containerd ruft CNI plugin über subcommandos: ADD, DEL, CHECK, VERSION auf (mehr subcommandos gibt es nicht)
- Was gemacht werden soll wird über JSON-Objekt übergeben
- Die Antwort kommt auch wieder als JSON zurück

Plugins die Standardmäßig schon da sind

- <https://www.cni.dev/plugins/current/>

CNI-Provider

- Ein Kubernetes-Cluster braucht immer ein CNI-Provider, sonst funktioniert die Kommunikation nicht und die Nodes im Cluster stehen auf NotReady
- Beispiele: Calico, WeaveNet, Antrea, Cilium, Flannel

IPAM - IP Address Management

- Ziel ist, dass Adressen nicht mehrmals vergeben werden.
- Dazu wird ein Pool bereitgestellt.
- Es gibt 3 CNI IPAM - Module:
 - host-local
 - dhcp
 - static

```
* IPAM: IP address allocation
dhcp : Runs a daemon on the host to make DHCP requests on behalf of a container
```

```
host-local : Maintains a local database of allocated IPs
static : Allocates static IPv4/IPv6 addresses to containers
```

Beispiel json für antrea (wird verwendet beim Aufruf von CNI)

```
root@worker1:/etc/cni/net.d# cat 10-antrea.conflist
{
  "cniVersion": "0.3.0",
  "name": "antrea",
  "plugins": [
    {
      "type": "antrea",
      "ipam": {
        "type": "host-local"
      }
    },
    {
      "type": "portmap",
      "capabilities": {"portMappings": true}
    },
    {
      "type": "bandwidth",
      "capabilities": {"bandwidth": true}
    }
  ]
}
```

Ueberblick CNI-Provider

CNI

- Common Network Interface
- Feste Definition, wie Pod mit Netzwerk-Bibliotheken kommunizieren

Welche gibt es ?

- Flannel
- Canal
- Calico
- Cilium
- Antrea (vmware)

Flannel

Generell

- Flannel is a CNI which gives a subnet to each host for use with container runtimes.

Overlay - Netzwerk

- virtuelles Netzwerk was sich oben drüber und eigentlich auf Netzwerkebene nicht existiert
- VXLAN

Vorteile

- Guter einfacher Einstieg
- reduziert auf eine Binary flanneld

Nachteile

- keine Firewall - Policies möglich
- keine klassischen Netzwerk-Tools zum Debuggen möglich.

Guter Einstieg in flannel

- <https://mvalim.github.io/kubernetes-under-the-hood/documentation/kube-flannel.html>

Canal

General

- Auch ein Overlay - Netzwerk
- Unterstützt auch policies

- Kombination aus Flannel (Overlay) und den NetworkPolicies aus Calico

Calico



Komponenten

Calico API server

- Lets you manage Calico resources directly with kubectl.

Felix

Main task: Programs routes and ACLs, and anything else required on the host to provide desired connectivity for the endpoints on that host. Runs on each machine that hosts endpoints. Runs as an agent daemon.

BIRD

- Gets routes from Felix and distributes to BGP peers on the network for inter-host routing. Runs on each node that hosts a Felix agent. Open source, internet routing daemon.

confd

Monitors Calico datastore for changes to BGP configuration and global defaults such as AS number, logging levels, and IPAM information. Open source, lightweight configuration management tool.

Conf dynamically generates BIRD configuration files based on the updates to data in the datastore. When the configuration file changes, confd triggers BIRD to load the new files

Dikastes

Enforces NetworkPolicy for istio service mesh

CNI plugin

Datastore plugin

IPAM plugin

kube-controllers

Main task: Monitors the Kubernetes API and performs actions based on cluster state. kube-controllers.

The tigera/kube-controllers container includes the following controllers:

Policy controller
 Namespace controller
 Serviceaccount controller
 Workloadendpoint controller
 Node controller

Typha

Typha maintains a single datastore connection on behalf of all of its clients like Felix and confd. It caches the datastore state and deduplicates events so that they can be fanned out to many listeners.

calicoctl

- Wird heute selten gebraucht, da das meiste heute mit kubectl über den Calico API Server realisiert werden kann
- Früher haben die neuesten NetworkPolicies/v3 nur über calicoctl funktioniert

Generell

- klassische Netzwerk (BGP) - kein Overlay
- klassische Netzwerk-Tools können verwendet werden.
- eBPF ist implementiert, aber muss aktiviert

Vorteile gegenüber Flannel

- Policy über Kubernetes Object (NetworkPolicies)

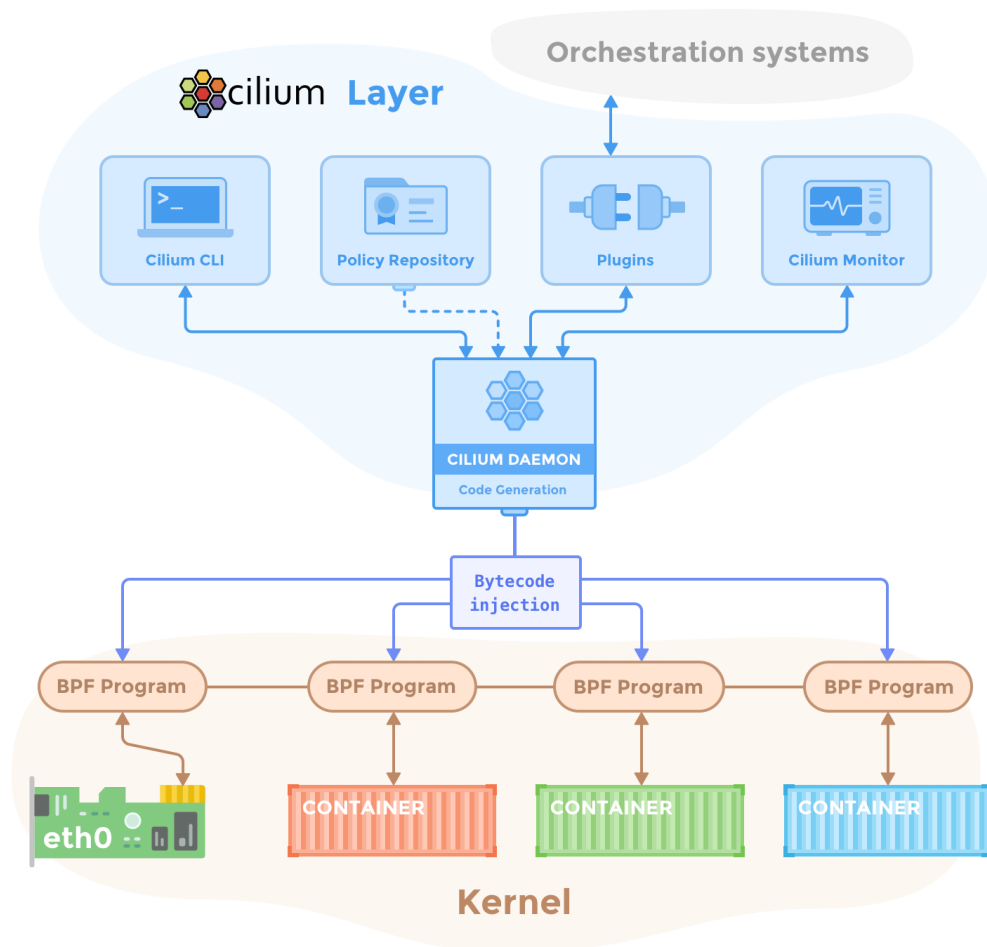
Vorteile

- ISTIO integrierbar (Service Mesh)
- Performance etwas besser als Flannel (weil keine Encapsulation)

Referenz

- <https://projectcalico.docs.tigera.io/security/calico-network-policy>

Cilium



Komponenten:

Cilium Agent

- Läuft auf jeder Node im Cluster
- Lauscht auf events from Orchestrierer (z.B. container gestoppt und gestartet)
- Managed die eBPF - Programme, die Linux kernel verwendet um den Netzwerkzugriff aus und in die Container zu kontrollieren

Client (CLI)

- Wird im Agent mit installiert (interagiert mit dem agent auf dem gleichen Node)
- Kann aber auch auf dem Client installiert werden auf dem kubect! läuft.

Cilium Operator

- Zuständig dafür, dass die Agents auf den einzelnen Nodes ausgerollt werden
- Es gibt ihn nur 1x im Cluster
- Ist unkritisch, sobald alles ausgerollt ist.
 - wenn dieser nicht läuft funktioniert das Networking trotzdem

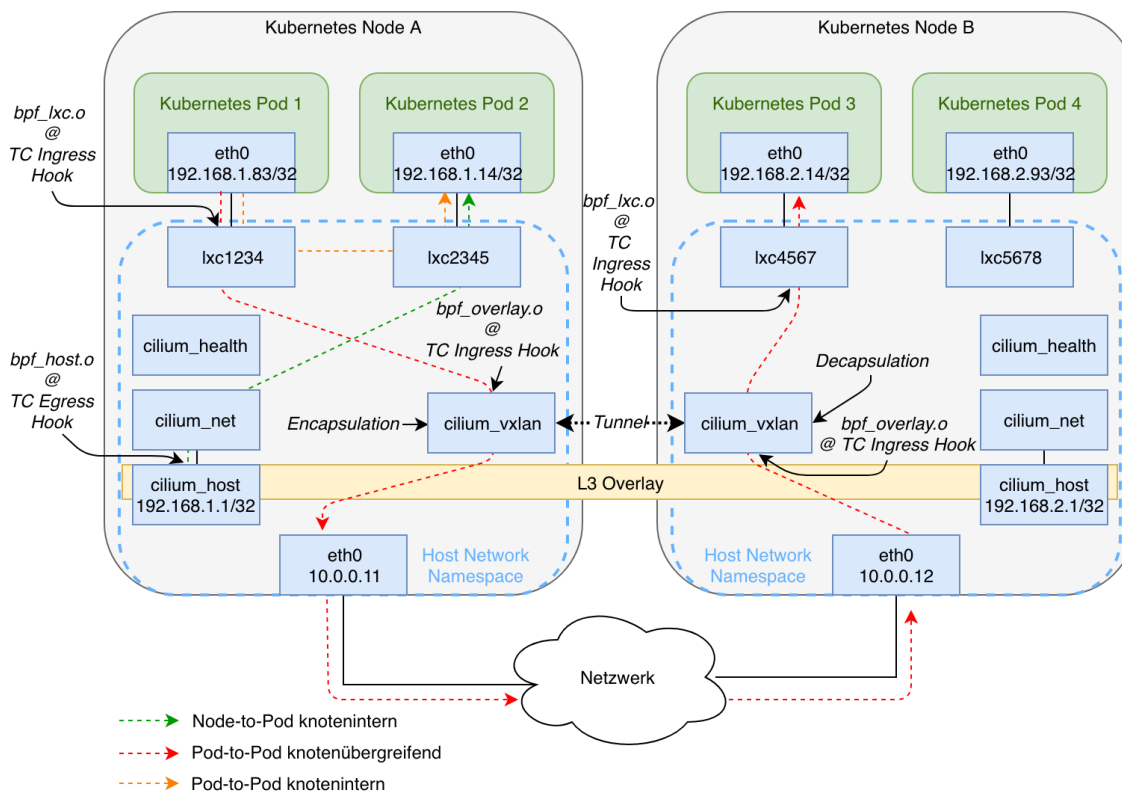
cilium CNI - Plugin

- Ist ein binary auf dem server (worker)
- wird durch die Container Runtime ausgeführt.
- cilium cni plugin interagiert mit der Cilium API auf dem Node

Datastore

- Daten werden per Default in CRD (Custom Resource Definitions) gespeichert
- Diese Resource Objekte werden von Cilium definiert und angelegt.
 - Wenn Sie angelegt sind, sind die Daten dadurch automatisch im etc - Speicher
 - Mit der weiteren Möglichkeit den Status zu speichern.
- Alternative: Speichern der Daten direkt in etcd

Generell



- Quelle: <https://www.inovex.de/de/blog/kubernetes-networking-2-calico-cilium-weavenet/>
- Verwendet keine Bridge sondern Hooks im Kernel, die mit eBPF aufgesetzt werden
 - Bessere Performance
- eBPF wird auch für NetworkPolicies unter der Haube eingesetzt
- Mit Ciliums Cluster Mesh lassen sich mehrere Cluster miteinander verbinden:

Vorteile

- Höhere Leistung mit eBPF-Ansatz. (extended Berkely Packet Filter)
 - JIT - Just in time compiled -
 - Bytecode wird zu Maschinencode kompiliert (Miniprogramme im Kernel)
- Ersatz für iptables (wesentlich schneller und keine Degredation wie iptables ab 5000 Services)
- Gut geeignet für größere Cluster

Weave Net

- Ähnlich calico
- Verwendet overlay netzwerk
- Sehr stabil bzgl IPV4/IPV6 (Dual Stack)
- Sehr grosses Feature-Set
- mit das älteste Plugin

Weg vom Pod zum Host -> veth / calicoctl get wep

Walkthrough (without calicoctl)

```
## Step 1: create pod
kubectl run nginx-master --image=nginx
## Find out on which node it runs
kubectl get pods -o wide
## create a debug container
kubectl debug -it nginx-master --image=busybox
```

```
## now within debug pod found out interface
ip a | grep @
```

```
## Ausgabe
3: eth0@if22: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
```

```
## Log in to worker node where pod runs and check interfaces
kubectl debug -it node/worker1 --image=busybox
```

```
## on worker node
## show matched line starting with 22 and then another 4 lines
ip a | grep -A 5 ^22
## e.g.
##
ip a | grep -A 5 ^22
22: cali42c2aab93f3@if3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether ee:ee:ee:ee:ee:ee brd ff:ff:ff:ff:ff:ff link-netns cni-5adf994b-3a7e-c344-5d82-ef1f7a293d88
    inet6 fe80::ecee:eeff:ffff:eeee/64 scope link
        valid_lft forever preferred_lft forever
```

Get information with calicoctl (installed on client)

```
## für den namespace default bzw. den konfigurierten
calicoctl get wep
calicoctl get workloadendpoints

## für alle namespaces
calicoctl get wep -A
```

Firewall - Regeln

```
## Now you are able to determine the firewall rules
## you will find fw and tw rules (fw - from workload and tw - to workload)
iptables-legacy -L -v | grep cali42c2aab93f3
```

```
## ... That is what you see as an example
Chain cali-tw-cali42c2aab93f3 (1 references)
  pkts bytes target prot opt in out source destination
    10 1384 ACCEPT all -- any any anywhere anywhere /* cali:WKA8EzdUNM0rVty1 */ ctstate
RELATED,ESTABLISHED
    0 0 DROP all -- any any anywhere anywhere /* cali:wr_OqGKKIN_LWnX0 */ ctstate
INVALID
    0 0 MARK all -- any any anywhere anywhere /* cali:kOUMqNj8np60A3Bi */ MARK and
0xffffffffff
```

Network Policies

Einfache Uebung NetworkPolicy (Standard)

Schritt 1: Deployment und Service erstellen

```
KURZ=jm
kubectl create ns policy-demo-$KURZ
```

```
cd
mkdir -p manifests
cd manifests
mkdir -p np
cd np
```

```
nano 01-deployment.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  replicas: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.23
          ports:
            - containerPort: 80
```

```
kubectl -n policy-demo-$KURZ apply -f .
```

```
nano 02-service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
spec:
  type: ClusterIP # Default Wert
  ports:
    - port: 80
      protocol: TCP
  selector:
    app: nginx
```

```
kubectl -n policy-demo-$KURZ apply -f .
```

Schritt 2: Zugriff testen ohne Regeln

```
## lassen einen 2. pod laufen mit dem auf den nginx zugreifen
kubectl run --namespace=policy-demo-$KURZ access --rm -ti --image busybox
```

```
## innerhalb der shell
wget -q nginx -O -
```

```
## Optional: Pod anzeigen in 2. ssh-session zu jump-host
kubectl -n policy-demo-$KURZ get pods --show-labels
```

Schritt 3: Policy festlegen, dass kein Zugriff erlaubt ist.

```
nano 03-default-deny.yaml
```

```
## Schritt 2: Policy festlegen, dass kein Ingress-Traffic erlaubt
## in diesem namespace: policy-demo-$KURZ
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata:
  name: default-deny
spec:
  podSelector:
    matchLabels: {}
```

```
kubectl -n policy-demo-$KURZ apply -f .
```

Schritt 3.5: Verbindung mit deny all Regeln testen

```
kubectl run --namespace=policy-demo-$KURZ access --rm -ti --image busybox
```

```
## innerhalb der shell
wget -q nginx -O -
```

Schritt 4: Zugriff erlauben von pods mit dem Label run=access (alle mit run gestarteten pods mit namen access haben dieses label per default)

```
nano 04-access-nginx.yaml
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: access-nginx
spec:
  podSelector:
    matchLabels:
      app: nginx
  ingress:
    - from:
        - podSelector:
            matchLabels:
              run: access
```

```
kubectl -n policy-demo-$KURZ apply -f .
```


Schritt 5: Testen (zugriff sollte funktionieren)

```
## lassen einen 2. pod laufen mit dem auf den nginx zugreifen
## pod hat durch run -> access automatisch das label run:access zugewiesen
kubectl run --namespace=policy-demo-$KURZ access --rm -ti --image busybox
```

```
## innerhalb der shell
wget -q nginx -O -
```

Schritt 6: Pod mit label run=no-access - da sollte es nicht gehen

```
kubectl run --namespace=policy-demo-$KURZ no-access --rm -ti --image busybox
```

```
## in der shell
wget -q nginx -O -
```

Schritt 7: Aufräumen

```
kubectl delete ns policy-demo-$KURZ
```

Ref:

- <https://projectcalico.docs.tigera.io/security/tutorials/kubernetes-policy-basic>

Warum Calico-Policies statt Standard-NetworkPolicy?

Kurzfassung

- Die Kubernetes-NetworkPolicy ist der kleinste gemeinsame Nenner: portabel ueber alle CNI-Provider (Calico, Cilium, ...), aber bewusst eingeschaenkt.
- Die Calico-Policies (projectcalico.org/v3) sind ein Superset: alles, was die Standard-Policy kann, plus cluster-weite Policies, Deny-Regeln, Reihenfolge und mehr.
- Beide lassen sich mischen - Calico wertet Standard- und Calico-Policies gemeinsam aus.

Was die Standard-NetworkPolicy kann

- Namespaced: gilt immer nur in ihrem Namespace
- Whitelist-Prinzip: sobald eine Policy auf einen Pod matcht, ist alles andere verboten - es gibt nur "Allow"-Regeln
- Selektoren: `podSelector`, `namespaceSelector`, `ipBlock`
- Ports/Protokolle: TCP, UDP, SCTP
- Wichtig: Kubernetes selbst setzt NICHTS durch - die Umsetzung macht immer der CNI-Provider (bei uns: Calico)

Grenzen der Standard-NetworkPolicy

- Kein cluster-weites default-deny mit EINEM Objekt - man braucht eine eigene Policy pro Namespace
- Keine expliziten Deny-Regeln (nur implizites Deny durch Whitelisting)
- Keine Reihenfolge/Prioritaeten zwischen Policies
- Kein Schutz der Nodes selbst (nur Pod-Traffic)
- Kein Logging von Policy-Entscheidungen
- Nur einfache Label-Gleichheit als Selektor

Was Calico zusaetzlich bietet

Feature	Standard NetworkPolicy	Calico
Geltungsbereich	nur Namespace	NetworkPolicy (Namespace) + GlobalNetworkPolicy (Cluster)
Aktionen	nur Allow (implizit)	Allow, Deny, Log, Pass
Reihenfolge	keine	<code>order</code> -Feld
Selektoren	Label-Gleichheit	Ausdruecke: <code>has()</code> , <code>in</code> , <code>!</code> , <code>==</code> , <code>all()</code>
ServiceAccounts	nein	<code>serviceAccountSelector</code>
Nodes/Hosts schuetzen	nein	<code>HostEndpoints</code> , <code>preDNAT</code> (z.B. <code>NodePorts</code>)
ICMP-Regeln	nein	ja
Policies testen	nein	Staged Policies (ab 3.29), Tiers (ab 3.30)

- Praktisch am wichtigsten fuer uns:
 - GlobalNetworkPolicy**: ein cluster-weites default-deny statt einer Kopie pro Namespace (siehe Uebung)
 - order**: definierte Auswertungsreihenfolge statt "alle Policies werden zusammengeworfen"
 - Log-Action**: sichtbar machen, WELCHE Regel Traffic verwirft

Daumenregel: Wann nehme ich was?

- Standard-NetworkPolicy**: einfache App-Isolation innerhalb eines Namespaces, oder wenn Manifests portabel bleiben sollen (z.B. Helm-Charts fuer fremde Cluster)
- Calico-Policies**: cluster-weite Grundregeln (default-deny), explizite Deny-Regeln, Compliance-Anforderungen, Node-Schutz, Policy-Debugging
- Mischen ist ueblich: Plattform-Team setzt GlobalNetworkPolicies, App-Teams schreiben Standard-Policies fuer ihre Namespaces. Calico liest die Standard-Policies direkt ein und wertet sie zusammen mit den Calico-Policies aus.

Referenzen

- <https://docs.tigera.io/calico/latest/network-policy/get-started/calico-policy/calico-network-policy>
- <https://docs.tigera.io/calico/latest/network-policy/get-started/kubernetes-policy/kubernetes-network-policy>

Calico-Policies - Grundlagen (Ordering, Implicit Deny, API-Version)

Version 3.30

- Introduced Tiers

Ordering (no order set)

- For the default deny GlobalNetworkPolicy use no order
 - it will then be evaluated as last rule (catch all)
- Ref: <https://docs.tigera.io/calico-cloud/network-policy/default-deny>

Ordering with Number for GlobalNetworkPolicy and NetworkPolicy

```
GlobalNetworkPolicies and NetworkPolicies from calico are mixed
They are all sorted by order
```

```
NetworkPolicy A order 50
GlobalNetworkPolicy B order 100
GlobalNetworkPolicy C order 70
NetworkPolicy D order 80
```

```
results in this execution order -->
```

```
NetworkPolicy A order 50
GlobalNetworkPolicy C order 70
NetworkPolicy D order 80
GlobalNetworkPolicy B order 100
```

Implicit Deny

- the selector() defines whom the NetworkPolicy or GlobalNetworkPolicy are for
- If no rules apply -> it will be an implicit deny

crd.projectcalico.org/v1 vs projectcalico.org/v3

```
Long story short: Please only use projectcalico.org/v3
They also work with kubectl (when calico apiserver is running - this is the case by default)
kubectl -n calico-system get pods | grep api
```

```
LONG STORY:
Don't touch crd.projectcalico.org/v1 resources. They are not currently supported for end-users and the entire API group is only
used internally within Calico. Using any API within that group means you will bypass API validation and defaulting, which is bad
and can result in symptoms like # 2 above. You should use projectcalico.org/v3 instead. Note that projectcalico.org/v3 requires
that you install the Calico API server in your cluster, and will result in errors similar to # 1 above if the Calico API server is
not running
```

- Ref: <https://github.com/projectcalico/calico/issues/6412>

Erweiterte Policies mit Calico - Übung

Step 1: Set global policy

```
cd
mkdir -p manifests/calico
cd manifests/calico
nano 01-gp.yml
```

- Best practice no "order" here, will be processed last then

```
apiVersion: projectcalico.org/v3
kind: GlobalNetworkPolicy
metadata:
  name: default-deny
spec:
  namespaceSelector: has(kubernetes.io/metadata.name) && kubernetes.io/metadata.name not in {"kube-system", "calico-system",
"tigera-operator"}
  types:
    - Ingress
    - Egress
  egress:
    # allow all namespaces to communicate to DNS pods
    - action: Allow
```

```
protocol: UDP
destination:
  selector: 'k8s-app == "kube-dns"'
  ports:
    - 53
- action: Allow
protocol: TCP
destination:
  selector: 'k8s-app == "kube-dns"'
  ports:
    - 53
```

```
kubectl apply -f .
```

Step 2: Namespace nptest anlegen und nginx ausrollen

```
kubectl create ns nptest
kubectl config set-context --current --namespace=nptest
```

```
cd
mkdir -p manifests/04-service
cd manifests/04-service
```

```
nano deploy.yml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-nginx
spec:
  selector:
    matchLabels:
      web: my-nginx
  replicas: 2
  template:
    metadata:
      labels:
        web: my-nginx
    spec:
      containers:
        - name: cont-nginx
          image: nginx
          ports:
            - containerPort: 80
```

```
nano service.yml
```

```
apiVersion: v1
kind: Service
metadata:
  name: svc-nginx
  labels:
    run: svc-my-nginx
spec:
  type: ClusterIP
  ports:
    - port: 80
      protocol: TCP
  selector:
    web: my-nginx
```

```
kubectl apply -f .
```

Step 3: Gleiche Applikation im Namespace fremd ausrollen

```
kubectl create ns fremd
```

```
## gleiche Applikation im anderen namespace ausrollen
kubectl -n fremd apply -f .
```

Step 4: Testen im eigenen Namespace "nptest" und zum -> anderen

```
kubectl run -it --rm access --image=busybox
```

Testen zum svc-nginx-Dienst im eigenen Namespace und google

```
## In der Busybox
## Test innerhalb meines namespaces
## bekomme die Ausgabe des Ziels nicht
wget -O - http://svc-nginx
## Google geht auch nicht
wget -O - http://www.google.de
## aber dns lookup geht
nslookup www.google.de
```

Test zum svc-nginx.fremd - Dienst - also im anderen Namespace

```
## test mit pod im **fremd** namespace
wget -O - http://svc-nginx.fremd
## --> geht auch nicht
```

Step 5: Traffic erlauben egress von busybox

```
cd ~/manifests/calico
nano 02-egress-allow-busybox.yml
```

```
apiVersion: projectcalico.org/v3
kind: NetworkPolicy
metadata:
  name: allow-busybox-egress
spec:
  order: 10
  selector: run == 'access'
  types:
    - Egress
  egress:
    - action: Allow
```

```
kubectl apply -f 02-egress-allow-busybox.yml
## cnp = calico network policy
kubectl get all,cnp
```

```
kubectl run -it --rm access --image=busybox
```

```
## sollte gehen
wget -O - http://www.google.de

## sollte nicht funktionieren
wget -O - http://svc-nginx

## sollte nicht funktionieren
## Ausgehender Traffic geht zwar, aber eingehender
## Traffic ist nicht gesetzt (auch nicht im anderen namespace)
## Hier haben wir bisher noch keine Regeln
wget -O - http://svc-nginx.fremd
```

Step 6: Traffic erlauben fuer nginx

```
nano 03-allow-ingress-my-nginx.yml
```

```
apiVersion: projectcalico.org/v3
kind: NetworkPolicy
metadata:
  name: allow-nginx-ingress
spec:
  order: 20
  # fuer welche pods soll das gelten
  selector: web == 'my-nginx'
  types:
    - Ingress
  ingress:
    - action: Allow
      source:
        selector: run == 'access'
```

```
kubectl apply -f .
```

```
kubectl run -it --rm access --image=busybox
```

```
## In der Busybox das geht ->
wget -O - http://svc-nginx
## das nicht
wget -O - http://svc-nginx.fremd
## das geht
wget -O - http://www.google.de
```

Step 7: Optional: Traffic innerhalb des Namespaces erlauben

```
## alte Regeln rausnehmen
kubectl delete -f 02-egress-allow-busybox.yml
kubectl delete -f 03-allow-ingress-my-nginx.yml
```

```
nano 04-traffic-allowed-inside-namespace.yml
```

```
apiVersion: projectcalico.org/v3
kind: NetworkPolicy
metadata:
  name: allow-intra-namespace
spec:
  order: 100
  selector: all()
  types:
    - Ingress
    - Egress
  ingress:
    - action: Allow
      source:
        selector: all()
  egress:
    - action: Allow
      destination:
        selector: all()
```

```
kubectl apply -f 04-traffic-allowed-inside-namespace.yml
```

```
kubectl run -it --rm access --image=busybox
```

```
## In der Busybox das geht ->
wget -O - http://svc-nginx
## das nicht
wget -O - http://svc-nginx.fremd
## das geht auch nicht
wget -O - http://www.google.de
```

Step 8 (Optional): Traffic vom Ingress Controller erlauben

- Nur relevant, wenn ein Ingress Controller (z.B. ingress-nginx) installiert ist

```
nano 05-allow-ingress-controller-to-nginx.yml
```

```
apiVersion: projectcalico.org/v3
kind: NetworkPolicy
metadata:
  name: allow-ingress-controller-to-nginx
spec:
  order: 30
  selector: web == 'my-nginx'
  types:
    - Ingress
  ingress:
    - action: Allow
      protocol: TCP
      source:
        namespaceSelector: kubernetes.io/metadata.name == "ingress-nginx"
      destination:
        ports: [80, 443]
```

```
kubectl apply -f 05-allow-ingress-controller-to-nginx.yml
```

Aufraeumen

```
kubectl delete gnp default-deny
kubectl delete ns fremd nptest
kubectl config set-context --current --namespace=default
```

RBAC & Identity

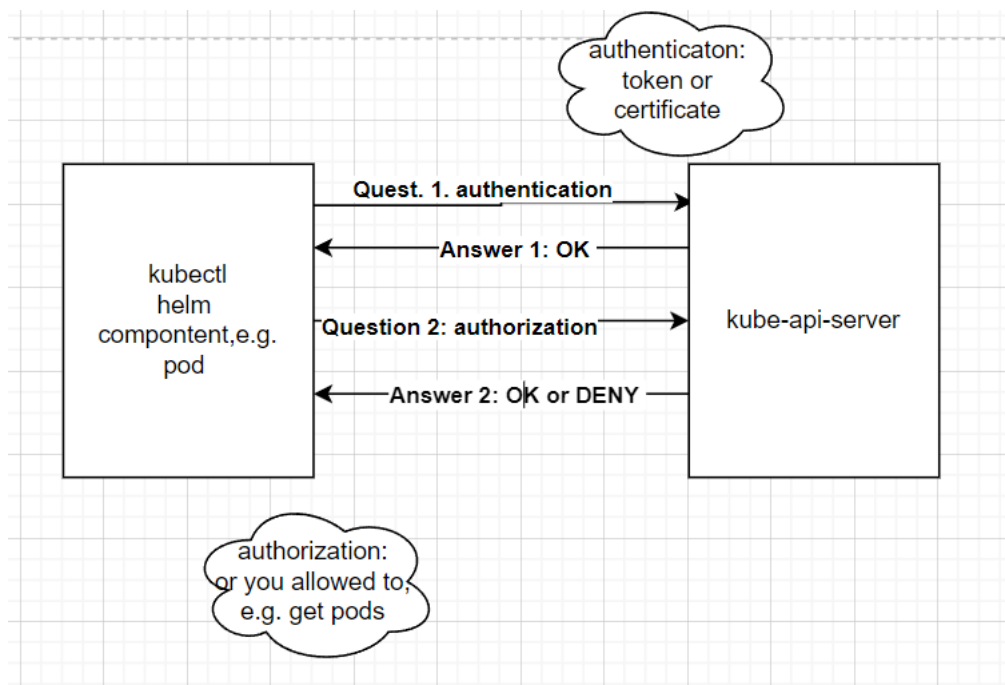
Least Privileges mit RBAC

The least privileges principles

- Always design your pods, user and components, that they really only have the minimal principles they need
- RBAC Resources help you to do that (ServiceAccounts, Roles, ClusterRoles, Rolebinding, Clusterrolebindings, Groups)

Wie funktioniert RBAC?

- Let us see in a picture



Wo spielt RBAC eine Rolle?

Users -> kube-api-server

- User how want to access the kube api server

Components -> kube-api-server

- e.g. kubelet -> kube-api-server

Pods / System Pods -> kube-api-server

- Pods and System Pods (e.g. kube-proxy a.k.a. CoreDNS) how want to access the kube-api-server

kubectl - Berechtigungen pruefen mit can-i

A specific command

```
kubectl auth can-i get pods
```

List all

```
kubectl auth can-i --list
```

Fuer einen anderen Nutzer (z.B. ServiceAccount)

```
kubectl auth can-i --list --as system:serviceaccount:default:training
```

Praktisch kombinierbar mit der [praktischen RBAC-Uebung](#): zeigt alle erlaubten Verben/Ressourcen des ServiceAccount auf einen Blick, statt jeden Befehl einzeln mit `can-i get ...` durchzutesten.

ServiceAccounts: kubectl im Pod - default ServiceAccount

Walkthrough

```
kubectl run -it --rm kubectltester --image=alpine -- sh
```

```
## in shell
apk add kubectl
## it uses in in-cluster configuration in folder
## /var/run/secrets/kubernetes.io/serviceaccount
kubectl auth can-i --list
```

ServiceAccounts: Automount - ja oder nein?

Why ?

- Every attacker tries to get as much information as possible
- Although there are not severe permissions in here, show as little information as possible
- For example, use will see, which namespace he is in ;o)

Disable ?

```
## enabled by default
kubectl explain pod.spec.automountServiceAccountToken
```

Praktische Uebung: User mit Zertifikat anlegen (kubeconfig)

Step 0: create an new rolebinding for the group (we want to use)

```
kubectl create rolebinding developers --clusterrole=view --group=developers
```

Step 1: on your client: create private certificate

```
cd
mkdir -p certs
## create your private key
openssl genrsa -out ~/certs/jochen.key 4096
```

Step 2: on your client: create csr (certificate signing request)

```
nano ~/certs/jochen.csr.cnf
```

```
[ req ]
default_bits = 2048
prompt = no
default_md = sha256
distinguished_name = dn
[ dn ]
CN = jochen
O = developers
[ v3_ext ]
authorityKeyIdentifier=keyid,issuer:always
basicConstraints=CA:FALSE
keyUsage=keyEncipherment,dataEncipherment
extendedKeyUsage=serverAuth,clientAuth
```

```
## Create Certificate Signing Request
openssl req -config ~/certs/jochen.csr.cnf -new -key ~/certs/jochen.key -nodes -out ~/certs/jochen.csr
openssl req -in certs/jochen.csr --noout -text
```

Step 3: Send approval request to server

```
## get csr (base64 decoded)
cat ~/certs/jochen.csr | base64 | tr -d '\n'
```

```
cd certs
nano jochen-csr.yaml
```

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: jochen-authentication
spec:
  signerName: kubernetes.io/kube-apiserver-client
  groups:
    - system:authenticated
  request:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURSBBSRVFVRVNL$0tLS0KTU1JRWF6Q0NBbE1QVFBd0pqRVBNQT$BHQTFVRUF3d0d$hbTlqYUdWdU1$STXdFUVlEVlF$R50RBcGtaWFpsYkc5
```

```
usages:
- client auth
```

```
kubectl apply -f jochen-csr.yaml
kubectl get -f jochen-csr.yaml
## show me the current state -> pending
kubectl describe -f jochen-csr.yaml
```

Step 4: approve signing request

```
kubectl certificate approve jochen-authentication
## or:
kubectl certificate approve -f jochen-csr.yaml

## see, that it is approved
kubectl describe -f jochen-csr.yaml
```

Step 5: get the approved certificate to be used

```
kubectl get csr jochen-authentication -o jsonpath='{.status.certificate}' | base64 --decode > ~/certs/jochen.crt
```

Step 6: construct kubeconfig for new user

```
cd
cd certs
```

```
## create new user
kubectl config set-credentials jochen --client-certificate=jochen.crt --client-key=jochen.key
```

```
## add a new context
kubectl config set-context jochen --user=jochen --cluster=kubernetes
```

Step 7: Use and test the new context

```
kubectl config use-context jochen
kubectl get pods
```

Ref:

- <https://kb.leaseweb.com/kb/users-roles-and-permissions-on-kubernetes-rbac/kubernetes-users-roles-and-permissions-on-kubernetes-rbac-create-a-certificate-based-kubeconfig/>

Praktische Uebung RBAC (ab Kubernetes 1.25)

Schritt 1: Nutzer-Account auf Server anlegen und secret anlegen / in Client

```
cd
mkdir -p manifests/rbac
cd manifests/rbac
```

Mini-Schritt 1: Definition für Nutzer

```
nano 01-service-account.yml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: training
  namespace: default
```

```
kubectl apply -f .
```

Mini-Schritt 1.5: Secret erstellen

- From Kubernetes 1.25 tokens are not created automatically when creating a service account (sa)
- You have to create them manually with annotation attached
- <https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/#create-token>

```
## vi 02-secret.yml
apiVersion: v1
kind: Secret
type: kubernetes.io/service-account-token
metadata:
  name: trainingtoken
  namespace: default
```



```
  annotations:
    kubernetes.io/service-account.name: training
```

```
kubectl apply -f .
```

Mini-Schritt 2: ClusterRolle festlegen - Dies gilt für alle namespaces, muss aber noch zugewiesen werden

```
nano 03-pods-clusterrole.yml
```

```
### Bevor sie zugewiesen ist, funktioniert sie nicht - da sie keinem Nutzer zugewiesen ist
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pods-clusterrole
rules:
- apiGroups: [""] # "" indicates the core API group
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

```
kubectl apply -f .
```

Mini-Schritt 3: Die ClusterRolle den entsprechenden Nutzern über RoleBinding zu ordnen

```
## vi 04-rb-training-ns-default-pods.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: rolebinding-ns-default-pods
  namespace: default
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: pods-clusterrole
subjects:
- kind: ServiceAccount
  name: training
  namespace: default
```

```
kubectl apply -f .
```

Mini-Schritt 4: Testen (klappt der Zugang)

```
kubectl auth can-i get pods -n default --as system:serviceaccount:default:training
## yes
kubectl auth can-i get deployment -n default --as system:serviceaccount:default:training
## no
kubectl auth can-i --list --as system:serviceaccount:default:training
```

Schritt 2: Context anlegen / Credentials auslesen und in kubeconfig hinterlegen (bis Version 1.25.)

Mini-Schritt 1: kubeconfig setzen

```
kubectl config set-context training-ctx --cluster kubernetes --user training

## extract name of the token from here

TOKEN=`kubectl get secret trainingtoken -o jsonpath='{.data.token}' | base64 --decode`
echo $TOKEN
kubectl config set-credentials training --token=$TOKEN
kubectl config use-context training-ctx

## Hier reichen die Rechte nicht aus
kubectl get deploy
## Error from server (Forbidden): pods is forbidden: User "system:serviceaccount:kube-system:training" cannot list # resource
"pods" in API group "" in the namespace "default"
```

Mini-Schritt 2:

```
kubectl config use-context training-ctx
kubectl get pods
```

Mini-Schritt 3: Zurück zum alten Default-Context

```
kubectl config get-contexts
```

CURRENT	NAME	CLUSTER	AUTHINFO	NAMESPACE
	kubernetes-admin@kubernetes	kubernetes	kubernetes-admin	
*	training-ctx	kubernetes	training	

```
kubectrl config use-context kubernetes-admin@kubernetes
```

Refs:

- <https://docs.oracle.com/en-us/iaas/Content/ContEng/Tasks/contengaddingserviceaccttoken.htm>
- <https://microk8s.io/docs/multi-user>
- <https://faun.pub/kubernetes-rbac-use-one-role-in-multiple-namespaces-d1d08bb08286>

Ref: Create Service Account Token

- <https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/#create-token>

Praktische Uebung: RBAC-Hygiene - Label-Konvention pruefen und Nutzung im Audit-Log nachweisen

Getestet gegen Kubernetes 1.37 (kubeadm, eigenes Cluster).

Die [Uebung zu Least Privilege](#) zeigt, wie man overprivilegierte ServiceAccounts findet. Diese Uebung geht einen Schritt weiter: Kubernetes speichert nirgends, wann eine Rolle, ClusterRole oder ein RoleBinding zuletzt tatsaechlich benutzt wurde - kein Feld, keine API. Wir bauen deshalb eine Label-Konvention (owner, purpose, review-by), pruefen sie automatisiert, und zeigen anschliessend, wie man ueber das Audit-Log des kube-apiserver einen echten Nutzungsnachweis bekommt - und wo dessen Grenzen liegen.

Voraussetzungen

- Eigenes kubeadm-Cluster mit kubectrl -Zugriff (Cluster-Admin-Kubeconfig)
- jq installiert

Ueberblick

```
Schritt 1: Namespace vorbereiten
Schritt 2: Drei ServiceAccounts + RoleBindings anlegen (gepflegt / unbeschriftet / abgelaufen)
Schritt 3: Audit-Query 1 - fehlende Pflicht-Labels finden
Schritt 4: Audit-Query 2 - abgelaufene review-by-Termine finden
Schritt 5: Audit-Log am kube-apiserver aktivieren
Schritt 6: Echten Zugriff erzeugen und im Audit-Log nachweisen
Schritt 7: Gegenprobe - unbenutzte Bindung im Audit-Log nicht nachweisbar
Schritt 8: Aufräumen
```

Schritt 1: Namespace vorbereiten

```
kubectrl create namespace rbac-hygiene
```

Schritt 2: Drei RBAC-Identitaeten anlegen

Ein sauber gepflegter ServiceAccount (owner, purpose, review-by in der Zukunft), ein komplett unbeschrifteter (Gegenbeispiel), und einer mit abgelaufenem review-by :

```
## vi 01-rbac-hygiene.yml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: rbac-hygiene
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: ci-deploy-payment-api
  namespace: rbac-hygiene
  labels:
    owner: team-payment
    purpose: ci-deploy-payment-api
    review-by: "2026-12-09"
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: ci-deploy-payment-api-binding
  namespace: rbac-hygiene
  labels:
    owner: team-payment
    purpose: ci-deploy-payment-api
    review-by: "2026-12-09"
```

```

subjects:
- kind: ServiceAccount
  name: ci-deploy-payment-api
  namespace: rbac-hygiene
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: batch-job-alt
  namespace: rbac-hygiene
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: batch-job-alt-binding
  namespace: rbac-hygiene
subjects:
- kind: ServiceAccount
  name: batch-job-alt
  namespace: rbac-hygiene
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: legacy-export
  namespace: rbac-hygiene
  labels:
    owner: team-data
    purpose: nightly-export-legacy-crm
    review-by: "2026-08-11"
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: legacy-export-binding
  namespace: rbac-hygiene
  labels:
    owner: team-data
    purpose: nightly-export-legacy-crm
    review-by: "2026-08-11"
subjects:
- kind: ServiceAccount
  name: legacy-export
  namespace: rbac-hygiene
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

```
kubectl apply -f 01-rbac-hygiene.yml
```

`ci-deploy-payment-api` ist der Normalfall: klarer Owner, klarer Zweck, ein Termin in der Zukunft, an dem jemand pruefen muss, ob es das noch braucht. `batch-job-alt` ist der haeufigste Fall in echten Clustern: irgendwann angelegt, nie beschriftet. `legacy-export` ist beschriftet, aber der Review-Termin ist laengst verstrichen - niemand hat reagiert.

Schritt 3: Fehlende Pflicht-Labels finden

```

kubectl get serviceaccounts,rolebindings -n rbac-hygiene -o json | jq -r '
.items[] |
  select(.metadata.name != "default") |
  select((.metadata.labels.owner == null) or (.metadata.labels.purpose == null) or (.metadata.labels["review-by"] == null)) |
  "\(.kind)/\(.metadata.name): fehlende Pflicht-Labels"
'

```

Erwartete Ausgabe:

```

ServiceAccount/batch-job-alt: fehlende Pflicht-Labels
RoleBinding/batch-job-alt-binding: fehlende Pflicht-Labels

```

Schritt 4: Abgelaufene review-by-Termine finden

```
kubectl get serviceaccounts,rolebindings -n rbac-hygiene -o json | jq -r --arg today "$(date +%F)" '
.items[] |
select (.metadata.labels["review-by"] != null) |
select (.metadata.labels["review-by"] < $today) |
"\(.kind)/\(.metadata.name): review-by \(.metadata.labels["review-by"]) liegt in der Vergangenheit"
,
```

Erwartete Ausgabe:

```
ServiceAccount/legacy-export: review-by 2026-08-11 liegt in der Vergangenheit
RoleBinding/legacy-export-binding: review-by 2026-08-11 liegt in der Vergangenheit
```

Wichtig: Diese beiden Queries pruefen nur, ob die Konvention eingehalten wird und ob ein Review faellig ist - nicht, ob die Berechtigung tatsaechlich noch gebraucht wird. Das kann Kubernetes ohne Weiteres gar nicht beantworten. Genau das zeigen die naechsten Schritte.

Schritt 5: Audit-Log am kube-apiserver aktivieren

Echte Nutzungsdaten liefert ausschliesslich das Audit-Log des kube-apiserver - nicht Kubelet- oder Anwendungs-Logs. Auf einem kubeadm-Cluster ist der kube-apiserver ein Static Pod, dessen Manifest direkt auf dem Control-Plane-Node liegt. Wir editieren es ueber `kubectl debug node` - kein SSH-Zugriff auf den Node noetig, nur die Cluster-Admin-Kubeconfig:

```
CP_NODE=$(kubectl get nodes -l node-role.kubernetes.io/control-plane -o jsonpath='{.items[0].metadata.name}')
kubectl debug node/$CP_NODE -it --image=alpine:3.20 --profile=general -- chroot /host sh
```

Im Node-Shell (`chroot /host`):

```
cp /etc/kubernetes/manifests/kube-apiserver.yaml /etc/kubernetes/kube-apiserver.yaml.orig
mkdir -p /var/log/kubernetes/audit

cat > /etc/kubernetes/audit-policy.yaml <<'EOF'
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
EOF

sed -i '/--authorization-mode=Node,RBAC/a\    --audit-policy-file=/etc/kubernetes/audit-policy.yaml\n    --audit-log-path=/var/log/kubernetes/audit/audit.log\n    --audit-log-maxage=1\n    --audit-log-maxbackup=1'
/etc/kubernetes/manifests/kube-apiserver.yaml

sed -i '/^  hostNetwork: true/i\    - mountPath: /etc/kubernetes/audit-policy.yaml\n      name: audit-policy\n      readOnly: true\n    - mountPath: /var/log/kubernetes/audit\n      name: audit-log' /etc/kubernetes/manifests/kube-apiserver.yaml

sed -i '/^status: {}/i\    - hostPath:\n      path: /etc/kubernetes/audit-policy.yaml\n      type: File\n      name: audit-policy\n    - hostPath:\n      path: /var/log/kubernetes/audit\n      type: DirectoryOrCreate\n      name: audit-log'
/etc/kubernetes/manifests/kube-apiserver.yaml

exit
```

Der Kubelet erkennt die Manifest-Aenderung und startet den kube-apiserver-Pod neu. `kubectl` haengt dabei fuer ca. 30-60 Sekunden (`connection refused`) - das ist erwartet, kein Fehler. Warten und erneut versuchen:

```
kubectl get nodes
```

Sobald die Nodeliste wieder kommt, laeuft der apiserver mit Audit-Log.

Schritt 6: Echten Zugriff erzeugen und nachweisen

Wir tun so, als waere `ci-deploy-payment-api` ein Pod, der ueber seinen ServiceAccount-Token auf Pods zugreift:

```
kubectl get pods -n rbac-hygiene --as=system:serviceaccount:rbac-hygiene:ci-deploy-payment-api
```

Jetzt im Audit-Log nachsehen, ob und wodurch dieser Zugriff erlaubt wurde:

```
CP_NODE=$(kubectl get nodes -l node-role.kubernetes.io/control-plane -o jsonpath='{.items[0].metadata.name}')
kubectl debug node/$CP_NODE --image=alpine:3.20 --profile=general -- chroot /host sh -c \
'grep ci-deploy-payment-api-binding /var/log/kubernetes/audit/audit.log | tail -1'
```

Erwartete Ausgabe (Auszug):

```
"authorization.k8s.io/reason": "RBAC: allowed by RoleBinding \"ci-deploy-payment-api-binding/rbac-hygiene\" of Role \"pod-reader\" to ServiceAccount \"ci-deploy-payment-api/rbac-hygiene\""
```

Das ist der einzige Weg, mit dem Kubernetes selbst belegt: Diese Bindung wurde tatsaechlich gezogen, nicht nur vergeben. Der RBAC-Authorizer schreibt bei jeder erlaubten Anfrage genau diese `authorization.k8s.io/reason` -Annotation.

Schritt 7: Gegenprobe - keine Nutzung nachweisbar

```
CP_NODE=$(kubectl get nodes -l node-role.kubernetes.io/control-plane -o jsonpath='{.items[0].metadata.name}')
kubectl debug node/$CP_NODE --image=alpine:3.20 --profile=general -- chroot /host sh -c \
'grep -c legacy-export-binding /var/log/kubernetes/audit/audit.log'
```

Erwartete Ausgabe:

```
0
```

legacy-export-binding taucht im Audit-Log nicht auf - seit das Log laeuft, hat niemand darueber zugegriffen. Das ist ein starkes Indiz, aber **kein Beweis**: Es belegt nur den beobachteten Zeitraum. Ein Quartalsjob, der einmal alle drei Monate laeuft, waere im Log genauso unsichtbar, obwohl er gebraucht wird. Deshalb bleibt die Entscheidung am review-by -Termin: abgelaufenes Label + keine Spur im Audit-Log seit der letzten Pruefung = starker Kandidat zum Loeschen, kein Automatismus.

Schritt 8: Aufräumen

Audit-Log-Konfiguration wieder vom Node entfernen (sonst waechst die Datei unbegrenzt weiter):

```
CP_NODE=$(kubectl get nodes -l node-role.kubernetes.io/control-plane -o jsonpath='{.items[0].metadata.name}')
kubectl debug node/$CP_NODE -it --image=alpine:3.20 --profile=general -- chroot /host sh
```

Im Node-Shell (die Sicherung aus Schritt 5 zurueckspielen, statt die Aenderungen einzeln per sed zu suchen):

```
cp /etc/kubernetes/kube-apiserver.yaml.orig /etc/kubernetes/manifests/kube-apiserver.yaml
rm -f /etc/kubernetes/kube-apiserver.yaml.orig /etc/kubernetes/audit-policy.yaml
rm -rf /var/log/kubernetes/audit
exit
```

Namespace loeschen:

```
kubectl delete namespace rbac-hygiene
```

Zusammenfassung

Frage	Beantwortet durch	Beweist
Sind Owner/Zweck/Review-Termin gepflegt?	Label-Query (Schritt 3)	Compliance mit der Konvention
Ist ein Review faellig?	review-by-Query (Schritt 4)	Faelligkeit, nicht Nutzung
Wurde die Berechtigung tatsaechlich gezogen?	Audit-Log-Reason (Schritt 6)	Echte Nutzung im beobachteten Zeitraum
Wird sie nicht mehr gebraucht?	Kein Tool zuverlaessig	- immer eine Team-Entscheidung am review-by-Termin

Referenzen

- <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/>
- <https://kubernetes.io/docs/reference/labels-annotations-taints/audit-annotations/>
- <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>

Secrets Management mit HashiCorp Vault

Vault-Architektur einfach erklart

Die Idee: ein Tresorraum fuer Geheimnisse

Stell dir Vault wie den Tresorraum einer Bank vor. Statt Goldbarren liegen darin Geheimnisse: Passwoerter, Datenbank-Zugaenge, Zertifikate. Niemand schreibt diese Geheimnisse mehr in den Programmcode oder in Konfigurationsdateien - wer eines braucht, geht zum Tresor und fragt danach.



Die 4 Schritte

- Ausweis-Kontrolle (Authentifizierung):** Deine App meldet sich an und beweist, wer sie ist. Dafuer bekommt sie einen Ausweis - den **Token**.
- Regel-Check (Policy):** Vault schaut in seine Regelliste: Was darf dieser Ausweis sehen? Die App bekommt nur genau die Geheimnisse, die fuer sie erlaubt sind - nicht mehr.
- Schublade oeffnen (Secret Engine):** Jede Art von Geheimnis liegt in einer eigenen Schublade. Manche Schubladen geben gespeicherte Passwoerter heraus, andere erzeugen sogar frische Zugangsdaten, die nach kurzer Zeit automatisch wieder ungueltig werden.
- Antwort:** Die App bekommt ihr Geheimnis und kann damit z.B. auf die Datenbank zugreifen.

Die wichtigsten Begriffe uebersetzt

Fachbegriff	Einfach gesagt
Token	Ausweis, den man nach dem Anmelden bekommt
Policy	Regelliste: wer darf was sehen
Secret Engine	Schublade fuer eine bestimmte Art von Geheimnis

Sealed / Unseal	Tresor verriegelt / Tresor aufschliessen
Audit Log	Besucherbuch: wer hat wann welches Geheimnis geholt

Warum der Aufwand?

- Passwoerter stehen nicht mehr im Code oder in Git - dort werden sie am haeufigsten gestohlen.
- Alle Geheimnisse liegen an EINEM zentralen Ort und sind dort verschluesselt.
- Jeder Zugriff wird protokolliert - man sieht, wer wann was geholt hat.
- Geheimnisse lassen sich zentral austauschen oder sperren, ohne dass irgendwo Code angefasst werden muss.

HashiCorp Vault als Password-Safe (Overview)

Zentrale Externer Server mit 3 Nodes (Produktion)

3-Wege für Kubernetes Daten zu bekommen

- VSO (Vault Secrets Operator)
- SideCar Injection
- Volumes

VSO

- Ich bestücke eine neue CRT mit dem Wunsch eines Credentials "Vault Static Secret"

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultStaticSecret
metadata:
  name: webapp-config
  namespace: default
spec:
  # Reference to VaultAuth in another namespace
  vaultAuthRef: vault-secrets-operator-system/default

  # Vault mount path (where the secret engine is mounted)
  mount: secret

  # Path to the secret within the mount
  path: webapp/config

  # Type of secret engine
  type: kv-v2

  # Destination Kubernetes secret configuration
  destination:
    create: true
    name: webapp-secret
    type: Opaque

  # How often to refresh the secret from Vault
  refreshAfter: 30s
```

Nachteil

- Das automatisch erstellte Secret wird in etc gespeichert, solange wie das VaultStaticSecret existiert

Vault Sidecar Injector

Vorteile

- Sicherste Variante
- Es wird kein Secret erstellt, passwort wird direkt im Pod zur Verfügung gestellt (in einer Datei)

Nachteile

- Relativ viele Einträge im Pod über Annotations zu machen, damit das funktioniert
- Overhead über SideCar (weil jeder Pod ein Sidecar bekommt)
- Bekommt mit, wenn sich das Passwort ändert

Volumes

Uebung: MariaDB-Deployment mit HashiCorp Vault ueber den Vault Secrets Operator (VSO)

Hintergrund

Der Vault Secrets Operator (VSO) synchronisiert Secrets aus HashiCorp Vault als native Kubernetes Secrets in dein Cluster. Ein Controller im Cluster loggt sich bei Vault ein, liest das Secret periodisch neu und legt es als `Secret`-Ressource ab - Anwendungen greifen ganz normal per `secretKeyRef` darauf zu, ohne selbst etwas von Vault zu wissen.



VSO Datenfluss

Setup: Alle Teilnehmer nutzen den **gleichen Vault-Server** (`https://vault-bka.do.t3isp.de`), aber jeder arbeitet mit seinem **eigenen Kubernetes-Cluster**. Das **MariaDB-Credential** (`username / password` unter `secret/mariadb`) ist bei allen Teilnehmern **identisch** - es ist ein Trainings-Demo-Wert, kein echtes Secret. Der **Kubernetes-Auth-Mount** (`kubernetes-<dein-tn>`) ist dagegen pro Teilnehmer eigenstaendig, weil er auf dein Cluster zeigt.

Voraussetzungen

- Eigenes kubernetes-Cluster (`kubectl get nodes` funktioniert)
- Helm v3 installiert
- Dein Cluster muss `vault-bka.do.t3isp.de` per HTTPS erreichen koennen
- Der Trainer hat fuer dein `<tln>` bereits einen Kubernetes-Auth-Mount in Vault eingerichtet (Rolle `mariadb` , gebunden an ServiceAccount `mariadb-sa` im Namespace `default`) - siehe Hintergrund unten

Hintergrund: Was der Trainer fuer dich schon eingerichtet hat

Vault muss deinem Cluster vertrauen koennen, bevor irgendein Pod sich dort einloggen darf. Das ist zweistufig aufgebaut - beide Stufen hat der Trainer per Skript (`training-vault-server`) bereits fuer dich erledigt, du musst sie nicht selbst anlegen, solltest aber verstehen, was da steht:

1. **In deinem Cluster:** Ein ServiceAccount `vault-auth` mit einem `ClusterRoleBinding` auf `system:auth-delegator` . Vault validiert jeden eingehenden Login-Versuch ueber die Kubernetes-TokenReview-API - da fuer braucht Vault selbst einen Token mit genau dieser Berechtigung. Dieser ServiceAccount ist NICHT der, mit dem sich MariaDB spaeter einloggt (das macht `mariadb-sa` , siehe Schritt 4) - er ist reine Vault-Infrastruktur, einmalig pro Cluster.
2. **Auf dem Vault-Server:** Ein eigener Kubernetes-Auth-Mount `kubernetes-<dein-tln>` (jeder Teilnehmer bekommt einen eigenen, weil ein Mount fest an EINEN API-Server + dessen CA-Zertifikat + den Reviewer-Token aus Schritt 1 gebunden ist - ein gemeinsamer Mount fuer alle Cluster ist technisch nicht moeglich). Darin ist eine Rolle `mariadb` konfiguriert, die zwei Bedingungen prueft: Der einloggende Pod muss den ServiceAccount `mariadb-sa` im Namespace `default` benutzen - und wenn das stimmt, bekommt er ein Vault-Token mit der Policy `mariadb-read` (liest ausschliesslich `secret/data/mariadb` , sonst nichts).

Kurz: Schritt 1 sagt Vault "ich kann Tokens aus diesem Cluster pruefen", Schritt 2 sagt Vault "und genau dieser ServiceAccount-Name in diesem Cluster darf dann das MariaDB-Secret lesen". Deine Aufgabe in dieser Uebung ist nur noch, den passenden ServiceAccount (`mariadb-sa`) anzulegen und die K8s-seitigen Ressourcen (VaultConnection/VaultAuth/ VaultStaticSecret) zu erstellen, die diesen Mount tatsaechlich benutzen.

Schritt 1: Vorschau - was steht ueberhaupt in Vault?

Bevor wir irgendetwas automatisieren, schauen wir uns das Secret einmal direkt per CLI an - auf `client-bka` ist `vault` bereits installiert.

Die Demo-Passwoerter des Trainings liegen auf `client-bka` zentral in `/etc/training-vault.env` - einmal sourcen, dann stehen sie als Variablen bereit und muessen nirgends im Klartext getippt werden:

```
source /etc/training-vault.env
export VAULT_ADDR=https://vault-bka.do.t3isp.de
vault login -method=userpass username=training password="$VAULT_TRAINING_PASSWORD"
```

Erwartete Ausgabe (Auszug):

```
Success! You are now authenticated.
...
policies                ["default" "mariadb-read"]
```

Das Secret lesen:

```
vault kv get secret/mariadb
```

Erwartete Ausgabe (Auszug):

```
===== Data =====
Key      Value
---      -
password <das-mariadb-demo-passwort>
username root
```

Genau dieses `username / password`-Paar liefern wir jetzt automatisiert an einen MariaDB-Pod aus - einmal per VSO (diese Uebung), einmal per Vault Agent Injector (naechste Uebung).

Schritt 2: Arbeitsverzeichnis anlegen

```
mkdir -p ~/manifests/vault-vso ~/helm-values/vault-vso
cd ~/manifests/vault-vso
```

Schritt 3: Vault Secrets Operator installieren

```
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update hashicorp
```

```
nano ~/helm-values/vault-vso/vault-secrets-operator-values.yml
```

```
defaultVaultConnection:
  enabled: false
controller:
  manager:
    clientCache:
      persistenceModel: none
```

```
helm install vault-secrets-operator hashicorp/vault-secrets-operator \
  -n vault-secrets-operator --create-namespace \
```

```
-f ~/helm-values/vault-vso/vault-secrets-operator-values.yml
```

Pruefen, ob der Operator laeuft:

```
kubectl -n vault-secrets-operator rollout status deploy/vault-secrets-operator-controller-manager
```

Erwartete Ausgabe:

```
deployment "vault-secrets-operator-controller-manager" successfully rolled out
```

Schritt 4: ServiceAccount fuer MariaDB anlegen

```
nano 00-mariadb-sa.yml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: mariadb-sa
```

```
kubectl apply -f 00-mariadb-sa.yml -n default
```

Schritt 5: VaultConnection erstellen

Die VaultConnection sagt dem Operator, mit welchem Vault-Server er reden soll.

```
nano 01-vault-connection.yml
```

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultConnection
metadata:
  name: vault-connection
spec:
  address: https://vault-bka.do.t3isp.de
```

```
kubectl apply -f 01-vault-connection.yml -n default
```

Schritt 6: VaultAuth erstellen

Die VaultAuth sagt dem Operator, WIE er sich bei Vault einloggen soll - die beiden Felder `mount` und `role` bedeuten dabei:

 VaultAuth: mount und role

```
nano 02-vault-auth.yml
```

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultAuth
metadata:
  name: vault-auth
spec:
  vaultConnectionRef: vault-connection
  method: kubernetes
  mount: kubernetes-tln1
  kubernetes:
    role: mariadb
    serviceAccount: mariadb-sa
```

Wichtig: `mount: kubernetes-tln1` auf deinen eigenen Teilnehmernamen anpassen (z.B. `kubernetes-tln4`)!

```
kubectl apply -f 02-vault-auth.yml -n default
```

Schritt 7: VaultStaticSecret erstellen

Das VaultStaticSecret definiert, welcher Pfad in Vault gelesen wird und wie das resultierende Kubernetes Secret heissen soll.

```
nano 03-vault-static-secret.yml
```

```
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultStaticSecret
metadata:
  name: mariadb-secret
spec:
  vaultAuthRef: vault-auth
  mount: secret
  type: kv-v2
```



```
path: mariadb
refreshAfter: 60s
destination:
  name: mariadb-vault-secret
  create: true
```

```
kubectl apply -f 03-vault-static-secret.yml -n default
```

Pruefen, ob das Secret synchronisiert wurde:

```
kubectl get vaultstaticsecret mariadb-secret -n default
```

Erwartete Ausgabe:

NAME	SYNCED	HEALTHY	READY	AGE
mariadb-secret	True	True	True	14s

Die entstandenen Keys im Kubernetes Secret pruefen (ohne Werte auszugeben):

```
kubectl describe secret mariadb-vault-secret -n default
```

Erwartete Ausgabe (Auszug):

```
Data
====
_raw:      197 bytes
password:  22 bytes
username:   4 bytes
```

Hinweis: `_raw` enthaelt das komplette Secret als JSON, `password` und `username` sind die einzelnen Felder aus Vault.

Schritt 8: MariaDB per Helm ausrollen

Wir nutzen den `cloudpirates/mariadb` Chart und referenzieren das eben erstellte Kubernetes Secret als Root-Passwort-Quelle.

```
nano ~/helm-values/vault-vso/mariadb-values.yml
```

```
auth:
  existingSecret: mariadb-vault-secret
  secretKeys:
    rootPasswordKey: password
persistence:
  enabled: false
```

`persistence.enabled: false` - dieses Training hat (noch) keine StorageClass fuer die kubeadm-Cluster eingerichtet, siehe README.

```
helm install mariadb-vso oci://registry-1.docker.io/cloudpirates/mariadb \
-n default \
-f ~/helm-values/vault-vso/mariadb-values.yml
```

```
kubectl -n default rollout status statefulset/mariadb-vso
```

Schritt 9: Verifikation - Login-Test

```
source /etc/training-vault.env
kubectl exec -n default mariadb-vso-0 -- mariadb -uroot -p"$MARIADB_ROOT_PASSWORD" -e "SELECT 1 AS login_test;"
```

Erwartete Ausgabe:

```
login_test
1
```

Das Passwort kam nicht aus einem `kubectl apply` -Manifest, sondern wurde vom Operator live aus Vault gezogen - `refreshAfter: 60s` sorgt dafuer, dass eine Aenderung in Vault spaetestens nach 60 Sekunden im Kubernetes Secret ankommt.

Aber: MariaDB merkt davon nichts. Der Container liest das Passwort nur einmal beim Start als Umgebungsvariable - eine Rotation erreicht die App erst mit einem Neustart (siehe Bonus in Schritt 10).

Schritt 10 (Bonus): Rotation automatisch bis in den Pod

VSO hat fuer den fehlenden Neustart ein Bordmittel eingebaut (das uebernimmt hier die Aufgabe, fuer die man sonst Tools wie Stakater Reloader installiert):

`rolloutRestartTargets`. Damit sagst du dem Operator: "Immer wenn du dieses Secret neu schreibst, starte auch das StatefulSet neu durch."

Ergaenze in `03-vault-static-secret.yml` unter `spec`: drei Zeilen:

```
## vi 03-vault-static-secret.yml
apiVersion: secrets.hashicorp.com/v1beta1
```

```
kind: VaultStaticSecret
metadata:
  name: mariadb-secret
spec:
  vaultAuthRef: vault-auth
  mount: secret
  type: kv-v2
  path: mariadb
  refreshAfter: 60s
  destination:
    name: mariadb-vault-secret
    create: true
  rolloutRestartTargets:
    - kind: StatefulSet
      name: mariadb-vso
```

```
kubectl apply -f 03-vault-static-secret.yml -n default
```

Die Kette laeuft ab jetzt automatisch durch:

1. Passwort in Vault aendern (macht der Trainer zentral - das Secret ist fuer alle Teilnehmer dasselbe, und eure Policy `mariadb-read` darf nur lesen)
2. VSO schreibt das Kubernetes Secret neu (spaaetestens nach 60s)
3. VSO macht einen Rolling Restart des StatefulSets - erkennbar an der Annotation `vso.secrets.hashicorp.com/restartedAt` und einem frischen Pod
4. Der neue Pod liest die Umgebungsvariable frisch und kennt das neue Passwort

Trainer-Hinweis - so wird zentral rotiert: Auf dem Vault-Server mit Admin-Token anmelden und `vault kv put secret/mariadb username=root password='<neuer-wert>'` ausfuehren - bei allen Teilnehmern folgen Secret-Update und Auto-Restart. Danach auf demselben Weg den Demo-Wert aus `/etc/training-vault.env` wiederherstellen.

Beobachten, waehrend der Trainer rotiert:

```
kubectl get pods -n default -w
```

Erwartete Ausgabe (nach ca. einer Minute):

```
mariadb-vso-0 1/1 Running 0 25m
mariadb-vso-0 1/1 Terminating 0 26m
mariadb-vso-0 0/1 Pending 0 0s
mariadb-vso-0 1/1 Running 0 31s
```

Wichtig: Das loest das eigentliche Problem noch nicht. Der Neustart bringt das neue Passwort nur bis zur App - in den Systemtabellen der Datenbank rotiert dabei nichts (hier klappt es nur, weil die DB ohne Persistenz leer neu startet). Echte Rotation - Datenbank und Vault im Gleichschritt - macht die Vault **Database Secrets Engine**.

Troubleshooting

Problem	Loesung
VaultAuth Status nicht Accepted	kubectl describe vaultauth vault-auth -n default - meist falscher mount-Name
VaultStaticSecret zeigt SYNCED: False	kubectl describe vaultstaticsecret mariadb-secret -n default - Events pruefen
permission denied beim Sync	Policy-Pfad in Vault pruefen: secret/data/mariadb (nicht secret/mariadb) - bei KV-v2 immer mit data/
MariaDB-Pod startet nicht, CrashLoopBackOff	kubectl logs mariadb-vso-0 -n default - haeufig falscher rootPasswordKey (muss exakt der Key im Secret sein, siehe kubectl describe secret)
VaultConnection/VaultAuth gefunden, aber Login schlaegt fehl	Dein kubernetes-<tl>-Mount existiert noch nicht - beim Trainer nachfragen, ob setup-participant.sh fuer dich gelaufen ist

Aufraeumen

```
helm uninstall mariadb-vso -n default
kubectl delete -f ~/manifests/vault-vso/ -n default
helm uninstall vault-secrets-operator -n vault-secrets-operator
kubectl delete namespace vault-secrets-operator
```

Zusammenfassung: Datenfluss

```
Gemeinsamer Vault-Server (vault-bka.do.t3isp.de)
├─ secret/mariadb (username + password, fuer alle TN identisch)
│
│   ▼ (K8s-Auth ueber kubernetes-<tl>, Rolle mariadb)
VSO-Controller im eigenen Cluster synct alle 60s
│
│   ▼
K8s Secret (mariadb-vault-secret)
│
```

```
▼
MariaDB-Pod (Helm-Chart cloudpirates/mariadb, auth.existingSecret)
```

Uebung: MariaDB-Deployment mit dem Vault Agent Injector

Hintergrund

Der Vault Agent Injector arbeitet grundlegend anders als der Vault Secrets Operator aus der letzten Uebung: Er erzeugt **kein** Kubernetes Secret. Stattdessen laeuft es so ab:

- 1. Ein Admission-Webhook mutiert jeden Pod mit der passenden Annotation: Er haengt ihm einen `vault-agent -Init-Container` und einen `vault-agent -Sidecar` an.
- 2. Der Init-Container loggt sich bei Vault ein, rendert das Secret als Datei in ein `emptyDir -Volume ( /vault/secrets/... )` und beendet sich.
- 3. Der Sidecar haelt das Secret aktuell (Renewal, periodisches Re-Rendern).
- 4. Die Anwendung liest ganz normal eine lokale Datei - ein Kubernetes Secret existiert zu keinem Zeitpunkt.

Agent Injector Datenfluss

Aspekt	VSO (letzte Uebung)	Agent Injector (diese Uebung)
Kubernetes Secret?	Ja (mariadb-vault-secret)	Nein - nie
Wo landet das Secret?	K8s-Secret-Objekt (etcd)	Datei in emptyDir im Pod
Wie kommt es in den Container?	secretKeyRef (Chart-Feature)	Datei + _FILE-Env-Var (App-Feature)
Zusaetzhche Container im Pod	Keiner	vault-agent-init (Init) + vault-agent (Sidecar)
Wo laeuft die Vault-Authentifizierung?	Operator-Pod (clusterweit)	Jeder App-Pod einzeln

Setup: Gleicher Vault-Server, gleiches Secret (`secret/mariadb`) wie in der VSO-Uebung. Wenn du die VSO-Uebung schon durchgespielt hast, ist die Vorschau (`vault kv get secret/mariadb`) bereits bekannt - dieser Schritt kann dann uebersprungen werden.

Voraussetzungen

- Eigenes kubernetes-Cluster, Helm v3
- Dein `kubernetes-<tln> -Auth-Mount` in Vault existiert bereits, inkl. der Rolle `mariadb` (gebunden an `ServiceAccount mariadb-sa /Namespace default` , Policy `mariadb-read`) - der Trainer hat das fuer dich eingerichtet, siehe "Hintergrund: Was der Trainer fuer dich schon eingerichtet hat" in der VSO-Uebung (`01-mariadb-vault-secrets-operator.md`) fuer die Details. Diese Uebung nutzt exakt denselben Mount und dieselbe Rolle wie die VSO-Uebung - der Unterschied liegt nur darin, WIE der Pod sich damit einloggt (siehe Vergleichstabelle oben), nicht in der Auth-Konfiguration selbst.
- `ServiceAccount mariadb-sa` im `Namespace default` existiert (aus der VSO-Uebung, sonst siehe Schritt 3 unten)

Schritt 1: Arbeitsverzeichnisse anlegen

```
mkdir -p ~/manifests/vault-agent ~/helm-values/vault-agent
cd ~/manifests/vault-agent
```

Schritt 2: Vault Agent Injector installieren

Anders als bei VSO installieren wir hier den **offiziellen HashiCorp Vault-Chart** - aber nur mit `injector.enabled` , ohne eigenen Vault-Server (`server.enabled: false`) , da wir den bereits laufenden Server extern ansprechen.

```
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update hashicorp
```

```
nano ~/helm-values/vault-agent/vault-injector-values.yml

global:
  externalVaultAddr: https://vault-bka.do.t3isp.de
injector:
  enabled: true
  authPath: "auth/kubernetes-tln1"
server:
  enabled: false
csi:
  enabled: false
```

Wichtig: `authPath` auf deinen eigenen Teilnehmernamen anpassen (z.B. `auth/kubernetes-tln4`)! Der Injector nutzt diesen Pfad als Default fuer alle Pods, die er mutiert - eine Pod-Annotation kann ihn zwar pro Pod ueberschreiben, wir setzen ihn hier aber gleich global richtig.

```
helm install vault hashicorp/vault \
  -n vault-injector --create-namespace \
  -f ~/helm-values/vault-agent/vault-injector-values.yml
```

```
kubect1 -n vault-injector rollout status deploy/vault-agent-injector
```

Erwartete Ausgabe:

```
deployment "vault-agent-injector" successfully rolled out
```

Schritt 3: ServiceAccount fuer MariaDB (falls noch nicht vorhanden)

```
nano 00-mariadb-sa.yml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: mariadb-sa
```

```
kubectl apply -f 00-mariadb-sa.yml -n default
```

Schritt 4: MariaDB mit Agent-Injector-Annotations ausrollen

Die Injection wird komplett ueber Pod-Annotations gesteuert - der `cloudpirates/mariadb` -Chart kennt Vault gar nicht, wir reichen die Annotations einfach ueber `podAnnotations` durch.

```
nano ~/helm-values/vault-agent/mariadb-values.yml
```

```
auth:
  enabled: false
persistence:
  enabled: false
podAnnotations:
  vault.hashicorp.com/agent-inject: "true"
  vault.hashicorp.com/role: "mariadb"
  vault.hashicorp.com/agent-inject-secret-mariadb-root-password: "secret/data/mariadb"
  vault.hashicorp.com/agent-inject-template-mariadb-root-password: |
    {{- with secret "secret/data/mariadb" -}}
    {{ .Data.data.password }}
    {{- end -}}
extraEnvVars:
  - name: MARIADB_ROOT_PASSWORD_FILE
    value: /vault/secrets/mariadb-root-password
serviceAccount:
  create: false
  name: mariadb-sa
  automountServiceAccountToken: true
```

Drei Stolpersteine, die hier schon geloest sind:

1. `auth.enabled: false` - sonst setzt der Chart selbst ein `MARIADB_ROOT_PASSWORD` (aus einem auto-generierten Secret), das sich mit `MARIADB_ROOT_PASSWORD_FILE` **beisst** (Both ... are set (but are exclusive) , Container crasht).
2. `serviceAccount.automountServiceAccountToken: true` - der Chart deaktiviert das Automounten standardmaessig aus Sicherheitsgrunden. Der Vault-Agent im Pod braucht aber genau dieses Token, um sich bei Vault einzuloggen (`failed to find service account volume mount` , Pod-Erstellung wird vom Webhook abgelehnt).
3. Das Template rendert **nur** das rohe Passwort (kein `key: value` , kein JSON) - passend zum `_FILE` -Konsum durch das offizielle MariaDB-Image (Docker-Secrets-Konvention).

```
helm install mariadb-agent oci://registry-1.docker.io/cloudpirates/mariadb \
-n default \
-f ~/helm-values/vault-agent/mariadb-values.yml
```

```
kubectl -n default rollout status statefulset/mariadb-agent
```

Schritt 5: Verifikation

Pod-Status - zwei zusaetzliche Container gegenueber der VSO-Uebung:

```
kubectl get pod mariadb-agent-0 -n default -o jsonpath='{.spec.initContainers[*].name}{ "\n" }{.spec.containers[*].name}{ "\n" }'
```

Erwartete Ausgabe:

```
vault-agent-init
mariadb vault-agent
```

Die vom Agent gerenderte Datei ansehen (Existenz + Groesse, nicht den Inhalt):

```
kubectl exec mariadb-agent-0 -n default -c mariadb -- ls -la /vault/secrets/
```

Erwartete Ausgabe:

```
-rw-r--r-- 1 100 mysql 22 ... mariadb-root-password
```

Login-Test:

```
source /etc/training-vault.env
kubect1 exec -n default mariadb-agent-0 -c mariadb -- mariadb -uroot -p"$MARIADB_ROOT_PASSWORD" -e "SELECT 2 AS agent_login_test;"
```

Erwartete Ausgabe:

```
agent_login_test
2
```

Zum Vergleich - es existiert wirklich kein Kubernetes Secret mit dem Passwort darin (nur das Helm-Release-Bookkeeping, kein `Opaque` -Secret):

```
kubect1 get secrets -n default --field-selector type=Opaque
```

Erwartete Ausgabe:

```
No resources found in default namespace.
```

Hinweis: `kubect1 get secrets -n default` zeigt trotzdem einen Treffer namens `sh.helm.release.v1.mariadb-agent.v1` - das ist Helms eigenes Release-Bookkeeping (Typ `helm.sh/release.v1`), kein Anwendungs-Secret und enthaelt keine MariaDB-Zugangsdaten.

Troubleshooting

Problem	Loesung
Pod-Erstellung schlaegt fehl: failed to find service account volume mount	<code>serviceAccount.automountServiceAccountToken: true</code> fehlt in den Values
Container <code>mariadb</code> crasht: Both <code>MARIADB_ROOT_PASSWORD</code> and <code>..._FILE</code> are set	<code>auth.enabled: false</code> fehlt - der Chart setzt sonst selbst ein Passwort
<code>vault-agent-init</code> haengt / Timeout	<code>authPath</code> in den Injector-Values falsch (nicht dein <code>kubernetes-<tln>-Mount</code>) - <code>kubect1 logs <pod> -c vault-agent-init</code> pruefen
Datei <code>/vault/secrets/mariadb-root-password</code> fehlt oder leer	Annotation-Name der Template-Annotation muss exakt zum <code>agent-inject-secret-<name></code> passen (<name> identisch in beiden Annotation-Keys)
permission denied im Agent-Log	Policy/Rolle pruefen - dieselbe Rolle <code>mariadb</code> wie in der VSO-Uebung, Pfad <code>secret/data/mariadb</code>

Aufraeumen

```
helm uninstall mariadb-agent -n default
kubect1 delete -f ~/manifests/vault-agent/ -n default
helm uninstall vault -n vault-injector
kubect1 delete namespace vault-injector
```

Zusammenfassung: Datenfluss

```
Gemeinsamer Vault-Server (vault-bka.do.t3isp.de)
└─ secret/mariadb (username + password, fuer alle TN identisch)
    |
    ▼ (K8s-Auth ueber kubernetes-<tln>, Rolle mariadb - Login IM Pod)
    vault-agent-init (Init-Container) rendert Datei, beendet sich
    |
    ▼
    /vault/secrets/mariadb-root-password (emptyDir, nur im Pod sichtbar)
    |
    ▼ (MARIADB_ROOT_PASSWORD_FILE)
    MariaDB-Container liest die Datei beim Start
```

Workload-Skalierung

Autoscaling Pods/Deployments - Grundlagen

Example: newest version with autoscaling/v2 used to be hpa/v1

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
```

```

    labels:
      app: hello
    spec:
      containers:
      - name: hello
        image: k8s.gcr.io/hpa-example
        resources:
          requests:
            cpu: 100m
---
kind: Service
apiVersion: v1
metadata:
  name: hello
spec:
  selector:
    app: hello
  ports:
  - port: 80
    targetPort: 80
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hello
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hello
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 80

```

- <https://docs.digitalocean.com/tutorials/cluster-autoscaling-ca-hpa/>

Reference

- <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/#autoscaling-on-more-specific-metrics>
- <https://medium.com/expedia-group-tech/autoscaling-in-kubernetes-why-doesnt-the-horizontal-pod-autoscaler-work-for-me-5f0094694054>

Uebung: Horizontal Pod Autoscaler (HPA)

Aufbau

```

## Aufbau des Containers
ROM php:5-apache
COPY index.php /var/www/html/index.php
RUN chmod a+rx index.php
This code defines a simple index.php page that performs some CPU intensive computations, in order to simulate load in your cluster.

<?php
    $x = 0.0001;
    for ($i = 0; $i <= 1000000; $i++) {
        $x += sqrt($x);
    }
    echo "OK!";
?>

```

Schritt 1: Metrics-Server installieren (Voraussetzung)

- Der HorizontalPodAutoscaler braucht die Metrics API (metrics-server), um die CPU-Auslastung der Pods auszulesen.
- Auf unseren kubeadm-Trainingsclustern ist der metrics-server NICHT vorinstalliert.
- Ohne ihn zeigt `kubectl get hpa` beim TARGET dauerhaft `<unknown>` und es wird nie skaliert.

```

helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
helm repo update

```

```

cd
mkdir -p helm-charts/metrics-server
cd helm-charts/metrics-server
nano values.yml

```

```
## Die kubelets im Trainingscluster nutzen selbst-signierte Zertifikate,
## daher braucht der metrics-server dieses Flag
args:
  - --kubelet-insecure-tls
```

```
helm -n kube-system upgrade --install metrics-server metrics-server/metrics-server --version 3.13.0 -f values.yml
```

```
## Pruefen - dauert ca. 1 Minute, bis der Pod Ready ist
kubectl -n kube-system get pods | grep metrics-server
```

```
## Sobald er Ready ist, liefert die Metrics API Daten:
kubectl top nodes
kubectl top pods -A
```

Walkthrough

```
## vi 01-php-apache-deploy.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-apache
spec:
  selector:
    matchLabels:
      run: php-apache
  replicas: 1
  template:
    metadata:
      labels:
        run: php-apache
    spec:
      containers:
        - name: php-apache
          image: k8s.gcr.io/hpa-example
          ports:
            - containerPort: 80
          resources:
            limits:
              cpu: 500m
            requests:
              cpu: 200m
---
apiVersion: v1
kind: Service
metadata:
  name: php-apache
  labels:
    run: php-apache
spec:
  ports:
    - port: 80
  selector:
    run: php-apache
```

```
kubectl apply -f 01-php-apache-deploy.yml
```

```
## autoscaler erstellen
## (--cpu-percent=50 ist deprecated, aktuelle Syntax:)
kubectl autoscale deployment php-apache --cpu=50% --min=1 --max=10
kubectl get hpa
kubectl get hpa -o yaml
```

```
## Output
###NAME          REFERENCE          TARGET    MINPODS  MAXPODS  REPLICAS  AGE
### php-apache   Deployment/php-apache/scale  0% / 50%  1        10       1
```

```
## Last erhöhen
## Run this in a separate terminal
## so that the load generation continues and you can carry on with the rest of the steps
kubectl run -i --tty load-generator --rm --image=busybox:1.28 --restart=Never -- /bin/sh -c "while sleep 0.01; do wget -q -O- http://php-apache; done"
```

```
## type Ctrl+C to end the watch when you're ready
kubectl get hpa php-apache --watch
```

```
## Nach ca. 1 Minute geht die Last hoch

## NAME          REFERENCE          TARGET          MINPODS   MAXPODS   REPLICAS   AGE
## php-apache    Deployment/php-apache/scale  305% / 50%      1          10         1           3m

## Und etwas später noch mehr

## NAME          REFERENCE          TARGET          MINPODS   MAXPODS   REPLICAS   AGE
## php-apache    Deployment/php-apache/scale  305% / 50%      1          10         7           7m

## Wie sieht es aus ?
kubectl get deployment php-apache
## You should see the replica count matching the figure from the HorizontalPodAutoscaler

NAME          READY   UP-TO-DATE   AVAILABLE   AGE
php-apache    7/7     7            7           19m
```

Ref:

- <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>

Übung: HPA mit eigener Metrik (KEDA + eigener Prometheus)

Hintergrund

- Der HorizontalPodAutoscaler (HPA) kennt von Haus aus nur die Kubernetes-Metrics-APIs: `metrics.k8s.io` (CPU/Memory, ueber den metrics-server), `custom.metrics.k8s.io` und `external.metrics.k8s.io`. Er kann NIE direkt PromQL gegen Prometheus sprechen.
- Fuer eine eigene (Business-/App-)Metrik braucht es einen Adapter, der zwischen Prometheus und dieser Metrics-API uebersetzt. Zwei Wege:
 - **Prometheus Adapter** - klassisch, aber fummelige Rule-Konfiguration
 - **KEDA** (Kubernetes-based Event-Driven Autoscaling, CNCF-Projekt) - du definierst ein einfaches `ScaledObject` mit einer PromQL-Query, KEDA fragt Prometheus selbst ab und erzeugt/pflegt automatisch ein ganz normales HPA im Hintergrund
- Wir verwenden KEDA. Als Beispiel-Metrik nehmen wir die **Worker-Auslastung eines PHP-FPM-Pools** (aktive / gesamt Worker in %) - ein typisches App-Level-Signal, das CPU-Auslastung nicht zeigen wuerde (ein Worker kann voll ausgelastet sein, obwohl er nur auf eine langsame Datenbank wartet und dabei kaum CPU braucht).

Voraussetzung

- Keine - diese Übung bringt ihren eigenen, schlanken Prometheus mit (Schritt 2, im Namespace `scaling-monitoring`) und braucht NICHT den vollen kube-prometheus-stack aus "Monitoring mit Prometheus" (Tag 2, Namespace `monitoring`, mit Grafana/Traefik-Ingress/Letsencrypt/basic-auth). Deshalb passt sie hier unter Workload-Skalierung, direkt nach der normalen HPA-Übung.

Schritt 1: KEDA installieren

```
helm repo add kedacore https://kedacore.github.io/charts
helm repo update
helm upgrade --install keda kedacore/keda --namespace keda --create-namespace
```

```
kubectl -n keda get pods
## 3 Pods sollten Running sein (operator, operator-metrics-apiserver, admission-webhooks)
```

Schritt 2: Eigenen, schlanken Prometheus installieren

- Nur fuer diese Übung gedacht - ohne Grafana, ohne Ingress/TLS/basic-auth (das kommt erst mit dem vollen Setup auf Tag 2). Reicht, damit KEDA intern PromQL-Queries stellen kann.
- Bewusst ein **eigener Namespace** `scaling-monitoring`, getrennt vom Namespace `monitoring`, den Tag 2 fuer den vollen Stack (Grafana, Ingress, ...) nutzt - keine Vermischung der beiden Installationen.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm upgrade --install prometheus prometheus-community/kube-prometheus-stack \
  --namespace scaling-monitoring --create-namespace --version 72.3.0 \
  --set grafana.enabled=false \
  --set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false
```

```
kubectl -n scaling-monitoring get pods
## Warten, bis u.a. prometheus-prometheus-kube-prometheus-prometheus-0 Running/Ready ist
```

Schritt 3: Vorbereitung

```
cd
mkdir -p manifests
cd manifests
mkdir php-fpm-hpa
cd php-fpm-hpa
```

Schritt 4: Namespace

```
## vi 01-namespace.yaml
apiVersion: v1
```



```
kind: Namespace
metadata:
  name: php-fpm-demo
```

```
kubectl apply -f 01-namespace.yaml
```

Schritt 5: Die App - ConfigMap mit PHP-Skript, FPM-Pool und nginx-Config

- `sleep(5)` simuliert eine "teure" Anfrage (z.B. langsamer DB-Call), die einen PHP-FPM-Worker fuer 5 Sekunden blockiert
- `pm = static + pm.max_children = 3` -> der Pool hat IMMER genau 3 Worker (bewusst klein gehalten, damit sich die Uebung schnell in eine Ueberlastsituation bringen laesst)
- `pm.status_path = /status` aktiviert die eingebaute FPM-Statusseite

```
## vi 02-app-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: php-fpm-app
  namespace: php-fpm-demo
data:
  index.php: |
    <?php
    // Simuliert eine "teure" Anfrage, die einen PHP-FPM-Worker
    // fuer 5 Sekunden blockiert (z.B. langsamer DB-Call, PDF-Export, ...)
    sleep(5);
    echo "OK - Worker war 5 Sekunden belegt\n";
  zz-status.conf: |
    ; Achtung: pm.status_path liefert NUR Zahlen des eigenen Pools.
    ; Ein zweiter, dedizierter "Status-Pool" (mit eigenem Port) wuerde
    ; sich nur selbst vermessen - nicht den Workload-Pool "www". Deshalb
    ; bleibt der Status-Endpoint bewusst im selben Pool wie die Worker.
    [www]

    pm = static
    pm.max_children = 3
    pm.status_path = /status
  default.conf: |
    server {
        listen 80;
        root /var/www/html;
        index index.php;

        location / {
            try_files $uri $uri/ /index.php$is_args$args;
        }

        location ~ \.php$ {
            fastcgi_pass 127.0.0.1:9000;
            fastcgi_index index.php;
            fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
            include fastcgi_params;
        }
    }
```

```
kubectl apply -f 02-app-configmap.yaml
```

- **Hinweis (warum das oben wichtig ist):** Wenn alle 3 Worker mit der 5-Sekunden-Anfrage beschaefigt sind, muss sich auch die Status-Abfrage selbst in dieselbe Warteschlange einreihen - sie teilt sich den Pool mit dem Workload. Bei extremer Ueberlast (z.B. 10+ parallele Anfragen gegen nur 3 Worker) kann das dazu fuehren, dass Prometheus den Scrape-Timeout reisst und die Metrik zeitweise fehlt. Deshalb erzeugen wir in Schritt 10 bewusst nur moderate Ueberlast (8 parallele Requests gegen 3 Worker) - genug zum Skalieren, aber die Status-Abfrage kommt trotzdem noch durch.

Schritt 6: Deployment (3 Container: php-fpm, nginx, Exporter) + Service

- `nginx` reicht `.php` -Requests per FastCGI an `php-fpm` weiter (Port 9000, localhost)
- `exporter` (`hipages/php-fpm-exporter`) fragt die FPM-Statusseite ab und wandelt sie in Prometheus-Metriken um (`phpfpm_active_processes` , `phpfpm_total_processes` , ...)

```
## vi 03-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: php-fpm-app
  namespace: php-fpm-demo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: php-fpm-app
  template:
```

```

metadata:
  labels:
    app: php-fpm-app
spec:
  containers:
    - name: php-fpm
      image: php:8.3-fpm
      volumeMounts:
        - name: app
          mountPath: /var/www/html/index.php
          subPath: index.php
        - name: app
          mountPath: /usr/local/etc/php-fpm.d/zz-status.conf
          subPath: zz-status.conf
      resources:
        requests:
          cpu: 50m
        limits:
          cpu: 200m
    - name: nginx
      image: nginx:stable
      ports:
        - containerPort: 80
      volumeMounts:
        - name: app
          mountPath: /var/www/html/index.php
          subPath: index.php
        - name: app
          mountPath: /etc/nginx/conf.d/default.conf
          subPath: default.conf
    - name: exporter
      image: hipages/php-fpm_exporter:2.2.0
      args:
        - "server"
        - "--phpfpm.scrape-uri=tcp://127.0.0.1:9000/status"
      ports:
        - containerPort: 9253
  volumes:
    - name: app
      configMap:
        name: php-fpm-app
---
apiVersion: v1
kind: Service
metadata:
  name: php-fpm-app
  namespace: php-fpm-demo
  labels:
    app: php-fpm-app
spec:
  selector:
    app: php-fpm-app
  ports:
    - name: http
      port: 80
      targetPort: 80
    - name: metrics
      port: 9253
      targetPort: 9253

```

```

kubectl apply -f 03-deployment.yaml
kubectl -n php-fpm-demo rollout status deployment/php-fpm-app

```

Schritt 7: Exporter-Metriken pruefen

```

kubectl -n php-fpm-demo run test-metrics --image=busybox:1.36 --restart=Never --rm -i --command -- wget -qO- http://php-fpm-app.php-fpm-demo:9253/metrics

```

```

## Erwartete Ausgabe (Ausschnitt):
phpfpm_active_processes{pool="www",...} 0
phpfpm_idle_processes{pool="www",...} 3
phpfpm_total_processes{pool="www",...} 3

```

Schritt 8: ServiceMonitor

```
## vi 04-servicemonitor.yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: php-fpm-app
  namespace: php-fpm-demo
  labels:
    release: prometheus # muss zu Helm-Werten passen!
spec:
  selector:
    matchLabels:
      app: php-fpm-app
  namespaceSelector:
    matchNames:
      - php-fpm-demo
  endpoints:
    - port: metrics
      path: /metrics
      interval: 15s
      scrapeTimeout: 10s
```

```
kubectl apply -f 04-servicemonitor.yaml
```

- **Warum das hier noetig ist:** KEDA fragt im naechsten Schritt NICHT unsere App direkt, sondern schickt seine PromQL-Query an den Prometheus-Server. Damit dort ueberhaupt etwas steht, muss Prometheus unsere Metrik vorher eingesammelt haben - genau das erledigt dieser ServiceMonitor (er sagt Prometheus: "scrape den Exporter alle 15s"). Ohne ihn liefe KEDAs Query ins Leere.

```
## Ist der Target in Prometheus gruen? Kein Ingress fuer diesen schlanken
## Stack (siehe Schritt 2) - deshalb per Port-Forward pruefen:
kubectl -n scaling-monitoring port-forward svc/prometheus-kube-prometheus-prometheus 9090:9090
```

```
## In einem zweiten Terminal (oder Browser):
http://localhost:9090/targets
## suchen nach: php-fpm-demo/php-fpm-app/0
```

Schritt 9: ScaledObject (KEDA) - die eigentliche Skalierungslogik

- Statt `%CPU` nehmen wir hier `avg(active) / avg(total) * 100` - die Worker-Auslastung in Prozent, ueber alle Pods gemittelt. Genau dasselbe Muster wie bei der CPU-HPA-Uebung (Ziel-Prozentwert), nur mit einer App-eigenen Kennzahl statt einer Infra-Kennzahl.
- `pollingInterval` / `cooldownPeriod` lassen wir bewusst weg - die sind nur relevant, wenn `minReplicaCount` (oder `idleReplicaCount`) auf 0 steht (Scale-to-Zero).

```
## vi 05-scaledobject.yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: php-fpm-app
  namespace: php-fpm-demo
spec:
  scaleTargetRef:
    name: php-fpm-app
  minReplicaCount: 1
  maxReplicaCount: 5
  triggers:
    - type: prometheus
      metadata:
        # serverAddress zeigt auf Prometheus, NICHT auf unsere App/den Exporter!
        # KEDA fragt hier den Prometheus-Server per PromQL ab (die Daten dafuer
        # hat der ServiceMonitor aus Schritt 8 vorher dort hineingescraped).
        # Der grosse Vorteil: avg(...) aggregiert automatisch ueber ALLE Pods -
        # ein einzelner Pod koennte seine eigene Auslastung kennen, aber nicht,
        # wie ausgelastet das gesamte Deployment gerade ist.
        serverAddress: http://prometheus-kube-prometheus-prometheus.scaling-monitoring.svc.cluster.local:9090
        query: avg/phpfpm_active_processes{namespace="php-fpm-demo"} / avg/phpfpm_total_processes{namespace="php-fpm-demo"} * 100
        threshold: "70"
```

```
kubectl apply -f 05-scaledobject.yaml
```

```
## KEDA erzeugt jetzt automatisch ein HPA im Hintergrund:
kubectl -n php-fpm-demo get scaledobject
kubectl -n php-fpm-demo get hpa
```

## NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
## keda-hpa-php-fpm-app	Deployment/php-fpm-app	0/70 (avg)	1	5	1	15s

Schritt 10: Last erzeugen und Skalierung beobachten

- 8 parallele Dauerschleifen gegen einen Pool mit nur 3 Workern - reicht zum Ueberschreiten der 70%-Schwelle, ohne die Status-Abfrage zu blockieren (siehe Hinweis oben bei Schritt 5)

```
## vi 06-load-generator.yaml
apiVersion: v1
kind: Pod
metadata:
  name: load-generator
  namespace: php-fpm-demo
spec:
  restartPolicy: Never
  containers:
  - name: load-generator
    image: busybox:1.36
    command: ["/bin/sh", "-c"]
    args:
    - |
      for i in $(seq 1 8); do
        (while true; do wget -q -O- http://php-fpm-app.php-fpm-demo >/dev/null; done) &
      done
      wait
```

```
kubectl apply -f 06-load-generator.yaml
```

```
## In einem zweiten Terminal beobachten:
kubectl -n php-fpm-demo get hpa keda-hpa-php-fpm-app --watch
```

```
## So in etwa laeuft es (Beispiel aus dem Test):
## NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
## keda-hpa-php-fpm-app Deployment/php-fpm-app    41667m/70 (avg)  1          5          1          28m
## keda-hpa-php-fpm-app Deployment/php-fpm-app    83333m/70 (avg)  1          5          1          28m  <- ueber 70% -> skaliert
## keda-hpa-php-fpm-app Deployment/php-fpm-app    50/70 (avg)      1          5          2          29m  <- 2 Pods, Last verteilt sich
```

```
kubectl -n php-fpm-demo get deployment php-fpm-app
## READY sollte jetzt 2/2 zeigen (oder mehr, je nach Last)
```

Schritt 11: Last stoppen und Scale-down beobachten

```
kubectl -n php-fpm-demo delete pod load-generator
kubectl -n php-fpm-demo get hpa keda-hpa-php-fpm-app --watch
```

- Der Zielwert faellt sofort unter 70%, die Anzahl der Replicas bleibt aber laut Standard-HPA- Verhalten noch ein paar Minuten stehen (Stabilization-Window, standardmaessig 5 Minuten), bevor auf 1 Replica zurueckskaliert wird - das verhindert staendiges Hoch-/Runterschaukeln bei schwankender Last.

Aufräumen

```
kubectl delete ns php-fpm-demo
kubectl delete ns scaling-monitoring
helm -n keda uninstall keda
kubectl delete ns keda
```

Ref

- <https://keda.sh/docs/latest/concepts/scaling-deployments/>
- <https://keda.sh/docs/latest/scalers/prometheus/>
- https://github.com/hinpages/php-fpm_exporter
- <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/#autoscaling-on-more-specific-metrics>

Monitoring mit Prometheus

Prometheus Monitoring Server (Overview)

What does it do ?

- It monitors your system by collecting data
- Data is pulled from your system by defined endpoints (http) from your cluster
- To provide data on your system, a lot of exporters are available, that
 - collect the data and provide it in Prometheus

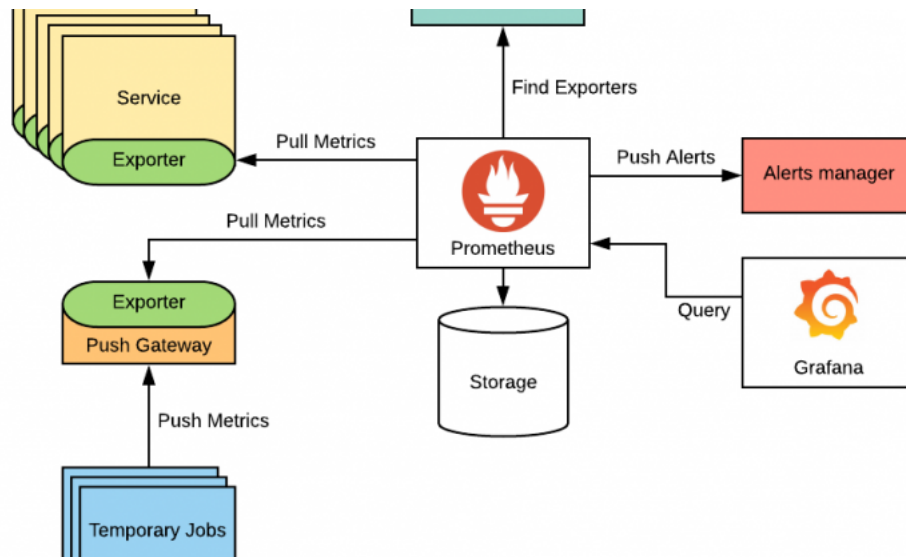
Technical

- Prometheus has a TDB (Time Series Database) and is good as storing time series with data
- Prometheus includes a local on-disk time series database, but also optionally integrates with remote storage systems.
- Prometheus's local time series database stores data in a custom, highly efficient format on local storage.
- Ref: <https://prometheus.io/docs/prometheus/latest/storage/>

What are time series ?

- A time series is a sequence of data points that occur in successive order over some period of time.
- Beispiel:
 - Du willst die täglichen Schlusspreise für eine Aktie für ein Jahr dokumentieren
 - Damit willst Du weitere Analysen machen
 - Du würdest das Paar Datum/Preis dann in der Datumsreihenfolge sortieren und so ausgeben
 - Dies wäre eine "time series"

Komponenten von Prometheus



Quelle: <https://www.devopsschool.com/>

Prometheus Server

1. Retrieval (Sammeln)
 - Data Retrieval Worker
 - pull metrics data
2. Storage
 - Time Series Database (TDB)
 - stores metrics data
3. HTTP Server
 - Accepts PromQL - Queries (e.g. from Grafana)
 - accept queries

Grafana ?

- Grafana wird meist verwendet um die grafische Auswertung zu machen.
- Mit Grafana kann ich einfach Dashboards verwenden
- Ich kann sehr leicht festlegen (Durch Data Sources), wo meine Daten herkommen

Prometheus/Grafana-Stack installieren mit helm (Traefik + Letsencrypt)

- Wir verwenden den kube-prometheus-stack (empfohlen! Bringt die wichtigen Metriken gleich mit)
- Als Ingress-Controller nutzen wir Traefik, die externe IP kommt von MetalLB (Kapitel Tag 1)

Achtung: Upgrades und Uninstall sind etwas tricky

- CRDs müssen nach einem Uninstall manuell gelöscht werden
- Vor einem Upgrade erst die CRDs aktualisieren
- <https://github.com/prometheus-community/helm-charts/blob/main/charts/kube-prometheus-stack/UPGRADE.md>

Was wollen wir erreichen?

- Prometheus und Alertmanager mit basic-auth schützen
- Zertifikate von Letsencrypt (http01-Challenge)
- Alles über Traefik als Ingress-Controller

Hintergrund: Warum funktioniert Letsencrypt (http01) mit MetalLB?

- Letsencrypt muss `http://<host>/.well-known/acme-challenge/...` auf Port 80 von aussen erreichen
- Unser MetalLB-Pool enthält die öffentlichen IPs der Worker-Nodes - der Traefik-LoadBalancer-Service bekommt also eine von aussen erreichbare IP
- Zeigt der DNS-Eintrag auf diese IP, läuft die Challenge ganz normal durch

Voraussetzungen

- MetalLB aus dem Tag-1-Kapitel ist installiert ([Kubernetes Load Balancer - metalb](#))
- `htpasswd` ist auf dem Client vorhanden (`sudo apt install apache2-utils` - schon erledigt)

- `/etc/training-dns.env` liegt auf dem Client (DNS-Token, stellt der Trainer bereit) - brauchen wir in Schritt 3 fuer den Wildcard-DNS-Eintrag

Schritt 1: Projekt-Ordner anlegen (nur der Ordnung halber)

```
cd
mkdir -p manifests
cd manifests
mkdir -p monitoring
cd monitoring
```

Schritt 2: Traefik installieren

```
helm repo add traefik https://traefik.github.io/charts
helm repo update
```

```
helm upgrade --install traefik traefik/traefik --namespace traefik --create-namespace --version 41.4.0
```

```
## Warten bis der Pod laeuft
kubectl -n traefik get pods
```

```
## WICHTIG: Die EXTERNAL-IP kommt von MetalLB - diese IP brauchen wir fuer die DNS-Eintraege
kubectl -n traefik get svc traefik
```

```
## Beispiel-Output:
## NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
## traefik       LoadBalancer  10.96.196.15   165.22.73.43   80:30848/TCP,443:31534/TCP  17s
```

Schritt 3: Wildcard-DNS auf deine Traefik-IP setzen

- Wir legen EINEN Wildcard-Record `*.<du>.do.t3isp.de` an - der deckt prometheus, grafana, alertmanager und alle spaeteren Hostnamen ab
- Das Script nutzt die DigitalOcean-DNS-API; das Token kommt aus `/etc/training-dns.env`
- WICHTIG: Die Namen erst NACH dem Anlegen per dig abfragen - wer vorher abfragt, handelt sich Negative-Caching ein (bis zu 30 min Wartezeit)

```
curl -sO https://raw.githubusercontent.com/jmetzger/workshop-kubernetes-advanced-2026-Q3/main/scripts/create-wildcard-dns.sh
chmod +x create-wildcard-dns.sh
```

```
## Name wird automatisch aus deinem Login-User gesetzt (z.B. tln1)
## <traefik-ip> = EXTERNAL-IP aus Schritt 2
./create-wildcard-dns.sh <traefik-ip>
```

```
## Pruefen (Beispiel):
dig +short prometheus.<du>.do.t3isp.de
## -> muss deine Traefik-IP zeigen
```

Schritt 4: basic-auth anlegen

- Traefik erwartet die httpswd-Daten in einem Secret unter dem Key `users`

```
kubectl create ns monitoring
htpasswd -c auth admin # Wunsch-Passwort eingeben
kubectl create secret generic prometheus-basic-auth --from-file=users=auth -n monitoring
```

- Bei Traefik wird basic-auth nicht per Annotation-Trio wie bei nginx konfiguriert, sondern ueber eine `Middleware`-Ressource (CRD von Traefik), die dann am Ingress referenziert wird

```
## vi 01-middleware.yml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: prometheus-basic-auth
spec:
  basicAuth:
    secret: prometheus-basic-auth
```

```
kubectl apply -f 01-middleware.yml -n monitoring
```

Schritt 5: cert-manager installieren

```
helm repo add jetstack https://charts.jetstack.io
helm repo update
```

```
nano cert-manager-values.yml
```

```
crds:
  enabled: true
```

```
helm upgrade --install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace --version 1.17.2 -f cert-manager-values.yml
```

Schritt 6: ClusterIssuer anlegen

```
nano 02-clusterissuer.yml
```

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    email: training.<du>@t3company.de
    server: https://acme-v02.api.letsencrypt.org/directory
    privateKeySecretRef:
      name: letsencrypt-prod
    solvers:
      - http01:
          ingress:
            ingressClassName: traefik
```

```
kubectl apply -f 02-clusterissuer.yml
kubectl get clusterissuer
```

Schritt 7: Monitoring-Stack vorbereiten (values-Datei)

- `<du>` ueberall durch deinen Teilnehmer-Namen ersetzen (z.B. `tlh5`)
- Das `adminPassword` fuer Grafana bitte durch ein eigenes ersetzen
- `basic-auth` haengt als Traefik-Middleware am Prometheus- und Alertmanager-Ingress (Format der Referenz: `<namespace>-<middleware-name>@kubernetescd`)

```
nano monitoring-values.yml
```

```
grafana:
  fullnameOverride: grafana
  enabled: true
  adminUser: admin
  adminPassword: "yourStrongPassword"
  ingress:
    enabled: true
    ingressClassName: traefik
    annotations:
      cert-manager.io/cluster-issuer: letsencrypt-prod
    hosts:
      - grafana.<du>.do.t3isp.de
    path: /
    pathType: Prefix
    tls:
      - hosts:
          - grafana.<du>.do.t3isp.de
        secretName: grafana-tls

prometheus:
  ingress:
    enabled: true
    ingressClassName: traefik
    annotations:
      cert-manager.io/cluster-issuer: letsencrypt-prod
      traefik.ingress.kubernetes.io/router.middlewares: monitoring-prometheus-basic-auth@kubernetescd
    hosts:
      - prometheus.<du>.do.t3isp.de
    paths:
      - /
    pathType: Prefix
    tls:
      - hosts:
          - prometheus.<du>.do.t3isp.de
        secretName: prometheus-tls

prometheusOperator:
  admissionWebhooks:
    enabled: true

alertmanager:
  ingress:
    enabled: true
```

```

    ingressClassName: traefik
    annotations:
      cert-manager.io/cluster-issuer: letsencrypt-prod
      traefik.ingress.kubernetes.io/router.middlewares: monitoring-prometheus-basic-auth@kubernetescrd
    hosts:
      - alertmanager.<du>.do.t3isp.de
    paths:
      - /
    pathType: Prefix
    tls:
      - hosts:
          - alertmanager.<du>.do.t3isp.de
        secretName: alertmanager-tls

kube-state-metrics:
  fullnameOverride: kube-state-metrics

prometheus-node-exporter:
  fullnameOverride: node-exporter

```

Schritt 8: Mit helm installieren

```

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm upgrade --install prometheus prometheus-community/kube-prometheus-stack \
  -f monitoring-values.yml --namespace monitoring --version 72.3.0

```

Schritt 9: Prüfen, ob alles funktioniert

```

kubectl -n monitoring get pods
kubectl -n cert-manager get pods

```

```

## Neue Ressourcen von cert-manager anschauen
kubectl get clusterissuer
kubectl -n monitoring get certificaterequests
kubectl -n monitoring get certificates

```

```

## Waehrend die Challenge laeuft, sieht man temporaere cm-acme-http-solver-Ingresse:
kubectl -n monitoring get ingress
kubectl -n monitoring get challenges

```

```

## Nach 1-3 Minuten sollten alle drei Zertifikate READY=True sein:
## NAME          READY  SECRET  AGE
## alertmanager-tls  True   alertmanager-tls  2m
## grafana-tls      True   grafana-tls       2m
## prometheus-tls   True   prometheus-tls    2m

## Es ist normal, dass ein Zertifikat 1-2 Minuten spaeter fertig wird als die
## anderen (ACME-Backoff nach dem ersten Versuch) - einfach nochmal abfragen.

## Falls ein Zertifikat laenger haengt:
kubectl -n monitoring describe challenge <name-aus-get-challenges>

```

Schritt 10: Prometheus von aussen erreichen

- Browser: `https://prometheus.<du>.do.t3isp.de` -> Login-Popup (basic-auth)

```

## oder per curl testen:
curl -s -o /dev/null -w "%{http_code}\n" https://prometheus.<du>.do.t3isp.de
## 401 (ohne Auth - gut so!)

curl -s -o /dev/null -w "%{http_code}\n" -u admin:<dein-passwort> https://prometheus.<du>.do.t3isp.de/graph
## 302/200 (mit Auth)

```

- Ohne Auth kommt ein sauberes 401 vom Traefik-Middleware (nicht von Prometheus selbst):

 Prometheus ohne Basic-Auth: 401 Unauthorized von Traefik

- Mit Auth: unter Status > Target health siehst Du alle ServiceMonitor-Targets (hier kube-state-metrics und node-exporter auf allen Workern, alle UP):

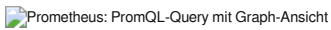
 Prometheus Target health: alle Targets UP

- Beispiel-Query (Reiter "Query" -> Feld oben, danach auf "Graph" wechseln): CPU-Rate pro Pod im eigenen monitoring -Namespace

```

sum(rate(container_cpu_usage_seconds_total{namespace="monitoring"}[5m])) by (pod)

```

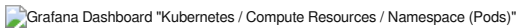



Schritt 11: Grafana von aussen erreichen

- Browser: `https://grafana.<du>.do.t3isp.de` -> Login mit admin + deinem adminPassword

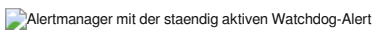


- Der kube-prometheus-stack bringt fertige Dashboards mit (Ordner "Kubernetes" unter "Dashboards"). Fuer Pods interessant: **Kubernetes / Compute Resources / Namespace (Pods)**
 - Namespace-Variable oben auf `monitoring` stellen, dann siehst Du CPU- und Memory-Verbrauch je Pod (hier die eigenen Prometheus/Grafana/Alertmanager-Pods):



Schritt 12: Alertmanager von aussen erreichen

- Browser: `https://alertmanager.<du>.do.t3isp.de` -> Login-Popup (basic-auth)
- Der kube-prometheus-stack legt eine `Watchdog` -Alert an, die dauerhaft feuert (Beweis, dass die Alerting-Pipeline lebt):



Achtung: Kein persistenter Storage

- Prometheus nutzt in diesem Chart per Default EmptyDir - Daten leben nur so lange wie der Pod
- Retention: aktuell 10d

```
prometheus-prometheus-kube-prometheus-prometheus-db:
  Type:          EmptyDir (a temporary directory that shares a pod's lifetime)
```

Optional: StorageClass verwenden (nur wenn im Cluster vorhanden!)

```
## Erst pruefen - auf unseren Trainingsclustern gibt es aktuell KEINE StorageClass:
kubectl get storageclass
```

- Falls eine StorageClass vorhanden ist, in `monitoring-values.yml` unter `prometheus:` ergaenzen (Name anpassen!) und erneut ausrollen:

```
prometheus:
  prometheusSpec:
    storageSpec:
      volumeClaimTemplate:
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 20Gi
          storageClassName: "<deine-storageclass>"
```

Aufraeumen

- ACHTUNG: Erst NACH der ServiceMonitor-Uebung aufraeumen - sie baut auf diesem Stack auf!

```
helm -n monitoring uninstall prometheus
kubectl delete ns monitoring
## CRDs bleiben nach dem Uninstall stehen - Liste zum manuellen Loeschen:
## https://github.com/prometheus-community/helm-charts/blob/main/charts/kube-prometheus-stack/UPGRADE.md
## cert-manager und traefik koennen stehen bleiben (werden ggf. weiterverwendet)
```

Referenzen:

- <https://github.com/prometheus-community/helm-charts/blob/main/charts/kube-prometheus-stack/README.md>
- <https://artifacthub.io/packages/helm/prometheus-community/kube-prometheus-stack>
- <https://doc.traefik.io/traefik/middlewares/http/basicauth/>

Uebung: nginx mit ServiceMonitor und Exporter (Sidecar)

Voraussetzung:

- kube-prometheus-stack muss installiert sein -> [Kube-Prometheus-Stack installieren](#)

Vorbereitung: Verzeichnisstruktur anlegen

```
cd ~
mkdir -p manifests
cd manifests
```

```
mkdir svcn-nginx
cd svcn-nginx
```

Alle YAML-Dateien werden in diesem Verzeichnis erstellt und mit `kubectl apply -f .` angewendet.

1. Namespace

```
nano 01-namespace.yaml
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: web-demo
```

```
kubectl apply -f .
```

2. ConfigMap: Stub Status aktivieren

```
nano 02-nginx-stubstatus-configmap.yaml
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-stubstatus
  namespace: web-demo
data:
  default.conf: |
    server {
      listen 80;
      location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
      }

      location /stub_status {
        stub_status;
        allow 127.0.0.1;
        deny all;
      }
    }
  }
```

```
kubectl apply -f .
```

3. Deployment mit Sidecar (Exporter)

```
nano 03-nginx-deployment-metrics.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: web-demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:stable
          ports:
            - containerPort: 80
          volumeMounts:
            - name: nginx-conf
```

```
        mountPath: /etc/nginx/conf.d/default.conf
        subPath: default.conf
      - name: exporter
        image: nginx/nginx-prometheus-exporter:latest
        args:
          - "-nginx.scrape-uri=http://localhost:80/stub_status"
        ports:
          - containerPort: 9113
    volumes:
      - name: nginx-conf
        configMap:
          name: nginx-stubstatus
```

```
kubectl apply -f .
```

4. Service mit zusätzlichem Metrics-Port

```
nano 04-nginx-service-with-metrics.yaml
```

```
apiVersion: v1
kind: Service
metadata:
  name: nginx
  namespace: web-demo
  labels:
    app: nginx
spec:
  selector:
    app: nginx
  ports:
    - name: http
      port: 80
      targetPort: 80
    - name: metrics
      port: 9113
      targetPort: 9113
```

```
kubectl apply -f .
```

5. Verbindung testen

```
kubectl run -it --rm podtest --image=busybox
```

```
## in der bash
wget -O - http://nginx.web-demo:9113/metrics
exit
```

6. Ingress (Optional)

```
nano 06-nginx-ingress.yaml
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx
  namespace: web-demo
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: app.tln1.do.t3isp.de
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: nginx
                port:
                  number: 80
```

```
kubectl apply -f .
```

7. ServiceMonitor

```
nano 05-nginx-servicemonitor.yaml
```

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: nginx
  namespace: web-demo
  labels:
    release: prometheus # muss zu Helm-Werten passen!
spec:
  selector:
    matchLabels:
      app: nginx
  namespaceSelector:
    matchNames:
      - web-demo
  endpoints:
    - port: metrics
      path: /metrics
      interval: 15s
```

```
kubectl apply -f .
```

```
## Welches Label prometheus hat, könnt ihr prüfen
kubectl -n monitoring get pods -l release=prometheus
```

```
## Ist der ServiceMonitor konfiguriert ?
kubectl -n web-demo get servicemonitor nginx
kubectl -n web-demo get smon nginx
kubectl -n web-demo describe smon nginx
```

8. Targets finden (in web gui)

```
## im Browser Öffnen und nach web-demn suchen
https://prometheus.<du>.do.t3isp.de/targets

## Dann menü links oben ausklappen, ganz runter scrollen
## serviceMonitor/web-demo/nginx/0
## oder
https://prometheus.tln10.do.t3isp.de/targets?pool=serviceMonitor%2Fweb-demo%2Fnginx%2F0
```

9. mit promql abfragen

```
1. Zunächst finden wir heraus, welche labels diese pods haben (siehe Punkt 8)
das sieht nach job="nginx" aus

Jeder ServiceMonitor (z.B. unser, der nginx heisst), wird beim Scrapen als job="<serviceMonitorName>"
automatisch von Kubernetes abgefragt.
```

d.h. wir können Fragen

```
## (gilt dann für alle pods)
up
## gilt für alle pods in einen für bestimmten job
up {job="nginx"}
## gilt für alle pods in einem bestimmten namespace
up {namespace="web-demo"}
## and combining all endpoints with job=nginx in the namespace web-demo
up {job="nginx",namespace="web-demo"}
```

```
## Regular Expressions also work:
up{namespace="web-demo", pod=~"nginx-.*"}
```

```
##Practical - on: https://prometheus1.tln1.do.t3isp.de/query
+ enter:
up {job="nginx"} + Press "Execute"
```

>_ up{job="nginx"}

Execute

Table

Graph

Explain

< Evaluation time >

Load time: 70ms Result series: 3

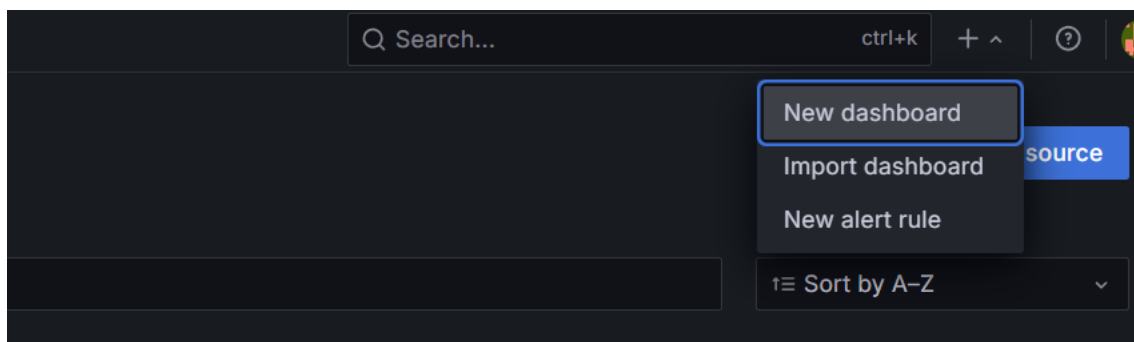
up{container="exporter", endpoint="metrics", instance="192.168.197.5:9113", job="nginx", namespace="web-demo", pod="nginx-69547b6f5-j5c8f", service="nginx"}	1
up{container="exporter", endpoint="metrics", instance="192.168.46.3:9113", job="nginx", namespace="web-demo", pod="nginx-69547b6f5-httq6", service="nginx"}	1
up{container="exporter", endpoint="metrics", instance="192.168.228.68:9113", job="nginx", namespace="web-demo", pod="nginx-69547b6f5-p4fjy", service="nginx"}	1

You can not also click on Graph

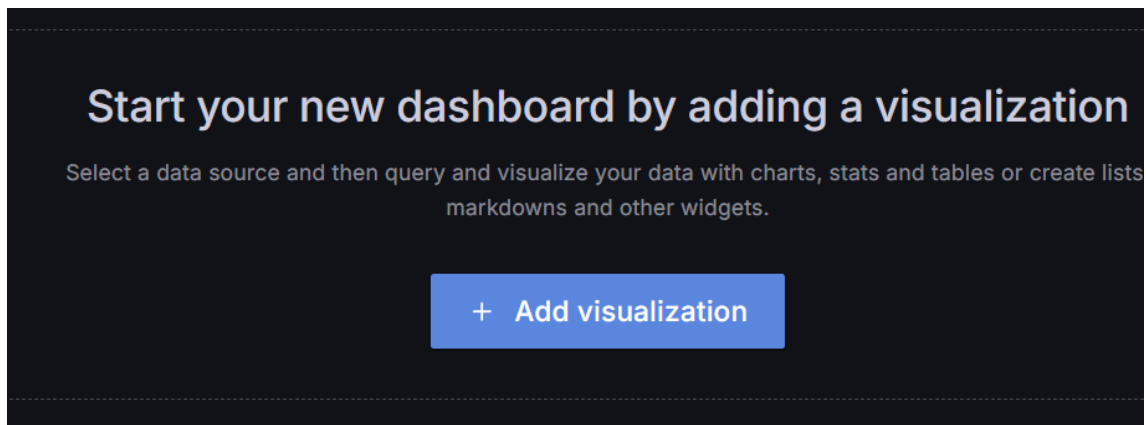
10. In Grafana ein Dashboard erstellen

Step 1: New Dashboard

Oben rechts auf neues Dashboard erstellen klicken
-->

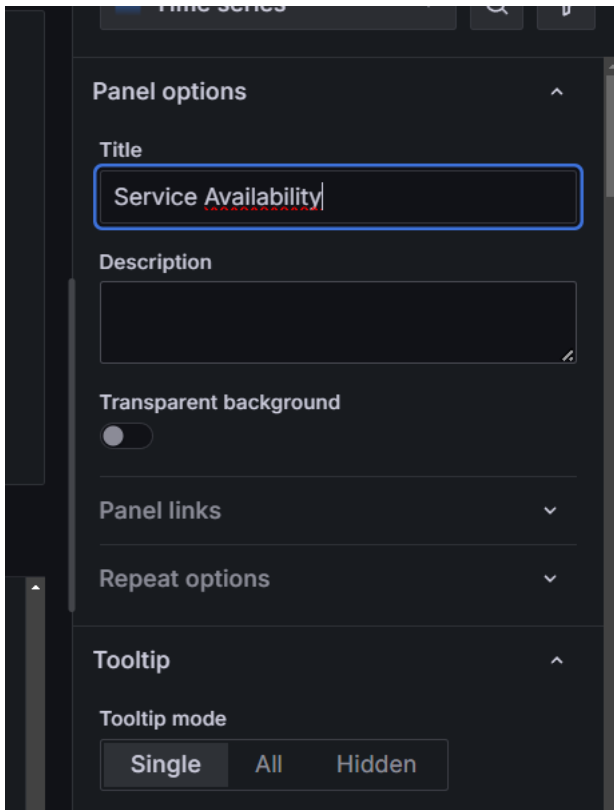


Step 2: Add Visualization

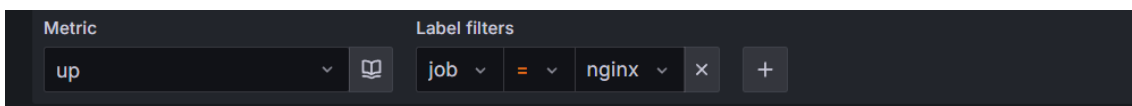


Step 3: Datasource -> Prometheus (Default) auswählen / Visualisation + Query definieren

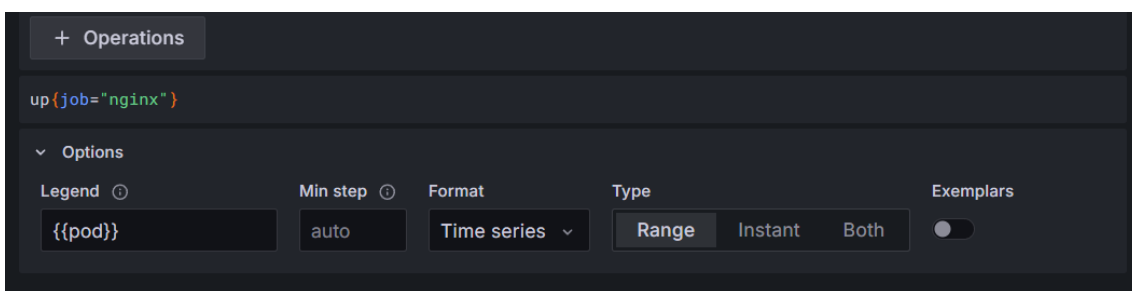
- DataSource Prometheus is bereits vorkonfiguriert



- Choose Visualisation (Stat, Gauge, or Bar Gauge)
- Set the query: `up | job | nginx`
- run query



- Damit immer der pod angezeigt wird, trage dies als custom label unter der query ein



Step 4: Save Dashboard (Button oben rechts)

Logging-Stack: EFK (Elasticsearch/Fluentd/Kibana)

EFK-Stack: Aufbau, Fluentd vs. Fluent Bit, DaemonSet vs. Sidecar

Kein Walkthrough - hier geht es um den Aufbau eines zentralen Logging-Stacks in Kubernetes und die Architektur-Entscheidungen dahinter: Welche Komponenten braucht man, ist Fluentd noch zeitgemäss, und läuft der Log-Collector als DaemonSet oder als Sidecar?

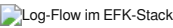
Warum überhaupt zentrales Logging?

- `kubectl logs` liest nur, was die Container-Run-time lokal auf dem Node vorhaelt - stirbt der Pod oder der Node, sind die Logs weg.
- Bei vielen Nodes und Pods will niemand Logs pro Pod einsammeln - man braucht eine zentrale, durchsuchbare Ablage mit Retention.
- Debugging ueber mehrere Services hinweg (wer hat wann welchen Request gesehen?) geht nur mit einer gemeinsamen Sicht.

Die Komponenten (E-F-K)

Buchstabe	Komponente	Aufgabe
E	Elasticsearch	Speichert und indiziert die Logs, macht sie durchsuchbar
F	Fluentd (bzw. heute meist Fluent Bit)	Sammelt die Logs auf den Nodes ein, reichert sie an, leitet sie weiter
K	Kibana	Web-Frontend fuer Suche, Filter und Dashboards auf den Elasticsearch-Indizes

Aufbau: Wie fliesst ein Log durch den Cluster?



Die wichtigsten Punkte daran:

- 1. Die App loggt einfach nach **stdout/stderr** (12-Factor-Prinzip) - sie weiss nichts von Elasticsearch.
- 2. Die Container-Runtime persistiert das pro Node unter `/var/log/pods/`, mit sprechenden Symlinks unter `/var/log/containers/`.
- 3. Der Collector laeuft **einmal pro Node**, mountet diese Verzeichnisse vom Host und liest alle Container-Logs des Nodes.
- 4. Er reichert jede Zeile ueber die Kubernetes-API mit Metadaten an (Namespace, Pod-Name, Labels) - dadurch kann man in Kibana z.B. nach `kubernetes.namespace_name` filtern.
- 5. Er puffert (Backpressure!) und schickt die Daten an Elasticsearch.

Ist Fluentd noch zeitgemaess?

Kurze Antwort: **Deprecated ist Fluentd nicht - aber als Node-Collector fuer einen Neuaufbau meist nicht mehr erste Wahl. Der Nachfolger im eigenen Haus heisst Fluent Bit.**

- **Fluentd** (seit 2011, Ruby + C, CNCF graduated) wird aktiv gepflegt (fluent-package v6 LTS mit Support bis mindestens Ende 2027) und hat ein riesiges Plugin-Oekosystem (1000+). Aber: relativ schwergewichtig (typisch einige hundert MB RAM pro Instanz) - als DaemonSet auf jedem Node zahlt man das mal Anzahl Nodes.
- **Fluent Bit** (gleiches Projekt-Umfeld, in C geschrieben, wenige MB RAM) ist heute der De-facto-Standard als Node-Collector. Wer heute "EFK" aufbaut, meint praktisch immer Elasticsearch + **Fluent Bit** + Kibana.
- Verbreitetes Muster in grossen Umgebungen: Fluent Bit als leichter Collector auf jedem Node, optional ein zentrales Fluentd als **Aggregator** (Routing, aufwendige Filter, viele Ziele) - dort spielt das Plugin-Oekosystem seine Staerke aus, ohne jeden Node zu belasten. Fluent Bit kann inzwischen auch selbst als Aggregator dienen - die zweistufige Architektur ist optional, nicht Pflicht.
- Bestehende Fluentd-Setups laufen zu lassen ist voellig legitim - es gibt keinen Migrationszwang.

Alternativen ausserhalb der Fluent-Familie:

Tool	Einordnung
OpenTelemetry Collector	Sinnvoll, wenn man OTel sowieso fuer Traces/Metriken einsetzt - ein Agent fuer alle drei Signale
Vector	Moderner Collector in Rust (Datadog), sehr performant, eigene Transformationssprache
Filebeat / Elastic Agent	Die Elastic-eigene Variante, eng mit dem Elastic-Stack verzahnt (dann "ELK" statt "EFK")
Promtail / Grafana Alloy	Collector fuer Grafana Loki - anderes Backend-Konzept (Label-Index statt Volltext), guenstiger im Betrieb als Elasticsearch

Mit den "drei Signalen" der Observability sind gemeint:

- 1. **Logs** - Textereignisse mit Zeitstempel ("was ist passiert?") - darum geht es in diesem Kapitel.
- 2. **Metriken** - numerische Zeitreihen ("wie viel / wie schnell?"), z.B. CPU, RAM, Requests/s - siehe Prometheus/Grafana-Kapitel.
- 3. **Traces** - der Weg eines einzelnen Requests durch mehrere Services ("wo haengt's?"), jeder Service steuert einen Span bei.

Klassisch braucht man dafuer drei verschiedene Agenten auf dem Node (z.B. Fluent Bit + Node Exporter + Tracing-Agent) - der OpenTelemetry Collector kann alle drei Signale mit einem einzigen Agenten einsammeln.

DaemonSet oder Sidecar?

Der Standard ist das DaemonSet. Ein Collector pro Node liest die Logs aller Container dieses Nodes:

- Ressourcen: 1 Collector pro **Node** statt 1 pro **Pod** - bei 30 Pods pro Node ist das Faktor 30.
- Zentrale Konfiguration: ein Ort fuer Parsing, Filter, Ziele.
- Die Anwendungen bleiben unangetastet - loggen nach stdout reicht, keine Aenderung an Pod-Specs noetig.

Der Sidecar ist die Ausnahme fuer Sonderfaelle. Dabei laeuft ein zusätzlicher Container im selben Pod, der ueber ein geteiltes `emptyDir` -Volume an die Logs der App kommt. Zwei Varianten:

- 1. **Streaming-Sidecar:** Die App schreibt in Dateien im Container (Legacy-Software, laesst sich nicht auf stdout umbiegen, oder mehrere getrennte Logdateien wie `access.log / error.log / audit.log`). Der Sidecar macht `tail -f` auf die Datei und schreibt sie auf sein eigenes stdout - ab da greift wieder der normale DaemonSet-Weg. Pro Logdatei ein Sidecar, und man kann die Streams in Kibana getrennt filtern.
- 2. **Sidecar mit eigenem Agent:** Der Sidecar ist selbst ein Fluent-Bit/Fluentd und schickt direkt an ein Backend - z.B. wenn ein Team seine Logs an ein **anderes Ziel** schicken muss als der Rest des Clusters, mit eigenem Parsing, ohne die clusterweite Collector-Konfiguration anzufassen. Oder wenn man gar keine DaemonSets ausrollen darf (stark eingeschraenkte Multi-Tenant- oder Managed-Umgebung ohne Zugriff auf Node-Ebene).

Warum man den Sidecar nicht als Default nimmt:

- RAM/CPU-Overhead in **jedem** Pod, nicht einmal pro Node.
- Konfiguration verteilt sich ueber viele Pod-Specs statt an einer Stelle.
- Beim Streaming-Sidecar liegen die Logs doppelt auf der Platte (Datei im Volume + stdout-Kopie der Runtime), und um die Log-Rotation im `emptyDir` muss man sich selbst kuemmern.
- Logs des Sidecar-Agent-Musters (Variante 2) tauchen nicht in `kubectl logs` auf.

--	--	--

Kriterium	DaemonSet	Sidecar
Ressourcenverbrauch	1x pro Node	1x pro Pod
Konfiguration	zentral	pro Pod/Team
App loggt nach stdout	perfekt	unnoetig
App loggt in Dateien	geht nicht direkt	genau dafuer (Streaming-Sidecar)
Abweichendes Log-Ziel pro Team	aufwendig (Routing-Regeln)	einfach (Variante 2)
Kein Node-Zugriff erlaubt	geht nicht	einzige Option

Faustregel: DaemonSet als Default. Sidecar nur gezielt dort, wo eine App nicht nach stdout loggen kann oder Logs ein abweichendes Ziel bzw. eigenes Handling brauchen.

Zusammenfassung

- Aufbau: App -> stdout -> Node-Dateisystem -> Collector (DaemonSet) -> Elasticsearch -> Kibana.
- Merksatz: "EFK" heisst heute praktisch **Elasticsearch + Fluent Bit + Kibana** - Fluentd lebt weiter als optionaler Aggregator und in Bestandsumgebungen.
- Fuer Neuaufbauten auch OpenTelemetry Collector (ein Agent fuer Logs, Metriken und Traces) oder Vector pruefen.
- DaemonSet ist der Normalfall, Sidecar das Werkzeug fuer Ausnahmen - und man sollte begrunden koennen, warum man es einsetzt.

Referenzen

- <https://kubernetes.io/docs/concepts/cluster-administration/logging/> (offizielle Doku zu genau diesen Mustern: Node-Agent, Streaming-Sidecar, Sidecar mit Agent)
- <https://www.fluentd.org/architecture>
- <https://docs.fluentbit.io/manual/about/fluentd-and-fluent-bit>

Alternative: Splunk-Integration

Theorie: Kubernetes mit Splunk verbinden

Was ist Splunk

Eine Plattform zum Einsammeln, Durchsuchen und Korrelieren von Maschinendaten — Logs, Metriken, Events. Schema-on-read: Rohdaten werden zuerst indexiert, die Struktur wird erst bei der Suche extrahiert.

Gegründet	2003, San Francisco — der Name kommt von „spelunking“, dem Erforschen von Höhlen
Lizenz	Proprietär. Trial 60 Tage, danach automatisch Free License (500 MB/Tag, Single-User)
Typische Daten	App-Logs, Syslog, Security-Events, Kubernetes-Events, Metriken via HEC (HTTP Event Collector)
Abfragesprache	SPL (Search Processing Language) — mächtiger als reine Volltextsuche

Wie kann ich Splunk betreiben (Architektur)

Splunk laesst sich als **Standalone**-Instanz (eine Instanz uebernimmt Indexer, Search Head und Web-UI gleichzeitig), als **verteiltes System** (separate Indexer-Cluster, Search-Head-Cluster, Lizenz-Manager), als **Cloud-Service** (Splunk Cloud Platform, gehostet von Splunk) oder **in Kubernetes** (via Splunk Operator) betreiben.

Fuer das Training arbeiten wir mit Splunk Standalone. Der Server ist bereits eingerichtet und erreichbar unter <https://splunk-external.do.t3isp.de>.

Die Anbindung: Splunk extern, Kubernetes liefert nur zu

Der Kernpunkt dieser Anbindung: Splunk läuft **außerhalb** des Kubernetes-Clusters, auf eigener Infrastruktur (dediziertem Server oder, wie hier zum Üben, einer einzelnen VM). Der Cluster selbst enthält nur einen leichten Log-Forwarder (DaemonSet), der Container-Logs und Kubernetes-Events einsammelt und per HEC (HTTP Event Collector, Port 8088) an die externe Splunk-Instanz überträgt.

 Architektur: Splunk extern auf eigener VM

Unser Setup:

- Kubernetes-Cluster** — Logs werden per DaemonSet (ein Pod auf jedem Node) geforwarded
- Splunk-VM** — bereits ausgerollt (per Terraform/Cloud-Init)

Hands-on: externe Anbindung

Schritt-für-Schritt-Übungen zur externen Variante:

- (Optional) [Externe Splunk-VM per Terraform aufsetzen](#)
- [Log-Forwarder an die externe Splunk-VM anbinden](#)
- [Log-Suche und CrashLoopBackOff-Debugging](#)
- [CrashLoopBackOff-Alert einrichten \(optional\)](#)
- [Aufräumen](#)

Welche Splunk-Menuepunkte davon in der Uebung tatsaechlich vorkommen (und welche bewusst aussen vor bleiben, weil sie den Betrieb von Splunk selbst statt die Kubernetes-Anbindung betreffen): [menues-und-ihre-funktion.md](#).

Funktionsuebersicht: Splunk-Menuepunkte und Kubernetes-Relevanz

Was Splunk Enterprise alles kann, zugeordnet zu den echten Menuepunkten der Web-UI - und die Einschaeztung, ob das jeweilige Feature fuer *dieses* Kubernetes-Training gebraucht wird.

App-Auswahl (linke Seitenleiste)

App	Was sie tut	Fuer K8s-Training gebraucht?
Search & Reporting	Kernanwendung: Suche, Dashboards, Alerts, Reports	Ja - das ist die App, in der die ganze Uebung 4 stattfindet
Audit Trail	Protokolliert Zugriffe/Aenderungen auf die Splunk-Instanz selbst	Nein - Audit der Splunk-Administration, nicht des Clusters
Data Management	Verwaltung von Splunk-Cloud-/Edge-Processor-Pipelines	Nein - zielt auf Splunk-eigene Cloud-Infrastruktur, nicht relevant fuer eine einzelne Standalone-VM
Discover Splunk Observability Cloud	Bewirbt das separate SaaS-Produkt Splunk Observability Cloud (APM/Infra-Monitoring)	Nein - anderes, kostenpflichtiges Produkt, kein Teil dieser Uebung
Splunk Secure Gateway	Koppelt die Splunk-Mobile-App per QR-Code an die Instanz	Nein - Mobile-Zugriff ist fuer eine Trainingsumgebung ohne Mehrwert
Upgrade Readiness App	Prueft Apps/Konfiguration vor einem Splunk-Versions-Upgrade	Nein - Instanz wird nach der Uebung wieder abgebaut, kein Upgrade-Pfad noetig

Search & Reporting App (linke Symbolleiste in der App)

Menuepunkt	Was er tut	Fuer K8s-Training gebraucht?
Search	SPL-Suche gegen den Index, Kern-Feature	Ja - zentral in Uebung 4 , Schritte 3+4
Analytics Workspace	Klick-basierte Alternative zu SPL (Pivot-Nachfolger) fuer Nutzer ohne SPL-Kenntnisse	Nein - Trainingsziel ist SPL selbst zu ueben, nicht der Klick-Weg drumherum
Datasets	Verwaltung von Data Models/Table Datasets als wiederverwendbare Datenbasis fuer Pivot	Nein - Aufbauthema, ueberschneidet sich mit Analytics Workspace, nicht im Scope
Reports	Gespeicherte Suchen mit Zeitplan, Ergebnis-Export	Nein direkt - waere ein sinnvoller naechster Schritt nach Uebung 4, aber kein eigener Uebungsinhalt
Alerts	Bedingte Benachrichtigung aus einer Suche heraus (E-Mail, Webhook, Skript)	Ja - eigene, optionale Uebung 5 (BackOff-Alert)
Dashboards	Visualisierungen/Panels aus gespeicherten Suchen	Optional - "Visualize your data" wird auf der Startseite beworben, ist aber keine eigene Uebung; waere naeheliegende Erweiterung fuer ein Kubernetes-Log-Dashboard
Modules	SPL2-Suchmodule (mehrere Suchen kombinieren, neueres API-Konzept)	Nein - SPL2 ist ein Splunk-internes Nachfolgekonzept zu SPL, kein Kubernetes-Bezug

Was ist SPL2: die von Splunk geplante Nachfolgesprache zu SPL - naeher an SQL/Pipe-Syntax aus einem Guss, gedacht um Suchbausteine als wiederverwendbare "Module" zu buendeln und gleichermassen in Splunk Cloud, Splunk Enterprise und Data-Pipeline-Produkten (z.B. Edge Processor) einsetzbar zu sein. Fuer dieses Training ohne Bedeutung: die Uebungen nutzen durchgehend klassisches SPL, siehe [Uebung 4](#).

Screenshots: jeder Menuepunkt einzeln

Jeder Punkt der linken Symbolleiste angeklickt, mit rotem Rahmen um das jeweilige Icon markiert (Screenshots aus der laufenden Instanz, Stand 31.08.2026):

Search  Search SPL-Eingabefeld plus Suchhistorie - der Standard-Einstiegspunkt der App. Sinnvoll fuer die Kubernetes-Anbindung: ja, das ist der zentrale Arbeitsplatz aus [Uebung 4](#).

Analytics Workspace  Analytics Workspace Klick-basiertes Analyse-Interface (Metriken/Datasets per Drag-and-Drop statt SPL zu tippen). Sinnvoll fuer die Kubernetes-Anbindung: nein - das Training vermittelt bewusst SPL direkt, dieser Weg drumherum ist kein Uebungsinhalt.

Datasets  Datasets Verwaltung wiederverwendbarer, strukturierter Datensichten (Basis fuer Analytics Workspace/Pivot). Sinnvoll fuer die Kubernetes-Anbindung: nein - Aufbauthema ohne eigenen Uebungsschritt.

Reports  Reports Liste gespeicherter Suchen mit Zeitplan (hier bereits 8 vorinstallierte Splunk-Standardreports zu sehen, z.B. "Errors in the last 24 hours" - keiner davon Kubernetes-spezifisch). Sinnvoll fuer die Kubernetes-Anbindung: nein direkt - waere aber der naeheliegende naechste Schritt, um die BackOff-Suche aus Uebung 4 dauerhaft zu speichern.

Alerts  Alerts Liste aller konfigurierten Alerts (Bedingung -> Aktion). Sinnvoll fuer die Kubernetes-Anbindung: ja - hier taucht der optionale BackOff-Alert aus [Uebung 5](#) auf, falls angelegt.

Dashboards  Dashboards Uebersicht/Neuanlage von Dashboards (Dashboard Studio oder Classic/Simple-XML). Sinnvoll fuer die Kubernetes-Anbindung: optional - kein Pflichtschritt der Uebung, aber naeheliegende Erweiterung fuer ein CrashLoopBackOff-Uebersichts-Dashboard.

Modules  Modules SPL2-Suchmodule. Auf dieser Instanz tatsaechlich nicht nutzbar - Splunk meldet "Unable to access SPL2 modules because the required 'data orchestrator' component is not available." Sinnvoll fuer die Kubernetes-Anbindung: nein - SPL2 ist ein separates, neueres Splunk-Suchkonzept ohne Kubernetes-Bezug, und auf dieser Standalone-Instanz fehlt ohnehin die dafuer noetige Zusatzkomponente.

Activity-Menue (oben, Symbol neben der Suchlupe)

Menuepunkt	Was er tut	Fuer K8s-Training gebraucht?
Jobs	Laufende/abgeschlossene Suchjobs verwalten (abbrechen, Ergebnisse nachladen)	Nein direkt - nuetzlich falls eine Suche in Uebung 4 haengt, aber kein eigener Uebungsinhalt
Triggered Alerts	Historie ausgeloester Alerts	Nein direkt - haengt am optionalen Alert aus Uebung 5

Fazit

Fuer dieses Training zaehlen im Kern nur wenige Menuepunkte wirklich: **Search** und **Alerts** aus der Search & Reporting App - das deckt den Weg "Log/Event trifft in Splunk ein -> wird per SPL gefunden -> loest optional einen Alert aus" ab, den [Uebung 4](#) und [Uebung 5](#) tragen. Analytics Workspace, Datasets, Reports und Modules sind Aufbau- bzw. Alternativkonzepte ohne eigenen Uebungsschritt, Dashboards eine naheliegende, aber optionale Erweiterung. Das Settings-Menue (Administration der Splunk-Instanz selbst: Server, Lizenz, Indizes, Forwarding, Clustering, Nutzer/Rollen) ist bewusst nicht Teil dieser Uebersicht - es betrifft den Betrieb von Splunk als Produkt, nicht die Bedienung durch einen Kubernetes-Nutzer, und wird stattdessen ueber die IaC-Schritte in [Uebung 2](#) und [Uebung 3](#) automatisiert statt per Klick konfiguriert.

Log-Forwarder an externen Splunk-Server anbinden

Hintergrund

 Warum ein Log-Forwarder: der Weg eines Log-Eintrags

Diese Uebung installiert den **Splunk OpenTelemetry Collector** als DaemonSet und richtet ihn auf den HEC-Endpoint der externen VM aus Uebung 2 aus.

► Ausführlicher Text (optional)

Schritt 1: Arbeitsverzeichnis anlegen

Alle Dateien dieser Uebung landen in einem eigenen Verzeichnis, die folgenden Schritte werden von dort ausgeführt:

```
cd
mkdir -p helm-values/splunk-otel
cd helm-values/splunk-otel
```

Schritt 2: Token in eine Secret-Values-Datei eintragen

Den Token **nicht** per `--set` auf der Kommandozeile uebergeben (landet sonst in der Bash-History und ggf. in `helm history`), sondern in eine eigene, kleine YAML-Datei schreiben. Diese Datei bleibt nur lokal auf dem Bastion und ist die einzige Stelle, an der der Token im Klartext steht:

```
nano hec-token-values.yml
```

Inhalt:

```
splunkPlatform:
  token: <HEC-Token - gibt der Trainer bekannt; bei selbst aufgesetzter VM: TF_VAR_splunk_hec_token aus Uebung 2 Schritt 1>
```

Schritt 3: Collector-Konfiguration als Values-Datei anlegen

Der Rest der Konfiguration ist nicht geheim und kommt in eine zweite Values-Datei. Den folgenden Block als Ganzes ins Terminal kopieren - `$(whoami)` ersetzt die Shell dabei automatisch durch den eigenen Bastion-Username. Der dient als eindeutige Cluster-Kennung: alle Teilnehmer senden an dieselbe Splunk-Instanz in denselben Index, und ueber das Feld `k8s.cluster.name`, das der Forwarder aus `clusterName` erzeugt und an jedes Event anhaengt, lassen sich die eigenen Daten spaeter wieder herausfiltern:

```
cat > collector-values.yml << EOF
clusterName: $(whoami)
splunkPlatform:
  # HEC-Endpoint der externen Splunk-VM aus Uebung 2
  endpoint: "https://splunk-external.do.t3isp.de:8088/services/collector"
  index: main
  # HEC (Port 8088) nutzt Splunks eigenes, selbstsigniertes Zertifikat -
  # das Let's-Encrypt-Zertifikat gilt nur fuer die Web-UI hinter Nginx
  insecureSkipVerify: true
EOF
cat collector-values.yml
```

Die erste Zeile der Ausgabe muss den eigenen Username zeigen (Beispiel Teilnehmer 5: `clusterName: tln5`).

Schritt 4: Helm-Repo hinzufuegen

```
helm repo add splunk-otel https://signalfx.github.io/splunk-otel-collector-chart
helm repo update
```

Schritt 5: Forwarder installieren

```
kubectl create namespace splunk-forwarder
helm install splunk-log-forwarder \
  -f collector-values.yml \
  -f hec-token-values.yml \
```

```
splunk-otel/splunk-otel-collector \
-n splunk-forwarder
```

Der Cluster enthaelt damit **nur** den Forwarder - kein Splunk-Operator, keine Splunk-Instanz. Der HEC-Endpoint in `collector-values.yml` zeigt auf die externe VM aus [Uebung 2](#).

Jedes Event traegt damit zwei Kennungen, ueber die sich in [Uebung 4](#) die eigenen Daten von denen der anderen Teilnehmer trennen lassen: die Cluster-Kennung aus Schritt 3 (Feld `k8s.cluster.name`, z.B. `tltn5`) und automatisch den echten Node-Hostnamen im `host`-Feld (die Nodes heissen `k8s-<username>-cp`, `k8s-<username>-w1`, ..., z.B. `k8s-tltn5-cp`).

Schritt 6: Status pruefen

```
kubectl get pods -n splunk-forwarder
kubectl logs -n splunk-forwarder -l app=splunk-otel-collector --tail=20
```

Erwartete Ausgabe: `DaemonSet-Pods` `Running`, keine `Exporting failed`-Meldungen. Falls doch:

- Endpoint in `collector-values.yml` gegen `terraform -chdir=terraform-external-splunk output pruefen`
- HEC (Port 8088) verlangt HTTPS, auch wenn andere Ports auf der VM ggf. nur HTTP sprechen
- `401 / 403` deutet auf einen falschen Token hin (Schritt 2 wiederholen, Token beim Trainer gegenpruefen)

Schritt 7: Aufräumen der Secret-Datei (optional, aber empfohlen)

Sobald der Forwarder laeuft, kann die lokale Token-Datei geloescht werden - der Token steckt danach nur noch im Kubernetes-Secret, das der Helm-Release selbst angelegt hat:

```
rm ~/helm-values/splunk-otel/hec-token-values.yml
```

Die `collector-values.yml` aus Schritt 3 enthaelt kein Secret und kann liegen bleiben. Fuer eine spaetere Aenderung (`helm upgrade`) einfach Schritt 2 wiederholen.

Weiter mit [Uebung 4: Log-Suche und CrashLoopBackOff-Debugging](#), um den Forwarder mit echten Daten zu fuehlen und in der Splunk-Web-UI zu suchen.

Abstuerzenden Pod ueber Splunk debuggen (CrashLoopBackOff)

Hintergrund

 Vom Absturz zur Ursache: Debugging ueber die zentrale Splunk-Suche

Wenn ein Pod wiederholt abstuerzt und neu gestartet wird (`CrashLoopBackOff`), sind seine Logs mit reinem `kubectl` schwer nachzuvollziehen: `kubectl logs` zeigt nur den aktuellen und den letzten Container-Versuch, alle aelteren Restarts sind verloren. Mit dem zentralen Log-Forwarder aus Uebung 3 landet dagegen **jeder** Neustart dauerhaft auf der externen Splunk-Instanz - inklusive der Kubernetes-Events, die den Grund fuer den Neustart dokumentieren. Faellt der Cluster spaeter komplett aus, bleiben die bereits gesendeten Logs dort erhalten - genau das Sammelbecken-Prinzip aus der [Architektur-Uebersicht](#).

Szenario: ein Service `payment-service` kann seine Datenbank nicht erreichen und beendet sich deshalb beim Start immer wieder selbst.

Schritt 1: Demo-Deployment anlegen und ausrollen

Arbeitsverzeichnis anlegen:

```
cd
mkdir -p manifests/crashloop-demo
cd manifests/crashloop-demo
```

```
nano 01-crashloop-demo.yml
```

Das Manifest per Copy & Paste im Editor anlegen (wichtig: im Editor, nicht per `cat` -Heredoc - die `$(date ...)`-Aufrufe gehoeren zum Container und duerfen nicht schon lokal von der Shell ersetzt werden):

```
## vi 01-crashloop-demo.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payment-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: payment-service
  template:
    metadata:
      labels:
        app: payment-service
    spec:
      containers:
        - name: payment-service
          image: busybox:1.36
          command:
            - sh
            - -c
            - |
              echo "$(date -Iseconds) INFO payment-service startet..."
```

```
echo "$(date -Iseconds) INFO Verbinde zu Datenbank db-payments.internal:5432 ..."  
sleep 2  
echo "$(date -Iseconds) ERROR Verbindung zu db-payments.internal:5432 fehlgeschlagen: Connection refused"  
echo "$(date -Iseconds) ERROR payment-service kann ohne Datenbankverbindung nicht starten, beende Prozess"  
exit 1
```

Ausrollen:

```
kubectl create namespace crashloop-demo  
kubectl apply -f . -n crashloop-demo
```

Schritt 2: CrashLoopBackOff beobachten

```
kubectl get pods -n crashloop-demo -w
```

Erwartete Ausgabe nach ca. 30-60 Sekunden:

NAME	READY	STATUS	RESTARTS	AGE
payment-service-f94fcbf65-d8gpz	0/1	CrashLoopBackOff	2 (17s ago)	26s

```
kubectl get events -n crashloop-demo --sort-by=.lastTimestamp | tail -5
```

Zeigt u.a. Warning BackOff ... Back-off restarting failed container payment-service .

Schritt 3: In der Splunk-Web-UI nach den Kubernetes-Events suchen

Login unter <https://splunk-external.do.t3isp.de> - die Zugangsdaten (admin + Passwort) gibt der Trainer bekannt (Details zur VM: [Uebung 2, Schritt 5](#)).

Suche in Splunk Web - bei mehreren angebundenen Clustern - (Suche > Neue Suche):

```
## <tlN> durch Deine Teilnehmer-Nr ersetzen, z.B. tln1  
## index=main k8s.container.name="payment-service" k8s.cluster.name="tln1"  
index=main k8s.container.name="payment-service" k8s.cluster.name="<dein-username>"
```

Alternativ ueber den Node-Hostnamen im host -Feld - die Node-Namen enthalten den eigenen user (tlN) (k8s-tln5-cp , k8s-tln5-w1 ,...):

```
index=main k8s.container.name="payment-service" host="k8s-<tlN>-*"
```

 Kubernetes BackOff-Event in Splunk

Das Event Back-off restarting failed container payment-service in pod ... ist der kube:events -Sourcetype - stammt direkt von der Kubernetes-API, nicht aus den Container-Logs selbst. Das allein sagt aber noch nicht, *warum* der Container abstuerzt.

Schritt 4: Die eigentliche Fehlerursache in den Container-Logs finden

Breitere Suche (auch aeltere, laengst rotierte Container-Log-Dateien sind hier noch vorhanden, weil sie zentral in Splunk liegen statt nur auf dem Node):

```
index=main payment k8s.cluster.name="<dein-username>"
```

 Container-Log-Zeilen mit der Fehlerursache

Sourcetype kube:container:payment-service zeigt die eigentlichen stdout-Zeilen:

```
INFO payment-service startet...  
INFO Verbinde zu Datenbank db-payments.internal:5432 ...  
ERROR Verbindung zu db-payments.internal:5432 fehlgeschlagen: Connection refused  
ERROR payment-service kann ohne Datenbankverbindung nicht starten, beende Prozess
```

Damit ist die Ursache klar, ganz ohne dass kubectl logs nach dem naechsten Neustart noch Zugriff auf genau diese Zeilen haette - in Splunk bleiben sie durchsuchbar, auch Tage spaeter und ueber beliebig viele Neustarts hinweg. Und: der Cluster selbst muss dafuer nicht einmal mehr laufen, da die Daten schon auf der externen VM liegen.

Praxis-Hinweis: Ein sehr kurzlebiger Container (hier: stirbt nach ~2 Sekunden) kann in der Splunk-Suche kurzzeitig fehlen, wenn der Log-Forwarder die neue Log-Datei noch nicht entdeckt hat (Poll-Intervall). Bei Bedarf 1-2 Neustarts abwarten und die Suche wiederholen.

Der crashloop-demo -Namespace bleibt fuer die naechste Uebung noch bestehen - der weiter abstuerzende payment-service liefert dort die Daten fuer den Alert.

Weiter mit [Uebung 5: CrashLoopBackOff-Alert einrichten](#), um Splunk bei zu vielen Neustarts automatisch Bescheid geben zu lassen.

CrashLoopBackOff-Alert einrichten (optional)

Hintergrund

 Vom BackOff-Zaehler zur Benachrichtigung: wie der Alert entsteht

Schritt 1: Die Suche fuer den Alert bauen

Ausgangspunkt ist eine eigene, praezise Suche - nicht die aus Uebung 4, denn dort ging es nur um Anzeigen, hier soll konkret **gezaehlt** und **gefiltert** werden. In Splunk Web (Suche > Neue Suche):

```
index=main sourcetype=kube:events k8s.event.reason=BackOff k8s.container.name="payment-service" k8s.cluster.name="<dein-username>"
k8s.event.count>2
```

<dein-username> wieder durch den eigenen Bastion-Username ersetzen (z.B. `tl5`) - wie in Übung 3/4 begrenzt das die Suche auf den eigenen Cluster, wenn mehrere Teilnehmer gleichzeitig gegen dieselbe externe Splunk-Instanz arbeiten. Ohne dieses Feld wuerde der Alert spaeter auf Neustarts **irgendeines** Teilnehmer-Clusters reagieren, nicht nur auf den eigenen.

Was die einzelnen Teile der Suche tun:

Teil	Zweck
<code>sourcetype=kube:events</code>	nur Kubernetes-Events, keine Container-Log-Zeilen
<code>k8s.event.reason=BackOff</code>	nur echte Neustart-Ereignisse, nicht z.B. <code>Pulled/Created</code>
<code>k8s.container.name="payment-service"</code>	nur dieser eine Container
<code>k8s.cluster.name="<dein-username>"</code>	nur der eigene Cluster
<code>k8s.event.count>2</code>	der eigentliche Schwellwert - Kubernetes' eigener Wiederholungszaehler

Oben rechts den Zeitraum auf **Letzte 15 Minuten** stellen (`Letzte 5 Minuten` gibt es im Zeitraum-Dropdown nicht als Preset - nur unter "Echtzeit", das hier nicht gewollt ist). Ausfuehren - solltet ihr noch im Anschluss an Übung 4 sein, liefert die Suche sofort mindestens ein Ergebnis (der Demo-Container crasht dort bereits laenger als eine Minute).

Schritt 2: Als Benachrichtigung speichern

Nach dem Ausfuehren oben rechts ueber der Ergebnisliste auf **Speichern als** klicken und **Benachrichtigung** waelhen.

Schritt 3: Einstellungen im Dialog "Als Benachrichtigung speichern"

 Alert-Dialog, oberer Teil: Titel, Zeitplan, Cron-Ausdruck und Trigger-Bedingung erklart

- **Titel:** z.B. `payment-service-backoff-alert`
- **Berechtigungen:** `Privat` reicht fuer die Übung
- **Benachrichtigungstyp:** `Geplant` (nicht `Echtzeit` - braucht mehr Ressourcen/Lizenz)
- Zeitplan-Dropdown (Standard "Jede Woche ausfuehren") auf **Nach Cron-Zeitplan ausfuehren** umstellen - die Presets bieten nur Stunde/Tag/Woche/Monat, fuer den gewünschten 5-Minuten-Takt braucht es einen eigenen Cron-Ausdruck
- **Zeitspanne:** `Letzte 15 Minuten` (aus Schritt 1 uebernommen)
- **Cron-Ausdruck:** `*/* * * * *` (alle 5 Minuten pruefen)
- **Trigger-Bedingungen** > "Benachrichtigung ausloesen, wenn": Standardwert `Anzahl der Ergebnisse groesser ist als 0` **unveraendert lassen** - die eigentliche Filterung (`k8s.event.count>2`) steckt schon in der Suche selbst
- **Trigger:** `Ein Mal` reicht (ein Treffer pro Lauf genuegt zum Ausloesen)
- **Einschraenkung aktivieren** (Checkbox) - **wichtig:** ohne sie feuert der Alert sonst alle 5 Minuten erneut, solange derselbe Pod weiter crasht. Danach "Ausloesung unterdruecken fuer" auf `30 Minute(n)` stellen

Schritt 4: Aktion hinzufuegen - wohin geht die Benachrichtigung wirklich?

 Alert-Dialog, unterer Teil: Einschraenkung, Aktionen und wohin die Benachrichtigung tatsaechlich geht

Ohne diesen Schritt passiert beim Ausloesen des Alerts gar nichts sichtbar - "Aktionen ausloesen" legt erst fest, wohin die Benachrichtigung tatsaechlich geht.

Unter **Aktionen ausloesen** auf **+ Aktionen hinzufuegen** klicken und **Zu "Ausgeloeste Benachrichtigungen" hinzufuegen** waelhen. Das ist die einzig sinnvolle Aktion auf dieser Trainings-VM: `E-Mail senden` waere die naehliegende Ergaenzung, wurde live geprueft und ist **nicht funktionsfaehig** - unter Einstellungen > E-Mail-Einstellungen steht als Mailhost nur der unkonfigurierte Standardwert `localhost`, es laeuft kein Mailserver auf der VM. Die Benachrichtigung bleibt also bewusst **innerhalb von Splunk:** sichtbar nur, wer sich einloggt und nachschaut.

Optional **Schweregrad** setzen (Info/Gering/Mittel/Hoch/Kritisch) - fuer einen `CrashLoopBackOff` ist `Hoch` angemessen.

Schritt 5: Speichern und Ergebnis pruefen

Speichern klicken. Splunk zeigt dabei die Warnung *"This scheduled search will not run after the Splunk Enterprise Trial License expires."* - diese Splunk-Instanz laeuft auf einer Trial-Lizenz, fuer die Dauer des Trainings ist das unproblematisch, die Warnung also ignorieren.

Ergebnis pruefen: links in der Navigation von Search & Reporting auf **Benachrichtigungen** - der neue Alert erscheint dort mit Status `Aktiviert` und dem naechsten geplanten Zeitpunkt. Nach dem naechsten Lauf (max. 5 Minuten warten) taucht er unter dem Aktivitaets-Symbol oben rechts (Pulslinie) > **Ausgeloeste Benachrichtigungen** auf - Spalten **Schweregrad**, **Typ** (`Geplant`) und **Modus** (`Digest`). Ueber **Ergebnisse anzeigen** in der Zeile laesst sich das genaue Event nachvollziehen, das den Alert ausgeloest hat.

Schritt 6: Aufräumen nach der Übung

Da alle Teilnehmer dieselbe externe Splunk-Instanz nutzen, den eigenen Alert danach wieder loeschen - **Benachrichtigungen**, Zeile des Alerts > **Bearbeiten** > **Loeschen**.

Anschliessend den Demo-Namespace aus Übung 4 abbauen:

```
kubectl delete namespace crashloop-demo
```

Fuer den vollstaendigen Abbau der gesamten Uebungsreihe (Forwarder, externe VM, Cluster) weiter mit [Übung 6: Aufräumen](#).

Optional: Splunk im Cluster betreiben (Splunk Operator)

Optionaler Anhang. Diese Übung und die folgenden (10-14) betreiben Splunk selbst **im Cluster**, als Alternative zur externen VM aus Übung 2. Fuer den Praxisbetrieb ist das der seltenere Weg (siehe [splunk-im-cluster-optional.md](#)) - hier aber lehrreich, weil sichtbar wird, was der Splunk Operator im Hintergrund automatisiert.

Hintergrund

Splunk wird nicht direkt als Deployment betrieben, sondern ueber den offiziellen [Splunk Operator](#) verwaltet. Der Operator bringt eigene Custom Resources mit (z.B. `Standalone`, `IndexerCluster`, `SearchHeadCluster`) und kuemmert sich um Lifecycle, Storage und Konfiguration von Splunk Enterprise im Cluster.

Schritt 1: Custom Resource Definitions installieren

Die CRDs sind groesser als 1 MB, deshalb liefert Helm sie nicht mit aus. Sie muessen separat per `kubectl apply --server-side` installiert werden.

```
kubectl apply --server-side -f https://github.com/splunk/splunk-operator/releases/download/3.1.0/splunk-operator-crd.yaml
kubectl get crd | grep splunk
```

Erwartete Ausgabe: mehrere CRDs mit `splunk.com` im Namen (z.B. `standalones.enterprise.splunk.com`).

Schritt 2: Helm-Repo hinzufuegen

```
helm repo add splunk https://splunk.github.io/splunk-operator/
helm repo update
```

Schritt 3: Namespace anlegen und Operator installieren

Der Operator akzeptiert beim Start die Splunk General Terms (SGT) - ohne das bleibt spaeter jede Splunk-Instanz im Status `Error: license not accepted` haengen.

```
kubectl create namespace splunk-operator
helm install splunk-operator-test splunk/splunk-operator -n splunk-operator \
  --set splunkOperator.splunkGeneralTerms="--accept-sgt-current-at-splunk-com"
```

Schritt 4: Operator-Status pruefen

```
kubectl get pods -n splunk-operator
kubectl logs -n splunk-operator -l app.kubernetes.io/name=splunk-operator --tail=30
```

Erwartete Ausgabe: ein Pod `splunk-operator-controller-manager-...` im Status `Running`, 1/1.

Weiter mit [Uebung 11: Splunk Standalone deployen](#).

Troubleshooting

Debugging von Pods (Logs, Events, typische Fehlerbilder)

How ?

1. Which pod is in charge
2. Problems when starting: `kubectl describe po mypod`
3. Problems while running: `kubectl logs mypod`

kubectl debug - Ephemeral Container

Walkthrough Debug Container

```
kubectl run ephemeral-demo --image=registry.k8s.io/pause:3.1 --restart=Never
kubectl exec -it ephemeral-demo -- sh

kubectl debug -it ephemeral-demo --image=busybox
```

Example with nginx

```
kubectl run --image=nginx nginx
### debug this container
kubectl debug -it nginx --image=busybox
```

```
## processe des original containers anzeigen
## z.B. nginx
## name des containers rausfinden
kubectl debug -it nginx --target=nginx --image=busybox
```

Walkthrough Debug Node

```
kubectl get nodes
## so auch root-rechte auf node
kubectl debug node/mynode -it --profile=sysadmin --image=ubuntu
```

Reference

- <https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/#ephemeral-container>

Host/Node erforschen mit kubectl debug (z.B. CNI)

```
kubectl debug node/worker1 -it --image=ubuntu
## in der bash
cd /host/etc/cni/net.d
ls -la
apt update
apt install jq # im container
cat 10-calico.conflist | jq
```

ClusterIP debuggen

Situation

- Kein Zugriff auf die Nodes, zum Testen von Verbindungen zu Pods und Services über die PodIP/ClusterIP

Lösung

```
## Wir starten eine Busybox und fragen per wget und port ab
## busytester ist der name
## long version
kubectl run -it --rm --image=busybox busytester
## wget <pod-ip-des-ziels>
## exit

## quick and dirty
kubectl run -it --rm --image=busybox busytester -- wget <pod-ip-des-ziels>
```

Service Mesh - Istio & Envoy verstehen

Einfuehrung in Istio & Service-Mesh-Architekturen

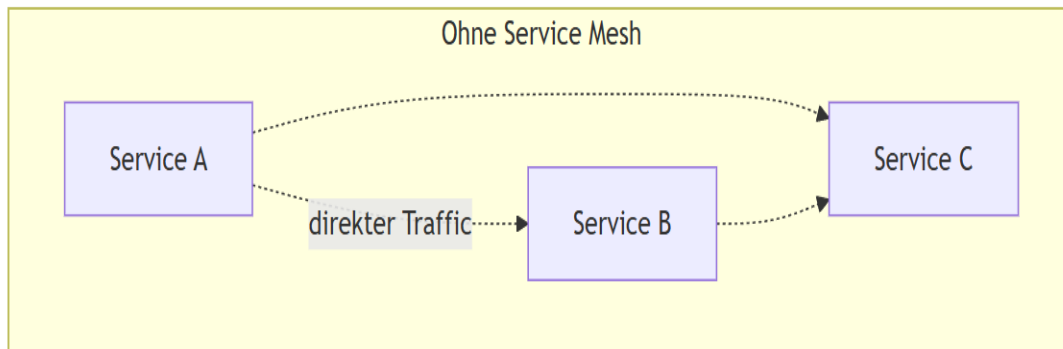
Was ist ein Service Mesh?

- Dedizierte Infrastrukturschicht für Service-zu-Service-Kommunikation
- Transparente Zwischenschicht ohne Code-Änderungen
- Zentrale Steuerung von Retry, Timeout, Verschlüsselung, Monitoring

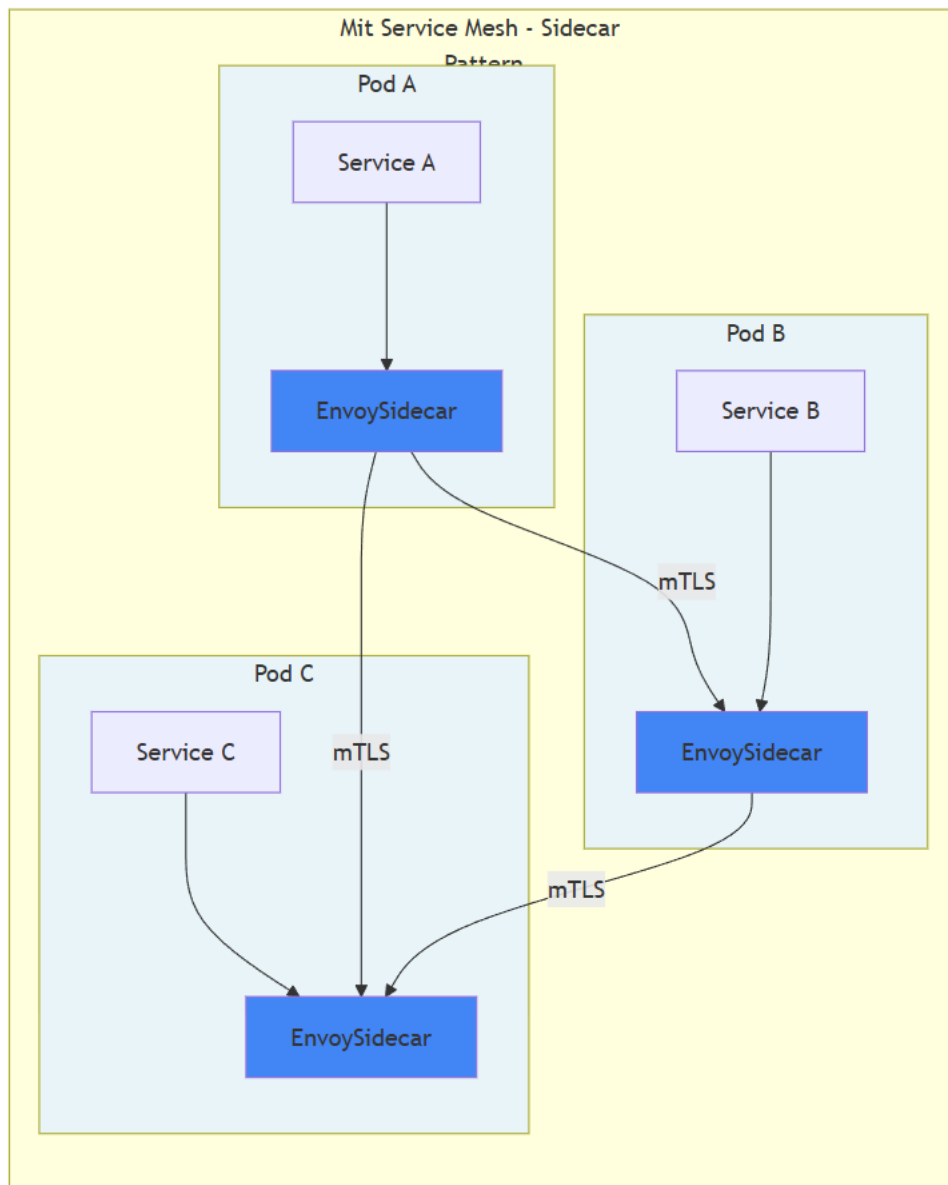
Istio im Überblick:

- Open-Source Service Mesh (Google, IBM, Lyft)
- Nutzt Envoy-Proxies als Sidecars
- Fängt gesamten Netzwerkverkehr ab

Vorher: Ohne Service-Mesh



Nachher: Mit Service-Mesh



Mermaid-Quelltexte

```

graph TB
    subgraph "Ohne Service Mesh"
        direction LR
        A1[Service A] -.direkter Traffic.-> B1[Service B]
        A1 -. -> C1[Service C]
        B1 -. -> C1
    end

```

```

graph TB
    subgraph "Mit Service Mesh - Sidecar Pattern"
        direction TB

        subgraph Pod1["Pod A"]
            direction LR
            SA[Service A] --> EA[EnvoySidecar]
        end

        subgraph Pod2["Pod B"]
            direction LR
            SB[Service B] --> EB[EnvoySidecar]
        end

        subgraph Pod3["Pod C"]
            direction LR
            SC[Service C] --> EC[EnvoySidecar]
        end

        EA -- mTLS --> EB
        EB -- mTLS --> EC
        EC -- mTLS --> EA
    end

```



```

subgraph Pod3["Pod C"]
  direction LR
  SC[Service C] --> EC[EnvoySidecar]
end

EA -->|mTLS| EB
EA -->|mTLS| EC
EB -->|mTLS| EC
end

style EA fill:#4285f4
style EB fill:#4285f4
style EC fill:#4285f4
style Pod1 fill:#e8f4f8
style Pod2 fill:#e8f4f8
style Pod3 fill:#e8f4f8

```

Warum ein Service Mesh?

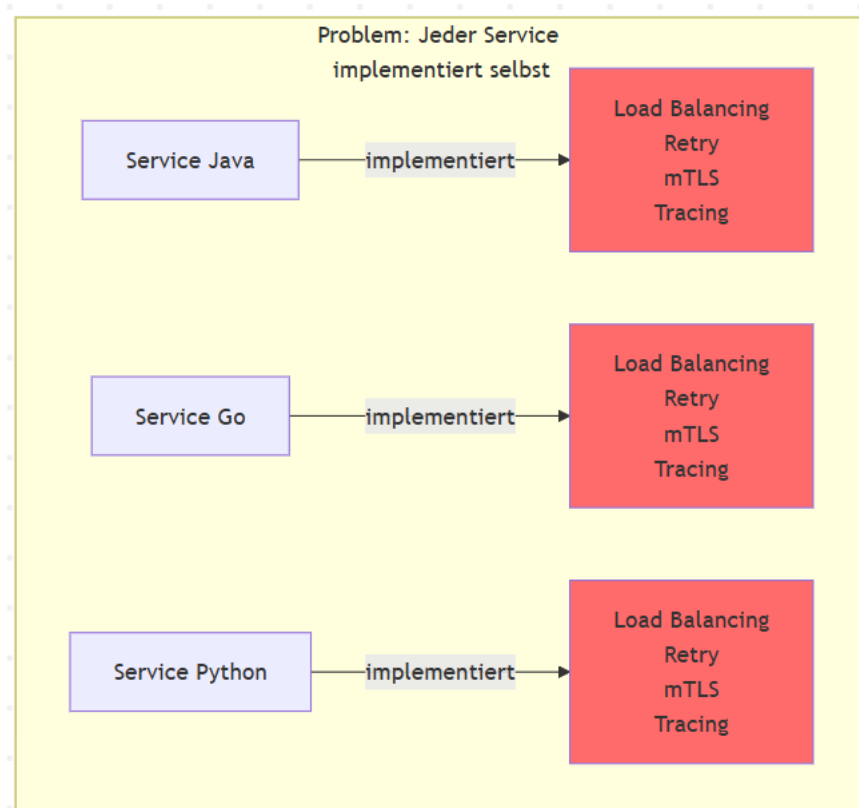
Probleme in Microservices:

- Dutzende/Hunderte Services → komplexe Kommunikation
- Jeder Service muss selbst implementieren:
 - Circuit Breaking
 - Load Balancing
 - mTLS-Verschlüsselung
 - Distributed Tracing
 - Retry-Logik
- Inkonsistente Implementierung über Sprachen hinweg
- Hoher Wartungsaufwand

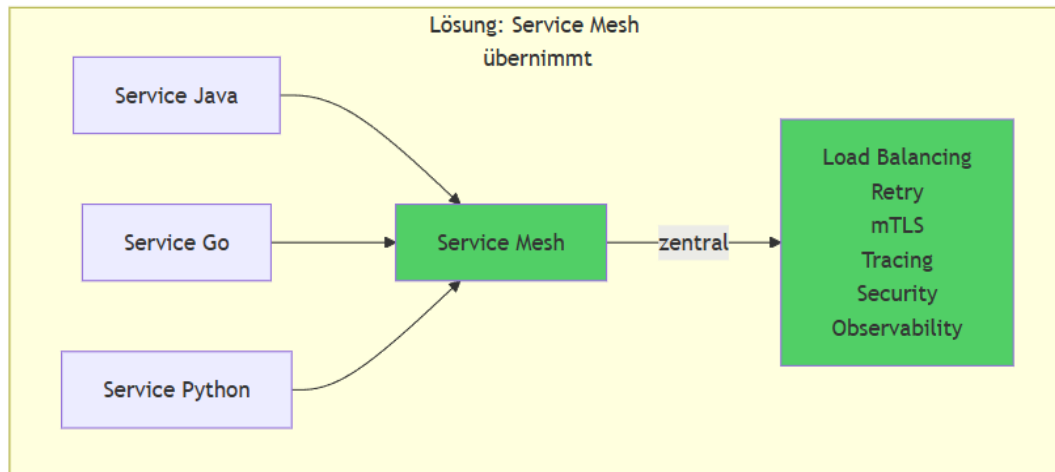
Lösung:

- Komplexität aus Anwendungscode → Infrastruktur
- Platform-Teams: zentrale Policies
- Entwickler: Fokus auf Business-Logik

Vorher: Ohne ServiceMesh



Nachher: Mit ServiceMesh



```
graph TD
    subgraph "Problem: Jeder Service implementiert selbst"
        SJ[Service Java] --> |implementiert| LJ[Load Balancing<br/>Retry<br/>mTLS<br/>Tracing]
        SG[Service Go] --> |implementiert| LG[Load Balancing<br/>Retry<br/>mTLS<br/>Tracing]
        SP[Service Python] --> |implementiert| LP[Load Balancing<br/>Retry<br/>mTLS<br/>Tracing]
    end

    subgraph "Lösung: Service Mesh übernimmt"
        S1[Service Java] --> SM[Service Mesh]
        S2[Service Go] --> SM
        S3[Service Python] --> SM
        SM -- |zentral| --> F[Load Balancing<br/>Retry<br/>mTLS<br/>Tracing<br/>Security<br/>Observability]
    end

    style LJ fill:#ff6b6b
    style LG fill:#ff6b6b
    style LP fill:#ff6b6b
    style SM fill:#51cf66
    style F fill:#51cf66
```

Herausforderungen & Vorteile

✔ Vorteile:

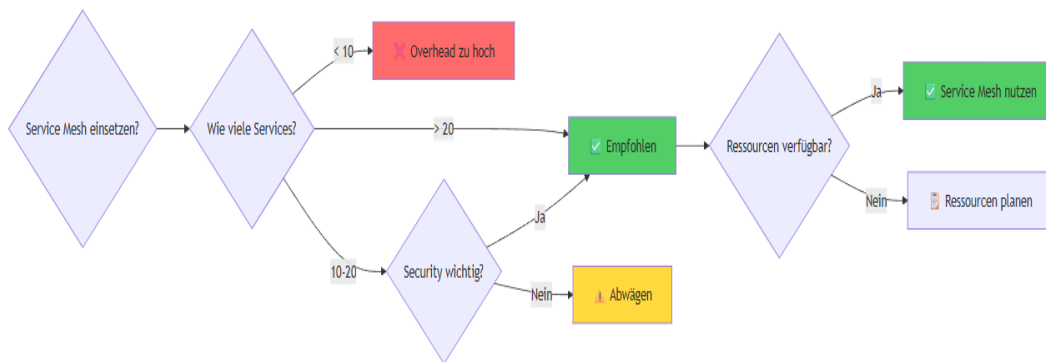
- Automatische mTLS zwischen allen Services
- Traffic-Steuerung: Canary, Blue-Green, A/B-Testing
- Einheitliches Observability (Metrics, Traces, Logs)
- Zentrale Security-Policies
- Keine Code-Änderungen nötig

⚠ Herausforderungen:

- Ressourcen-Overhead: CPU/RAM pro Sidecar
- Zusätzliche Latenz (Proxy-Hops)
- Steile Lernkurve
- Komplexeres Debugging

Wann lohnt es sich?

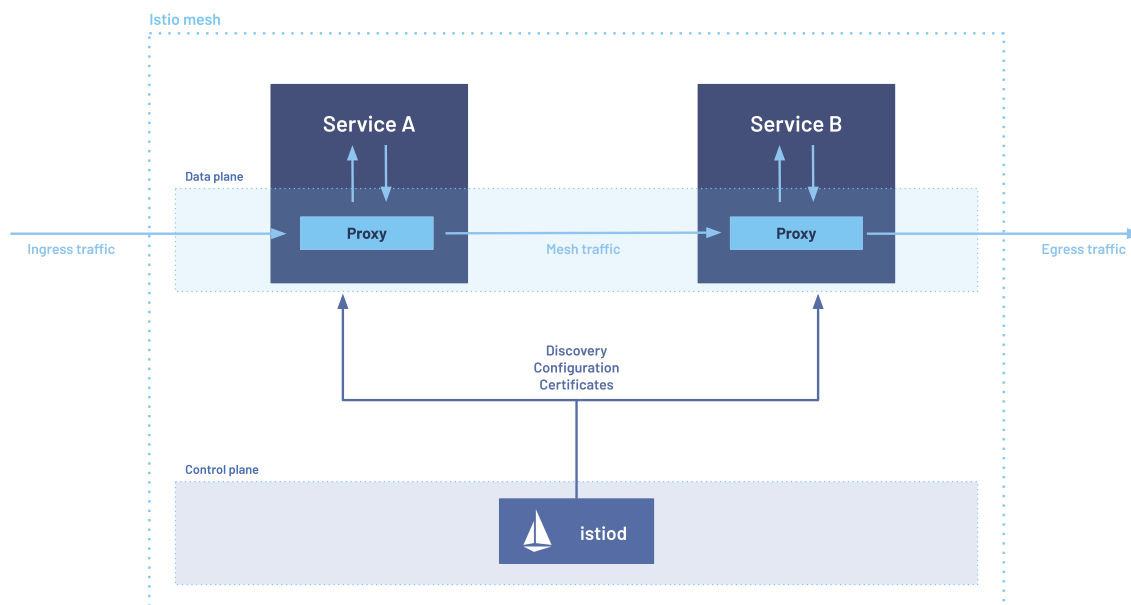
- Ab ~20-30 Services
- Hohe Security/Compliance-Anforderungen
- Multi-Team-Umgebungen



```

graph LR
  START{Service Mesh einsetzen?}
  START --> Q1{Wie viele Services?}
  Q1 -- "< 10" --> NEIN[Overhead zu hoch]
  Q1 -- "10-20" --> Q2{Security wichtig?}
  Q1 -- "> 20" --> JA[Empfohlen]
  Q2 -- "Ja" --> JA
  Q2 -- "Nein" --> MAYBE[Abwägen]
  JA --> CHECK{Ressourcen verfügbar?}
  CHECK -- "Ja" --> GO[Service Mesh nutzen]
  CHECK -- "Nein" --> PLAN[Ressourcen planen]
  style NEIN fill:#ff6b6b
  style JA fill:#51cf66
  style GO fill:#51cf66
  style MAYBE fill:#ffd93d
  
```

Architektur & Komponenten von Istio



Data Plane:

- Envoy-Proxies als Sidecars
- Fangen Traffic ab
- Setzen Policies durch

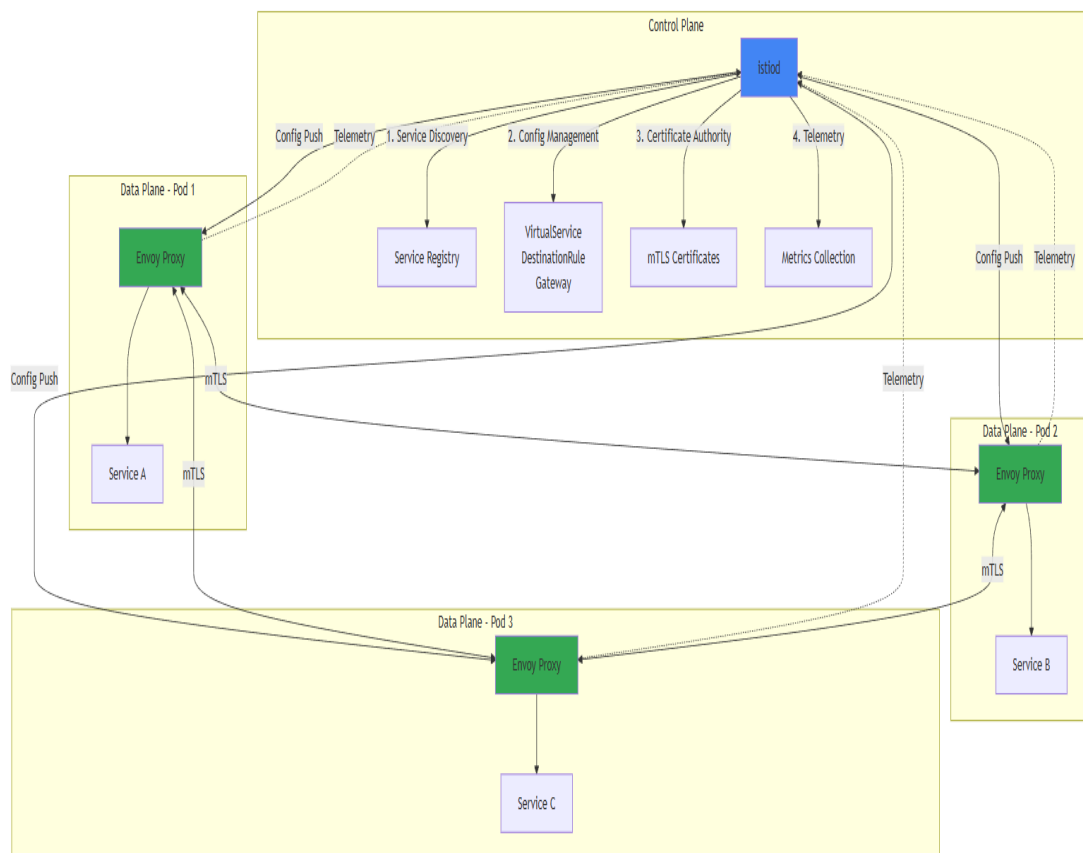
Control Plane (istiod):

- Konfigurationsverteilung
- Service Discovery
- Certificate Management
- Telemetrie-Sammlung

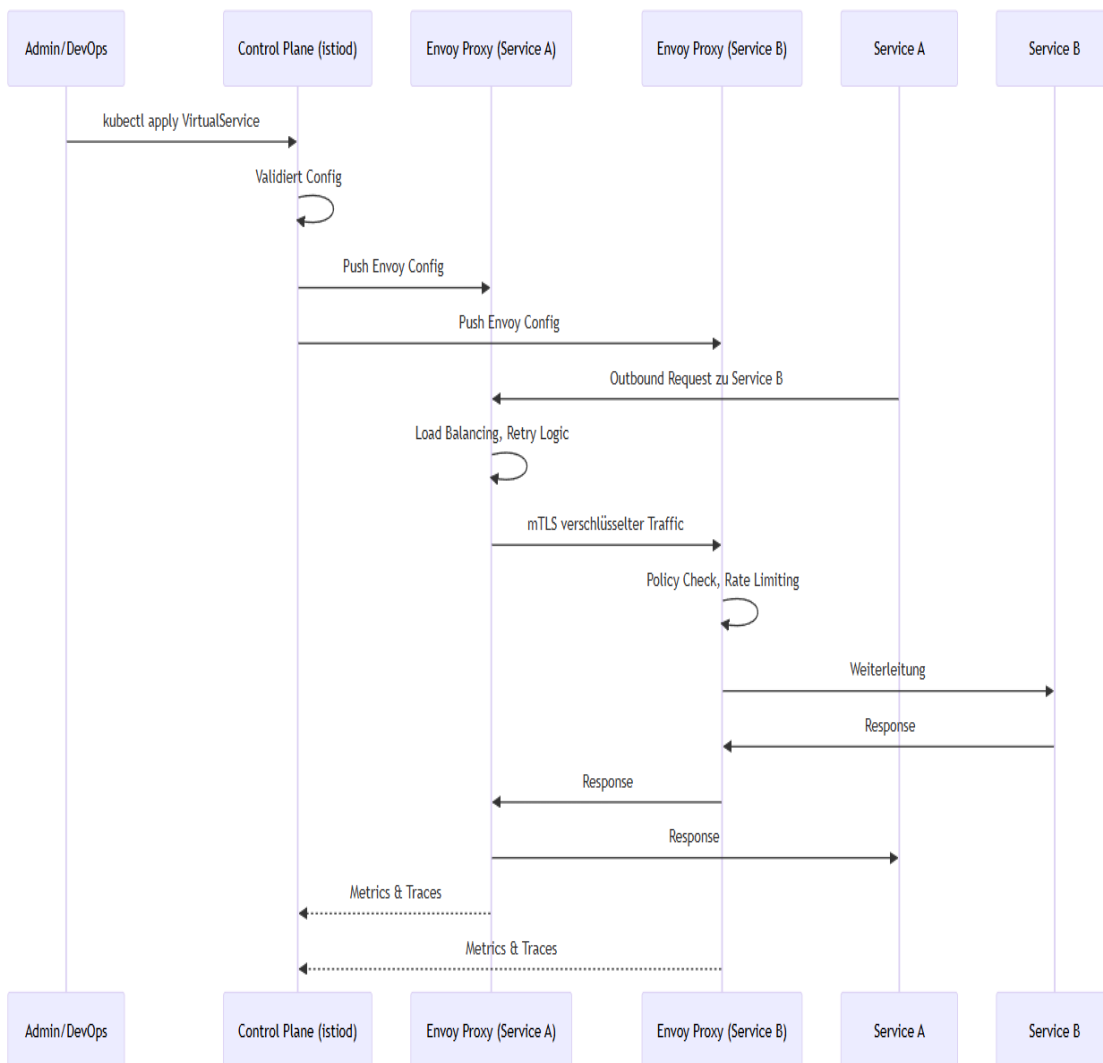
Zusammenspiel:

1. High-level Config (VirtualService, DestinationRule)
2. istiod übersetzt → Envoy-Config
3. Push an alle Proxies
4. Proxies setzen um

Grafik (Komponenten)



Grafik (Ablauf)



Komponenten

```

graph TB
    subgraph "Control Plane"
        ISTIOD[Istiod]
        ISTIOD --> |1. Service Discovery| SD[Service Registry]
        ISTIOD --> |2. Config Management| CM[VirtualService<br/>DestinationRule<br/>Gateway]
        ISTIOD --> |3. Certificate Authority| CA[mTLS Certificates]
        ISTIOD --> |4. Telemetry| TEL[Metrics Collection]
    end

    subgraph "Data Plane - Pod 1"
        E1[Envoy Proxy] --> S1[Service A]
    end

    subgraph "Data Plane - Pod 2"
        E2[Envoy Proxy] --> S2[Service B]
    end

    subgraph "Data Plane - Pod 3"
        E3[Envoy Proxy] --> S3[Service C]
    end

    ISTIOD --> |Config Push| E1
    ISTIOD --> |Config Push| E2
    ISTIOD --> |Config Push| E3
  
```

```

E1 <-->|mTLS| E2
E2 <-->|mTLS| E3
E1 <-->|mTLS| E3

E1 -.->|Telemetry| ISTIOD
E2 -.->|Telemetry| ISTIOD
E3 -.->|Telemetry| ISTIOD

style ISTIOD fill:#4285f4
style E1 fill:#34a853
style E2 fill:#34a853
style E3 fill:#34a853

```

Traffic Flow:

```

sequenceDiagram
    participant Admin as Admin/DevOps
    participant Istiod as Control Plane (istiod)
    participant E1 as Envoy Proxy (Service A)
    participant E2 as Envoy Proxy (Service B)
    participant S1 as Service A
    participant S2 as Service B

    Admin->>Istiod: kubectl apply VirtualService
    Istiod->>Istiod: Validiert Config
    Istiod->>E1: Push Envoy Config
    Istiod->>E2: Push Envoy Config

    S1->>E1: Outbound Request zu Service B
    E1->>E1: Load Balancing, Retry Logic
    E1->>E2: mTLS verschlüsselter Traffic
    E2->>E2: Policy Check, Rate Limiting
    E2->>S2: Weiterleitung
    S2->>E2: Response
    E2->>E1: Response
    E1->>S1: Response

    E1-->>Istiod: Metrics & Traces
    E2-->>Istiod: Metrics & Traces

```

Istio Proxy-Konzepte (Envoy als Sidecar)

1. Was ist Envoy?

Envoy ist ein High-Performance L4/L7 Proxy, entwickelt von Lyft und heute ein CNCF-Graduated-Projekt. Er ist in C++ geschrieben und wurde von Anfang an für dynamische, Cloud-native Umgebungen konzipiert.

Istio nutzt Envoy als **Data Plane** — jeder Proxy im Mesh ist eine Envoy-Instanz. Istio erweitert Envoy über eigene Filter (z.B. für mTLS, Telemetrie) und steuert ihn zentral über istiod (Control Plane).

2. Wo sitzt der Proxy?

Der Envoy Proxy läuft als **Native Sidecar** im gleichen Pod wie die Applikation.

Ab Kubernetes 1.28+ wird `istio-proxy` als Init-Container mit `restartPolicy: Always` definiert (Native Sidecar Feature). Das bedeutet:

- Er erscheint unter `spec.initContainers`, läuft aber für die gesamte Pod-Laufzeit
- Er startet **vor** der App und stoppt **nach** der App
- `kubectl get pods` zeigt ihn trotzdem als laufenden Container (z.B. `2/2 Ready`)

Da er sich das **Network Namespace** mit dem App-Container teilt, sieht er sämtlichen ein- und ausgehenden Traffic des Pods.

```

Pod
├─ initContainers
│   └─ istio-init      ← setzt iptables-Regeln (entfällt mit CNI Plugin)
├─ istio-proxy        ← Native Sidecar (Envoy), läuft dauerhaft
└─ containers
    └─ app             ← Applikations-Container

```

3. Wie kommt der Traffic zum Proxy?

Der Traffic wird **transparent** zum Envoy umgeleitet — die Applikation bemerkt nichts davon. Dafür werden iptables-Regeln gesetzt, die allen ein- und ausgehenden Traffic über den Envoy-Proxy routen.

Variante A: `istio-init` Init-Container (Standard)

- Läuft einmalig vor dem App-Start
- Setzt iptables-Regeln im Network Namespace des Pods
- Benötigt `NET_ADMIN` und `NET_RAW` Capabilities

Variante B: Istio CNI Plugin (empfohlen)

- Arbeitet als Chained CNI Plugin auf Node-Ebene
- Setzt die gleichen iptables-Regeln, aber **bevor** irgendein Container startet
- `istio-init` Init-Container entfällt komplett

Vorteile CNI Plugin:

- Keine `NET_ADMIN` / `NET_RAW` Capabilities im Pod → kompatibel mit `restricted` Pod Security Standard
- Keine Race Conditions → kein Traffic geht am Proxy vorbei, auch wenn andere Init-Container Netzwerk-Requests machen
- Sicherheitsaspekt: Ohne CNI Plugin können Init-Container, die vor `istio-init` laufen, unverschlüsselt und ohne Policy kommunizieren (kein mTLS, keine AuthorizationPolicy)
- Im Ambient Mode zwingend erforderlich

Hinweis: Das CNI Plugin setzt die identischen iptables-Regeln wie `istio-init` — es gibt keinen Runtime-Performance-Unterschied.

4. Was macht der Proxy?

Envoy übernimmt als Sidecar eine Vielzahl von Aufgaben, ohne dass die Applikation angepasst werden muss:

Bereich	Funktion
Security	mTLS (automatische Verschlüsselung zwischen Services), Zertifikatsrotation
Traffic Management	Routing, Retries, Timeouts, Fault Injection, Traffic Shifting (Canary, Blue/Green)
Resilienz	Circuit Breaking, Outlier Detection, Rate Limiting
Load Balancing	Round Robin, Least Connections, Random, Consistent Hashing
Observability	Metrics (RED: Rate, Errors, Duration), Distributed Traces, Access Logs

All das wird **nicht in der App konfiguriert**, sondern über Istio-Ressourcen (siehe nächster Abschnitt).

5. Wie wird der Proxy konfiguriert?

Envoy wird **nie direkt konfiguriert**. Stattdessen:

1. Der Anwender erstellt Istio CRDs (z.B. VirtualService, DestinationRule, AuthorizationPolicy)
2. **istiod** (Control Plane) übersetzt diese in Envoy-native Konfiguration
3. istiod pusht die Konfiguration über die **xDS API** (gRPC-Stream) an die Envoy-Proxies

xDS API — Hot Reload ohne Neustart

Envoy öffnet eine langlebige gRPC-Verbindung zu istiod (Watch/Subscribe-Pattern). Bei Änderungen pusht istiod die neue Konfiguration über den bestehenden Stream.

Wichtig: Envoy wendet Änderungen **in-memory** an — kein Schreiben auf die Platte, kein Prozess-Reload. Bestehende Connections werden nicht unterbrochen. Das unterscheidet Envoy fundamental von klassischen Proxies wie Nginx oder HAProxy, die ein Config-File und einen Reload benötigen.

Die einzige Datei-basierte Config ist die **Bootstrap-Konfiguration** beim ersten Start. Sie enthält im Wesentlichen nur die Verbindungsdaten zu istiod (Port 15012). Alles Weitere kommt dynamisch über xDS.

6. Sidecar Injection

Der Envoy Sidecar wird über einen **Mutating Admission Webhook** in den Pod injiziert. Es gibt zwei Varianten:

Automatic Injection (empfohlen)

Namespace mit Label versehen:

```
kubectl label namespace <NAMESPACE> istio-injection=enabled
```

Alle neuen Pods in diesem Namespace erhalten automatisch den Sidecar.

Manual Injection

```
istioctl kube-inject -f deployment.yaml | kubectl apply -f -
```

Nützlich für einzelne Workloads oder zum Debugging (um die generierte Pod-Spec zu inspizieren).

7. Memory-Overhead

Envoy bekommt per Default die Konfiguration für **alle erreichbaren Services** im Mesh — nicht nur die, die der Pod tatsächlich anspricht. In großen Meshes führt das zu erheblichem Speicherverbrauch:

Mesh-Größe	Typischer RAM pro Sidecar
Klein (< 20 Services)	50–80 MB
Mittel (50–100 Services)	100–200 MB
Groß (100+ Services)	200–300+ MB

Gegenmaßnahme: Sidecar CRD

Mit der Istio Sidecar CRD kann die Sichtbarkeit eines Envoy-Proxies eingeschränkt werden:

```
apiVersion: networking.istio.io/v1
kind: Sidecar
metadata:
  name: frontend-sidecar
  namespace: frontend
spec:
  egress:
    - hosts:
        - ".*backend.backend.svc.cluster.local"
        - "istio-system/*"
```

Der Envoy kennt dann nur noch die explizit genannten Services → deutlich weniger Memory.

Vorsicht: Falsch konfiguriert bricht die Kommunikation, weil der Envoy den Ziel-Service nicht kennt. Service-Abhängigkeiten müssen explizit gepflegt werden.

Alternative: Ambient Mode

Im Ambient Mode entfällt der Envoy-Sidecar pro Pod komplett. Stattdessen:

- **ztunnel** (pro Node) für L4 (mTLS, TCP-Routing)
- **Waypoint Proxies** (optional, pro Namespace/Service) für L7 (HTTP-Routing, Retries etc.)

Der Memory-Overhead reduziert sich erheblich, da nicht mehr jeder Pod seinen eigenen Envoy betreibt.

8. Jobs und CronJobs im Mesh

Problem (vor Native Sidecars)

Ein Job führt seinen Task aus → App-Container beendet sich → `istio-proxy` Sidecar läuft weiter → Pod bleibt `Running` statt `Completed` → Job wird nie als fertig erkannt.

Folgen:

- CronJobs häufen sich, weil der vorherige nie abschließt
- `activeDeadlineSeconds` / `backoffLimit` greifen → Job wird als **Failed** markiert
- Ressourcen bleiben durch Zombie-Envoys belegt

Workarounds (vor K8s 1.28)

```
## Envoy manuell herunterfahren am Ende des Job-Scripts
curl -X POST http://localhost:15000/quitquitquit
```

Lösung: Native Sidecars (K8s 1.28+)

Mit Native Sidecars weiß Kubernetes, dass `istio-proxy` ein Sidecar ist und beendet ihn automatisch, sobald alle regulären Container fertig sind. Keine Workarounds mehr nötig.

9. Zusammenfassung



Vergleich mit Linkerd, Cilium, Consul

Feature	Istio	Linkerd	Cilium	Consul
Proxy	Envoy (C++)	Rust-Proxy	eBPF (Kernel)	Envoy
Komplexität	Hoch	Niedrig	Mittel	Mittel
Overhead	Hoch	Niedrig	Sehr niedrig	Mittel
Features	Maximal	Basis	Netzwerk-fokus	Multi-Plattform

K8s-Native	Ja	Ja	Ja	Teilweise
Use Case	Enterprise, viele Features	Einfachheit	Performance	VM + K8s

Kernunterschiede:

- **Linkerd:** Einfach, schnell, weniger Features
- **Cilium:** eBPF = keine Sidecars, extrem performant
- **Consul:** Multi-Plattform (VMs, Bare Metal)
- **Istio:** Feature-Champion, größte Community

Service Mesh - Praktischer Aufbau im Cluster (Sidecar-Modus)

Istio-Installation mit istioctl (demo-Profil)

- Most simplistic way
- Doing the right setup is done with profiles
- Interestingly it uses an compile-in helm chart (see also: Show what a profile does)

Hint for production

- Best option (in most cases) is default

in our case: Including demo (tracing is activated)

- Not suitable for production !!

Show what a profile does

```
istioctl manifest generate > istio-manifest.yaml
## If not profile is mentioned, it uses the default profile
## it does not use an operator
cat istio-manifest.yaml | grep -i -A20 "^Kind" | less
## If you want you can apply it like so:
## kubectl apply -f istio-manifest.yaml
```

Installation including Demo

[!CAUTION] This profile (demo) enables high levels of tracing and access logging so it is not suitable for performance tests.

Schritt 1: istio runterladen und installieren

```
cd
## aktuelle stabile Version ist 1.31.0 (Stand 2026-09)
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.31.0 sh -
ln -s ~/istio-1.31.0 ~/istio
echo "export PATH=~/istio-1.31.0/bin:$PATH" >> ~/.bashrc
source ~/.bashrc
```

*[!TIP] Istio empfiehlt, dass `istioctl` (Client) exakt dieselbe Version wie die Control-Plane (`istiod`) hat - "using matching versions helps avoid unforeseen issues".
Prüfen mit `istioctl version` (zeigt Client-, Control-Plane- und Data-Plane-Version).*

Schritt 2: bash completion integrieren

```
cp ~/istio/tools/istioctl.bash ~/istioctl.bash
echo "source ~/istioctl.bash" >> ~/.bashrc
source ~/istioctl.bash
```

Schritt 2.5. See what it would install

```
## dry-run
istioctl install -f ~/istio/samples/bookinfo/demo-profile-no-gateways.yaml -y --dry-run
```

Schritt 3: Installation with demo (by using operator)

```
## cat ~/istio/samples/bookinfo/demo-profile-no-gateways.yaml
## Wird vom ControlPlane ausgewertet
## Hier wird das ingressgateway abgeschaltet,
## Weil wir das nicht benötigen, wenn wir
## die Kubernetes Gateway API verwenden
apiVersion: install.istio.io/v1alpha1
kind: IstioOperator
spec:
  profile: demo
  components:
    ingressGateways:
      - name: istio-ingressgateway
        enabled: false
    egressGateways:
```

```
- name: istio-egressgateway
  enabled: false
```

```
## Der Trend geht Richtung Kuberneetes Gateway API
istioctl install -f ~/istio/samples/bookinfo/demo-profile-no-gateways.yaml -y
```

Schritt 4: Gateway API's CRD's installieren

```
kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.6.2/standard-install.yaml
```

Reference: Get started

- <https://istio.io/latest/docs/setup/getting-started/>

istioctl Cheatsheet zum Debuggen

Ohne Install-/Uninstall-/Manifest-/Profile-Kommandos. Alle Kommandos verifiziert gegen die offizielle Istio v1.31 Referenz.

Version & Preflight

```
## Client- und Control-Plane-Version anzeigen
istioctl version
```

Proxy Status (ps)

Zeigt den Sync-Status aller Envoy-Proxies mit Istiod.

```
## Alle Proxies im Mesh
istioctl proxy-status
istioctl ps          # Kurzform

## Nach Namespace filtern
istioctl ps --namespace bookinfo

## Diff zwischen Envoy-Config und Istiod für einen bestimmten Proxy
istioctl ps <pod-name>.<namespace>
```

Proxy Config (pc)

Envoy-Konfiguration eines Pods inspizieren.

```
## Listeners
istioctl proxy-config listeners <pod>.<ns>
istioctl pc l <pod>.<ns>

## Routes
istioctl pc routes <pod>.<ns>
istioctl pc r <pod>.<ns>

## Clusters (Upstream-Services)
istioctl pc clusters <pod>.<ns>
istioctl pc c <pod>.<ns>

## Endpoints
istioctl pc endpoints <pod>.<ns>
istioctl pc ep <pod>.<ns>

## Bootstrap-Konfiguration
istioctl pc bootstrap <pod>.<ns>
istioctl pc b <pod>.<ns>

## ECDS (Extension Config Discovery)
istioctl pc ecds <pod>.<ns>

## Alles auf einmal (JSON-Dump)
istioctl pc all <pod>.<ns> -o json

## Envoy Log-Level abfragen (einzelner Pod)
istioctl pc log <pod>.<ns>

## Envoy Log-Level setzen (einzelner Pod)
istioctl pc log <pod>.<ns> --level debug
istioctl pc log <pod>.<ns> --level info    # zurücksetzen
```

```
## Einzelne Envoy-Logger gezielt setzen
istioctl pc log <pod>.<ns> --level connection:debug,router:debug
istioctl pc log <pod>.<ns> --level rbac:debug,conn_handler:warning

## Per Deployment - iteriert über ALLE Pods im Deployment
istioctl pc log deploy/productpage-v1 -n bookinfo # abfragen
istioctl pc log deploy/productpage-v1 -n bookinfo --level debug # setzen
istioctl pc log deploy/productpage-v1 -n bookinfo --level rbac:debug,conn_handler:warning

## Funktioniert analog auch mit svc/ und rs/ (ReplicaSet)
istioctl pc log svc/productpage -n bookinfo --level debug
istioctl pc log rs/productpage-v1-abc123 -n bookinfo --level debug

## Andere pc-Subcommands akzeptieren ebenfalls deploy/svc/rs
istioctl pc l deploy/productpage-v1 -n bookinfo
```

Analyze

Konfiguration auf Fehler und Warnungen prüfen.

```
## Aktuellen Namespace analysieren
istioctl analyze

## Bestimmten Namespace
istioctl analyze -n bookinfo

## Alle Namespaces
istioctl analyze --all-namespaces

## Lokale YAML-Dateien prüfen (ohne Cluster)
istioctl analyze my-virtualservice.yaml --use-kube=false

## Bestimmte Meldungen unterdrücken
istioctl analyze -n default --suppress "IST0102=Namespace default"

## Mehrere Meldungen unterdrücken + Wildcards
istioctl analyze --all-namespaces \
  --suppress "IST0102=Namespace frod" \
  --suppress "IST0107=Pod *.baz"
```

Validate

YAML-Dateien gegen das Istio-Schema validieren.

```
istioctl validate -f my-resource.yaml

## Kurzform
istioctl v -f my-resource.yaml

## Ganzes Verzeichnis
istioctl validate -f samples/bookinfo/networking/

## Aus stdin
kubectl get vs -o yaml | istioctl validate -f -
```

Dashboard (dash / d)

Dashboards per Port-Forward öffnen.

```
istioctl dashboard kiali
istioctl dashboard grafana
istioctl dashboard jaeger
istioctl dashboard zipkin

## Envoy Admin UI eines Pods
istioctl dashboard envoy <pod>.<ns>
istioctl dash envoy deploy/productpage-v1

## Proxy Dashboard (auch für ztunnel/waypoint)
istioctl dashboard proxy <pod>.<ns>

## ControlZ UI (Istiod)
istioctl dashboard controlz deploy/istiod.istio-system
```

Kube-Inject

Sidecar manuell in ein Deployment injizieren.

```
## On-the-fly beim Apply
kubectl apply -f <(istioctl kube-inject -f deployment.yaml)

## In eine Datei schreiben
istioctl kube-inject -f deployment.yaml -o deployment-injected.yaml

## Bestimmte Revision verwenden
istioctl kube-inject -f deployment.yaml --revision canary
```

Admin Log

Istiod-Logging-Level abrufen und ändern.

```
## Aktuelle Log-Level von Istiod
istioctl admin log

## Bestimmten Istiod-Pod abfragen
istioctl admin log <istiod-pod>

## Log-Level ändern
istioctl admin log --level ads:debug,authorization:debug

## Alle zurücksetzen
istioctl admin log --log-reset
```

Bug Report

Diagnose-Bundle für Support erstellen.

```
## Vollständiger Bug-Report
istioctl bug-report

## Auf bestimmte Namespaces beschränken
istioctl bug-report --include default,bookinfo

## Zeitraum begrenzen
istioctl bug-report --duration 30m
```

Tag (Revision Tags)

Revision-Tags für Canary-Upgrades verwalten.

```
## Alle Tags auflisten
istioctl tag list

## Tag erstellen/setzen
istioctl tag set prod --revision 1-22-0

## Tag entfernen
istioctl tag remove prod
```

Waypoint (Ambient Mode)

Waypoint-Proxies für Ambient Mode verwalten.

```
## Waypoint deployen
istioctl waypoint apply -n default
istioctl waypoint apply -n default --name my-waypoint

## Für Workloads statt Services
istioctl waypoint apply -n default --name wp --for workload

## YAML generieren (ohne Apply)
istioctl waypoint generate -n default --for service

## Alle Waypoints auflisten
istioctl waypoint list -n default
istioctl waypoint list -A # clusterweilt

## Status prüfen
```

```
istioctl waypoint status -n default

## Waypoint löschen
istioctl waypoint delete my-waypoint -n default
istioctl waypoint delete --all -n default
```

Ztunnel Config (Ambient Mode)

Ztunnel-Konfiguration inspizieren.

```
## Workloads
istioctl ztunnel-config workload
istioctl ztunnel-config workload <ztunnel-pod>.<ns> --node <node-name>

## Services
istioctl ztunnel-config service

## Zertifikate
istioctl ztunnel-config certificates --node <node-name>

## Policies
istioctl ztunnel-config policies

## Logging
istioctl ztunnel-config log <ztunnel-pod>.<ns>
istioctl ztunnel-config log <ztunnel-pod>.<ns> --level debug

## Alles (JSON-Dump)
istioctl ztunnel-config all <ztunnel-pod>.<ns> -o json
```

Experimental (x) Kommandos

```
## AuthorizationPolicy eines Pods prüfen
istioctl x authz check <pod>.<ns>

## Pod beschreiben (mTLS-Status, Policies, Traffic)
istioctl x describe pod <pod> -n <ns>

## Sidecar-Injection-Status prüfen
istioctl x check-inject <pod>.<ns>
istioctl x check-inject deploy/<name> -n <ns>

## Envoy-Stats abrufen
istioctl x envoy-stats <pod>.<ns>
istioctl x envoy-stats deploy/<name> --type clusters

## Service-Metriken (benötigt Prometheus)
istioctl x metrics productpage-v1.default

## Root-CA vergleichen (Multi-Cluster)
istioctl x rootca-compare <pod1>.<ns1> <pod2>.<ns2>

## Internal Debug (Istiod xDS)
istioctl x internal-debug syncz

## VM-Workload konfigurieren
istioctl x workload entry configure -f workloadgroup.yaml -o config
istioctl x workload group create --name foo --namespace bar
```

Globale Flags

Flag	Beschreibung
-n, --namespace	Kubernetes Namespace
-c, --kubeconfig	Kubeconfig-Datei
--context	Kubernetes Context
--istioNamespace	Istio Control Plane Namespace (default: istio-system)
--revision	Istio Revision auswählen
-o, --output	Ausgabeformat: json, yaml, short

Aliase & Kurzformen

Langform	Kurzform
proxy-status	ps
proxy-config	pc
proxy-config listeners	pc l
proxy-config routes	pc r
proxy-config clusters	pc c
proxy-config endpoints	pc ep
proxy-config bootstrap	pc b
dashboard	dash / d
experimental	x
validate	v

Debugging-Workflow (Kurzreferenz)

```
1. istioctl ps                # Proxies synced?
2. istioctl analyze -n <ns>   # Config-Fehler?
3. istioctl x describe pod <pod> -n <ns> # Pod-Details
4. istioctl pc l <pod>.<ns> --port <port> # Listener OK?
5. istioctl pc r <pod>.<ns> --name <port>  # Routes OK?
6. istioctl pc c <pod>.<ns> --fqdn <svc>   # Cluster/Upstream OK?
7. istioctl pc ep <pod>.<ns> --cluster <c>  # Endpoints OK?
```

Quelle: istio.io/latest/docs/reference/commands/istioctl/ — Stand: Istio v1.31

Uebung: Sidecar-Injection

1. Verzeichnis anlegen

```
cd
mkdir -p ~/manifests/nginx
```

2. Nginx-Deployment erstellen

```
cat <<'EOF' > ~/manifests/nginx/nginx.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: nginx-istio
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: nginx-istio
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx
```

```

namespace: nginx-istio
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
EOF

```

3. Sidecar injizieren und anwenden

```
kubectl apply -f <(istioctl kube-inject -f ~/manifests/nginx/nginx.yaml)
```

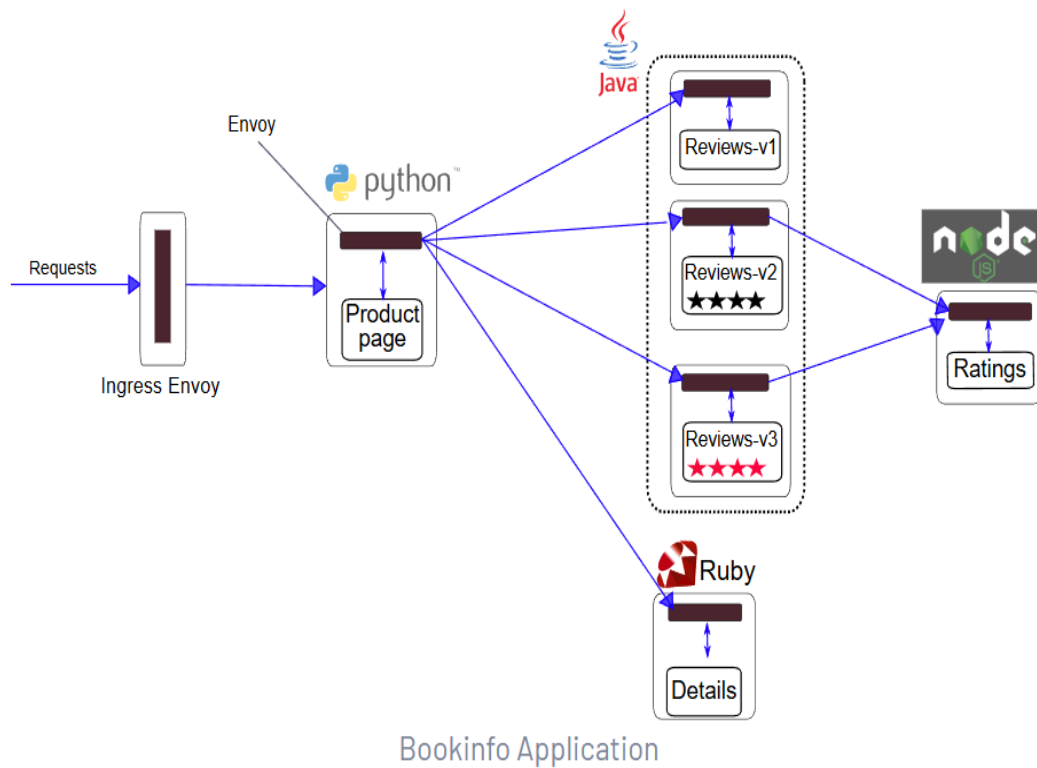
4. Injection prüfen

```
kubectl get pods -n nginx-istio
```

Erwartetes Ergebnis: `READY 2/2`

Demo-App bookinfo installieren

Überblick



Vorbereitung

Gateway API - CRD's installieren (Stand 2026-09-11)

- falls nicht bereits vorher geschehen

```

kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.6.2/standard-install.yaml

```

```

kubectl create ns bookinfo
kubectl label namespace bookinfo istio-injection=enabled

```

bookdemo app ausrollen

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl -n bookinfo get all
```

testen ob die app funktioniert

```
kubectl -n bookinfo exec "$(kubectl -n bookinfo get pod -l app=ratings -o jsonpath='{.items[0].metadata.name}')" -c ratings -- curl -sS productpage:9080/productpage | grep -o "<title>.*</title>"
```

App mit gateway api nach aussen öffnen

```
## That's what we do ...
cat ~/istio/samples/bookinfo/gateway-api/bookinfo-gateway.yaml
```

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/gateway-api/bookinfo-gateway.yaml
kubectl -n bookinfo get gateways
kubectl -n bookinfo get httproutes -o yaml
```

```
## note the external-ip from this output
## gateway automatically creates a service
kubectl -n bookinfo get svc bookinfo-gateway-istio
```

```
http://<external-ip>/productpage
## or in your browser
```

Uebung: Header-basiertes Routing

Prerequisites

- Bookinfo - Projekt aufgesetzt.

Schritt 1: Vorbereitung: Status review-pods

- Status: alle Pods sind unter einem Service erreichbar

```
## Es gibt 3 verschieden review-pods (v1, v2, v3)
kubectl -n bookinfo get pods --show-labels | grep review
```

```
## Ein Service zeigt auf alle pods (Alle versionen der Review - Pods)
kubectl -n bookinfo get svc | grep reviews
```

Schritt 2: Vorher (ohne request routing)

- Es werden alle Pods angezeigt, die das Label: app:reviews haben
- D.h. jedesmal wenn ich die Seite öffne, wird eine andere Version angezeigt (v1, v2 oder v3) - * d.h. es werden ganz normal die Services von Kubernetes verwendet *
- Service (selector: app:reviews)

```
kubectl -n bookinfo get svc reviews -o yaml
kubectl -n bookinfo get pods -l app=reviews --show-labels
```

```
## Gateway wurde in der Übung vorher angelegt
## du findest so die IP des gateways raus
kubectl -n bookinfo get gateway
```

```
GATEWAY_URL=<ip-aus-der-vorigen-Ausgabe-eintragen>
```

```
## Im Browser mehrmals ausführen
## Im Block mit den Reviews wechselt die Version
$GATEWAY_URL/productpage
```

Schritt 3: Übung (jetzt request - routing)

Voraussetzung:

- Bookinfo-App läuft bereits im Namespace `bookinfo`
- Service Reviews existiert
- Es gibt 3 verschieden Pods an Reviews (v1, v2 und v3)
- Ingress/Gateway + `GATEWAY_URL` (IP: <http://164.90.237.35/productpage> aus der vorherigen Übung vorhanden)

0. Vorbereitung

```
mkdir -p ~/manifests/requests
cd ~/manifests/requests

## Die Service-Versionen anlegen
cp -a ~/istio/samples/bookinfo/platform/kube/bookinfo-versions.yaml bookinfo-versions.yaml
kubectl -n bookinfo apply -f .
```

1. HTTPRoute: Alle Requests → reviews-v1

```
cat <<EOF > ~/manifests/requests/httproute-reviews-v1.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF

kubectl apply -f httproute-reviews-v1.yaml
kubectl -n bookinfo get httproute reviews -n bookinfo
```

```
## Vergleich mit allen httproutes
kubectl -n bookinfo get httproute
## booking for north - south traffik
```

```
## Anzeige im Browser - es ist immer die v1
http://164.90.237.35/productpage
```

2. HTTPRoute anpassen: User jason → reviews-v2 , Rest → reviews-v1

```
cat <<EOF > ~/manifests/requests/httproute-reviews-jason-v2.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - matches:
    - headers:
      - name: end-user
        value: jason
    backendRefs:
    - name: reviews-v2
      port: 9080
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF

kubectl apply -f httproute-reviews-jason-v2.yaml
kubectl -n bookinfo get httproute reviews -n bookinfo -o yaml
```

3. Testen im Browser

```
echo "$GATEWAY_URL"
## Beispiel: http://<IP>:<PORT>

## 1. Im Browser: $GATEWAY_URL/productpage aufrufen (nicht eingeloggt oder anderer User)
##   → Reviews ohne Sterne (v1)

## 2. Im Browser: als User "jason" einloggen
##   → Reviews mit Sternen (v2)
```

4. Aufräumen

```
kubectl delete -f httproute-reviews-v1.yaml --ignore-not-found
kubectl delete -f httproute-reviews-jason-v2.yaml --ignore-not-found
```

Reference:

- <https://istio.io/latest/docs/examples/bookinfo/#define-the-service-versions>

Uebung: Traffic-Shifting / Load-Balancing

- Schrittweise Umleitung von Netzwerk-Traffic zwischen zwei Service-Versionen

0. Vorbereitung

```
mkdir -p ~/manifests/traffic-shifting
cd ~/manifests/traffic-shifting

## Die Service-Versionen anlegen
cp -a ~/istio/samples/bookinfo/platform/kube/bookinfo-versions.yaml bookinfo-versions.yaml
kubectl -n bookinfo apply -f .
```

1. 100% Traffic -> reviews.v1

```
cat <<'EOF' > ~/manifests/traffic-shifting/route-reviews-v1.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-v1.yaml
kubectl get httproute -n bookinfo reviews -o yaml | less
```

```
## Status ist interessant !
```

```
status:
  parents:
  - conditions:
    - lastTransitionTime: "2025-11-17T10:32:34Z"
      message: Route was valid
      observedGeneration: 3
```

2. Testen

```
## Seite Öffnen
http://<deine-ip>/productpage

## Egal wie oft du die Seite lädst, es bleibt immer v1
```

3. 50% (v1) /50% (v3) Traffic

```
cat <<'EOF' > ~/manifests/traffic-shifting/route-reviews-50-50.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
```

```
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
      weight: 50
    - name: reviews-v3
      port: 9080
      weight: 50
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-50-50.yaml
kubectl get httproute -n bookinfo reviews -o yaml | head -n 40
```

4. Testen

```
## Seite Öffnen
http://<deine-ip>/productpage

## Abwechselnd bei mehrmals laden v1 (keine Sterne) und v3 (sterne)
```

5. 100% auf v3

```
cat <<'EOF' > ~/manifests/traffic-shifting/route-reviews-v3.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v3
      port: 9080
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-v3.yaml
kubectl get httproute -n bookinfo reviews -o yaml | head -n 50
```

```
status:
  parents:
  - conditions:
    - lastTransitionTime: "2025-11-17T10:32:34Z"
      message: Route was valid
      observedGeneration: 4
      reason: Accepted
      status: "True"
      type: Accepted
    - lastTransitionTime: "2025-11-17T10:37:32Z"
      message: All references resolved
      observedGeneration: 4
      reason: ResolvedRefs
      status: "True"
      type: ResolvedRefs
```

```
## Seite Öffnen
http://<deine-ip>/productpage

## bei mehrmals laden immer v3
```

6. Aufräumen

```
kubectl delete -n bookinfo httproute reviews --ignore-not-found
```

Reference:

- <https://istio.io/latest/docs/tasks/traffic-management/traffic-shifting/>

Debugging mit debug/run pod

Why like this ?

- We need a way that a service is mounted into the pod (service-account is used)
- Same serviceAccount that would be used by productpage - pod itself

Variante 1: Debug-Container zum Debuggen

- Debug Container in productpage - pod starten, um Verbindung zu pod -> Review zu debuggen

```
kubectl -n bookinfo get pods | grep productpage
## diesen entsprechend hier verwenden
kubectl -n bookinfo debug productpage-v1-54bb874995-rr7cv -it --image=busybox
```

```
## in der bash
wget -O - http://reviews:9080/reviews/1
```

```
exit
```

V2 - Eigener Pod - Podtester

```
kubectl -n bookinfo run --rm -it podtester --image=busybox --overrides='{ "spec": { "serviceAccount": "bookinfo-productpage" } }'
```

Service Mesh - Praktischer Aufbau mit Ambient-Mode (Gateway API statt Sidecar)

Istio-Installation mit istioctl (Ambient-Profil)

- Genau wie im Sidecar-Modus die einfachste Installationsart
- Statt einem Envoy-Sidecar pro Pod: ein `ztunnel` -Agent pro Node (Layer 4) + optional Waypoint-Proxies pro Namespace (Layer 7)
- `istioctl install --set profile=ambient` installiert automatisch: Istio-Core, Istiod, das Istio-CNI-Plugin UND `ztunnel` (alles in einem Schritt)

Unterschied zum Helm-Weg

- Mit Helm installiert man `base`, `istiod`, `cni` und `ztunnel` als vier einzelne Charts (siehe [03-install-with-helm.md](#))
- Mit istioctl reicht ein Kommando

Schritt 1: istio runterladen und installieren

- Falls schon aus der Sidecar-Installation vorhanden, kann dieser Schritt übersprungen werden - istioctl kann beide Profile

```
cd
## aktuelle stabile Version ist 1.31.0 (Stand 2026-09)
curl -L https://istio.io/downloadIstio | ISTIO_VERSION=1.31.0 sh -
ln -s ~/istio-1.31.0 ~/istio
echo "export PATH=~/istio-1.31.0/bin:$PATH" >> ~/.bashrc
source ~/.bashrc
```

*[TIP] Istio empfiehlt, dass `istioctl` (Client) exakt dieselbe Version wie die Control-Plane (`istiod`) hat - "using matching versions helps avoid unforeseen issues".
Prüfen mit `istioctl version` (zeigt Client-, Control-Plane- und Data-Plane-Version).*

Schritt 2: Installation mit dem Ambient-Profil

```
istioctl install --set profile=ambient --skip-confirmation
```

[CAUTION] Wenn hier eine Warnung kommt: detected Calico CNI with 'bpfConnectTimeLoadBalancing=TCP'; this must be set to 'Disabled' - siehe Schritt 3, VOR dem weiteren Testen fixen.

Erwartetes Ergebnis:

```
kubectl get pods -n istio-system
```

- `istio-cni-node-*` (DaemonSet, ein Pod pro Node)
- `istiod-*`
- `ztunnel-*` (DaemonSet, ein Pod pro Node)
- KEIN `istio-ingressgateway` - das brauchen wir nicht, wir nutzen die Kubernetes Gateway API

Schritt 3: Calico-Fix (nur bei Calico-CNI-Clustern nötig)

Warum das nötig ist (einfach erklärt): Calico klinkt sich mit dem Feature "Connect-Time Load Balancing" (CTLB) schon beim `connect()` -Aufruf einer Anwendung ein und schreibt die Ziel-IP direkt um (z.B. Service-IP -> Pod-IP), bevor das Paket überhaupt losgeschickt wird. Istio Ambient braucht aber das unveränderte Paket, um es per iptables/eBPF zum `ztunnel` umzuleiten (dort passiert mTLS + Routing). Schreibt Calico die Ziel-IP vorher schon um, sieht `ztunnel` die Verbindung nicht mehr richtig - die

Ambient-Umleitung greift dann nicht zuverlässig, oft ohne sichtbaren Fehler. Deshalb muss dieses eine Calico-Feature abgeschaltet werden (der Rest von Calico - Networking, NetworkPolicies - bleibt unangetastet).

```
kubectl patch felixconfiguration default --type merge \
-p '{"spec":{"bpfConnectTimeLoadBalancing":"Disabled"}}'

## Kontrolle
kubectl get felixconfiguration default -o jsonpath='{.spec.bpfConnectTimeLoadBalancing}'
```

Erwartete Ausgabe: Disabled

Schritt 4: Gateway API CRD's installieren

```
kubectl get crd gateways.gateway.networking.k8s.io &> /dev/null || \
kubectl apply --server-side -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.6.2/standard-install.yaml
```

Reference: Get started

- <https://istio.io/latest/docs/ambient/getting-started/>

istioctl Cheatsheet zum Debuggen

Ohne Install-/Uninstall-/Manifest-/Profile-Kommandos. Alle Kommandos verifiziert gegen die offizielle Istio v1.31 Referenz.

Version & Preflight

```
## Client- und Control-Plane-Version anzeigen
istioctl version
```

Proxy Status (ps)

Zeigt den Sync-Status aller Envoy-Proxies mit Istiod.

```
## Alle Proxies im Mesh
istioctl proxy-status
istioctl ps                # Kurzform

## Nach Namespace filtern
istioctl ps --namespace bookinfo

## Diff zwischen Envoy-Config und Istiod für einen bestimmten Proxy
istioctl ps <pod-name>.<namespace>
```

Proxy Config (pc)

Envoy-Konfiguration eines Pods inspizieren.

```
## Listeners
istioctl proxy-config listeners <pod>.<ns>
istioctl pc l <pod>.<ns>

## Routes
istioctl pc routes <pod>.<ns>
istioctl pc r <pod>.<ns>

## Clusters (Upstream-Services)
istioctl pc clusters <pod>.<ns>
istioctl pc c <pod>.<ns>

## Endpoints
istioctl pc endpoints <pod>.<ns>
istioctl pc ep <pod>.<ns>

## Bootstrap-Konfiguration
istioctl pc bootstrap <pod>.<ns>
istioctl pc b <pod>.<ns>

## ECDS (Extension Config Discovery)
istioctl pc ecds <pod>.<ns>

## Alles auf einmal (JSON-Dump)
istioctl pc all <pod>.<ns> -o json

## Envoy Log-Level abfragen (einzelner Pod)
istioctl pc log <pod>.<ns>
```

```
## Envoy Log-Level setzen (einzelner Pod)
istioctl pc log <pod>.<ns> --level debug
istioctl pc log <pod>.<ns> --level info      # zurücksetzen

## Einzelne Envoy-Logger gezielt setzen
istioctl pc log <pod>.<ns> --level connection:debug,router:debug
istioctl pc log <pod>.<ns> --level rbac:debug,conn_handler:warning

## Per Deployment - iteriert über ALLE Pods im Deployment
istioctl pc log deploy/productpage-v1 -n bookinfo      # abfragen
istioctl pc log deploy/productpage-v1 -n bookinfo --level debug # setzen
istioctl pc log deploy/productpage-v1 -n bookinfo --level rbac:debug,conn_handler:warning

## Funktioniert analog auch mit svc/ und rs/ (ReplicaSet)
istioctl pc log svc/productpage -n bookinfo --level debug
istioctl pc log rs/productpage-v1-abc123 -n bookinfo --level debug

## Andere pc-Subcommands akzeptieren ebenfalls deploy/svc/rs
istioctl pc l deploy/productpage-v1 -n bookinfo
```

Analyze

Konfiguration auf Fehler und Warnungen prüfen.

```
## Aktuellen Namespace analysieren
istioctl analyze

## Bestimmten Namespace
istioctl analyze -n bookinfo

## Alle Namespaces
istioctl analyze --all-namespaces

## Lokale YAML-Dateien prüfen (ohne Cluster)
istioctl analyze my-virtualservice.yaml --use-kube=false

## Bestimmte Meldungen unterdrücken
istioctl analyze -n default --suppress "IST0102=Namespace default"

## Mehrere Meldungen unterdrücken + Wildcards
istioctl analyze --all-namespaces \
  --suppress "IST0102=Namespace frod" \
  --suppress "IST0107=Pod *.baz"
```

Validate

YAML-Dateien gegen das Istio-Schema validieren.

```
istioctl validate -f my-resource.yaml

## Kurzform
istioctl v -f my-resource.yaml

## Ganzes Verzeichnis
istioctl validate -f samples/bookinfo/networking/

## Aus stdin
kubectl get vs -o yaml | istioctl validate -f -
```

Dashboard (dash / d)

Dashboards per Port-Forward öffnen.

```
istioctl dashboard kiali
istioctl dashboard grafana
istioctl dashboard jaeger
istioctl dashboard zipkin

## Envoy Admin UI eines Pods
istioctl dashboard envoy <pod>.<ns>
istioctl dash envoy deploy/productpage-v1

## Proxy Dashboard (auch für ztunnel/waypoint)
istioctl dashboard proxy <pod>.<ns>
```

```
## ControlZ UI (Istiod)
istioctl dashboard controlz deploy/istiod.istio-system
```

Kube-Inject

Sidecar manuell in ein Deployment injizieren.

```
## On-the-fly beim Apply
kubectl apply -f <(istioctl kube-inject -f deployment.yaml)

## In eine Datei schreiben
istioctl kube-inject -f deployment.yaml -o deployment-injected.yaml

## Bestimmte Revision verwenden
istioctl kube-inject -f deployment.yaml --revision canary
```

Admin Log

Istiod-Logging-Level abrufen und ändern.

```
## Aktuelle Log-Level von Istiod
istioctl admin log

## Bestimmten Istiod-Pod abfragen
istioctl admin log <istiod-pod>

## Log-Level ändern
istioctl admin log --level ads:debug,authorization:debug

## Alle zurücksetzen
istioctl admin log --log-reset
```

Bug Report

Diagnose-Bundle für Support erstellen.

```
## Vollständiger Bug-Report
istioctl bug-report

## Auf bestimmte Namespaces beschränken
istioctl bug-report --include default,bookinfo

## Zeitraum begrenzen
istioctl bug-report --duration 30m
```

Tag (Revision Tags)

Revision-Tags für Canary-Upgrades verwalten.

```
## Alle Tags auflisten
istioctl tag list

## Tag erstellen/setzen
istioctl tag set prod --revision 1-22-0

## Tag entfernen
istioctl tag remove prod
```

Waypoint (Ambient Mode)

Waypoint-Proxies für Ambient Mode verwalten.

```
## Waypoint deployen
istioctl waypoint apply -n default
istioctl waypoint apply -n default --name my-waypoint

## Für Workloads statt Services
istioctl waypoint apply -n default --name wp --for workload

## YAML generieren (ohne Apply)
istioctl waypoint generate -n default --for service

## Alle Waypoints auflisten
```

```
istioctl waypoint list -n default
istioctl waypoint list -A           # clusterweilt

## Status prüfen
istioctl waypoint status -n default

## Waypoint löschen
istioctl waypoint delete my-waypoint -n default
istioctl waypoint delete --all -n default
```

Ztunnel Config (Ambient Mode)

Ztunnel-Konfiguration inspizieren.

```
## Workloads
istioctl ztunnel-config workload
istioctl ztunnel-config workload <ztunnel-pod>.<ns> --node <node-name>

## Services
istioctl ztunnel-config service

## Zertifikate
istioctl ztunnel-config certificates --node <node-name>

## Policies
istioctl ztunnel-config policies

## Logging
istioctl ztunnel-config log <ztunnel-pod>.<ns>
istioctl ztunnel-config log <ztunnel-pod>.<ns> --level debug

## Alles (JSON-Dump)
istioctl ztunnel-config all <ztunnel-pod>.<ns> -o json
```

Experimental (x) Kommandos

```
## AuthorizationPolicy eines Pods prüfen
istioctl x authz check <pod>.<ns>

## Pod beschreiben (mTLS-Status, Policies, Traffic)
istioctl x describe pod <pod> -n <ns>

## Sidecar-Injection-Status prüfen
istioctl x check-inject <pod>.<ns>
istioctl x check-inject deploy/<name> -n <ns>

## Envoy-Stats abrufen
istioctl x envoy-stats <pod>.<ns>
istioctl x envoy-stats deploy/<name> --type clusters

## Service-Metriken (benötigt Prometheus)
istioctl x metrics productpage-v1.default

## Root-CA vergleichen (Multi-Cluster)
istioctl x rootca-compare <pod1>.<ns1> <pod2>.<ns2>

## Internal Debug (Istiod xDS)
istioctl x internal-debug syncz

## VM-Workload konfigurieren
istioctl x workload entry configure -f workloadgroup.yaml -o config
istioctl x workload group create --name foo --namespace bar
```

Globale Flags

Flag	Beschreibung
-n, --namespace	Kubernetes Namespace
-c, --kubeconfig	Kubeconfig-Datei
--context	Kubernetes Context
--istioNamespace	Istio Control Plane Namespace (default: istio-system)
--revision	Istio Revision auswählen

-o, --output	Ausgabeformat: json, yaml, short
--------------	----------------------------------

Aliase & Kurzformen

Langform	Kurzform
proxy-status	ps
proxy-config	pc
proxy-config listeners	pc l
proxy-config routes	pc r
proxy-config clusters	pc c
proxy-config endpoints	pc ep
proxy-config bootstrap	pc b
dashboard	dash / d
experimental	x
validate	v

Debugging-Workflow (Kurzreferenz)

1. istioctl ps	# Proxies synced?
2. istioctl analyze -n <ns>	# Config-Fehler?
3. istioctl x describe pod <pod> -n <ns>	# Pod-Details
4. istioctl pc l <pod>.<ns> --port <port>	# Listener OK?
5. istioctl pc r <pod>.<ns> --name <port>	# Routes OK?
6. istioctl pc c <pod>.<ns> --fqdn <svc>	# Cluster/Upstream OK?
7. istioctl pc ep <pod>.<ns> --cluster <c>	# Endpoints OK?

Quelle: istio.io/latest/docs/reference/commands/istioctl/ — Stand: Istio v1.31

Uebung: Workload ins Ambient-Mesh aufnehmen (statt Sidecar-Injection)

- Im Sidecar-Modus wird pro Pod ein Envoy-Container injiziert (`istioctl kube-inject` , `READY 2/2`)
- Im Ambient-Modus gibt es KEINE Injection - ein Namespace-Label reicht, der Pod bleibt `READY 1/1`
- Die Umleitung zum `ztunnel` -Agent des Nodes passiert transparent über das Istio-CNI-Plugin (kein Init-Container/Sidecar im Pod sichtbar)

1. Verzeichnis anlegen

```
cd
mkdir -p ~/manifests/nginx-ambient
```

2. Nginx-Deployment erstellen (Namespace direkt mit Ambient-Label)

```
cat <<'EOF' > ~/manifests/nginx-ambient/nginx.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: nginx-ambient
  labels:
    istio.io/dataplane-mode: ambient
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
  namespace: nginx-ambient
  labels:
    app: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
```

```
        image: nginx:1.25
        ports:
          - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx
  namespace: nginx-ambient
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
EOF
```

3. Anwenden (KEIN kube-inject nötig)

```
kubectl apply -f ~/manifests/nginx-ambient/nginx.yaml
```

4. Aufnahme ins Mesh prüfen

```
kubectl get pods -n nginx-ambient
```

Erwartetes Ergebnis: `READY 1/1` - kein zweiter Container, obwohl der Namespace im Mesh ist.

```
istioctl ztunnel-config workload -n istio-system | grep nginx-ambient
```

Erwartetes Ergebnis: Der Pod taucht mit Protokoll `HBONE` auf (vom `ztunnel` erfasst) und `WAYPOINT=None` (nur Layer 4, da kein Waypoint für diesen Namespace deployt wurde).

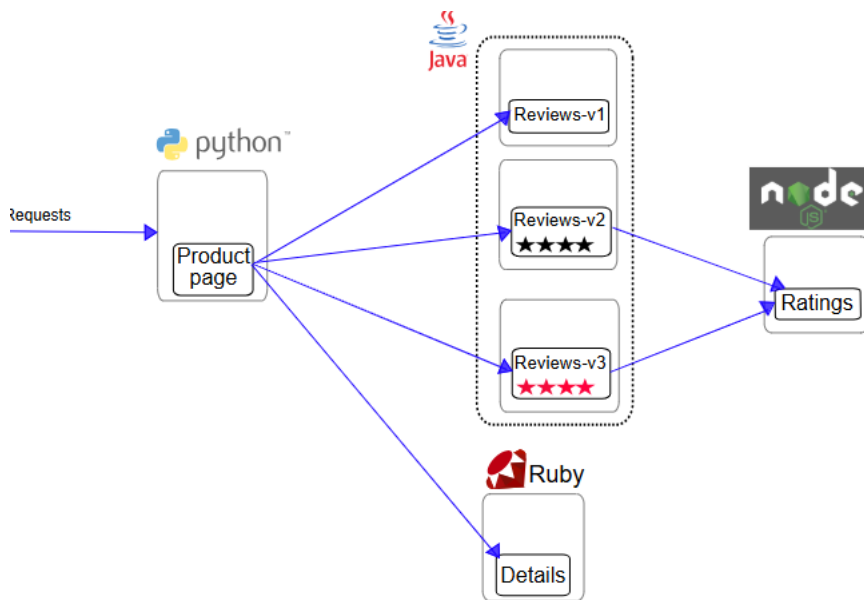
NAMESPACE	POD NAME	ADDRESS	NODE	WAYPOINT	PROTOCOL
nginx-ambient	nginx-xxxxxxxx-xxxxx	<pod-ip>	<node>	None	HBONE

Zusammenfassung

	Sidecar	Ambient
Aktivierung	istio-injection=enabled + Injection (Webhook/kube-inject)	istio.io/dataplane-mode=ambient (Label reicht)
Sichtbar im Pod	zusätzlicher Container <code>istio-proxy</code> (<code>READY 2/2</code>)	kein zusätzlicher Container (<code>READY 1/1</code>)
Nachweis der Aufnahme	<code>kubectl get pods</code>	<code>istioctl ztunnel-config workload</code>
Layer 7 (HTTP-Routing, Policies)	immer vorhanden (Envoy pro Pod)	erst mit zusätzlichem Waypoint-Proxy pro Namespace

Demo-App bookinfo installieren (Ambient/Waypoint)

Überblick



Vorbereitung

- Statt `istio-injection=enabled` (Sidecar) gibt es im Ambient-Mode das Label `istio.io/dataplane-mode=ambient` - Pods bekommen dadurch KEINEN Sidecar, sondern werden transparent über den `ztunnel`-Agent des Nodes geroutet (Layer 4, mTLS)

```
kubectl create ns bookinfo
kubectl label namespace bookinfo istio.io/dataplane-mode=ambient
```

Waypoint Proxy ausrollen

- Der Waypoint ist die Ambient-Entsprechung des Sidecars für alles, was Layer 7 braucht (HTTP-Routing, JWT/RBAC-Policies, Retries, ...) - `ztunnel` allein kann nur Layer 4

```
cd
mkdir -p manifests/waypoint
cd manifests/waypoint
```

```
## YAML generieren (dry-run)
istioctl waypoint generate --namespace bookinfo --for all > waypoint.yaml
```

```
## Anschauen was passiert
cat waypoint.yaml
```

```
## Ausrollen
kubectl apply -f waypoint.yaml
kubectl label namespace bookinfo istio.io/use-waypoint=waypoint
```

```
## Überprüfen
kubectl -n bookinfo get gateways
istioctl waypoint list --namespace bookinfo
```

Erwartetes Ergebnis: `PROGRAMMED=True` (kann ein paar Sekunden dauern)

Optional: Falls hier `~/istio` - Ordner noch nicht existiert

```
cd
## current version of istio is 1.31.0
curl -L https://istio.io/downloadIstio | sh -
ln -s ~/istio-1.31.0 ~/istio
```

bookinfo App ausrollen

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/platform/kube/bookinfo-versions.yaml
kubectl -n bookinfo get pods
```

Erwartetes Ergebnis: `READY 1/1` bei allen Pods (kein Sidecar, anders als bei der Installation mit dem demo-Profil, wo `2/2` erwartet wird)

testen ob die app funktioniert

```
kubectl -n bookinfo exec deployments/ratings-v1 -c ratings -- curl -sS productpage:9080/productpage | grep -o "<title>.*</title>"
```

App mit gateway api nach aussen öffnen

```
## That's what we do ...
cat ~/istio/samples/bookinfo/gateway-api/bookinfo-gateway.yaml
```

```
kubectl -n bookinfo apply -f ~/istio/samples/bookinfo/gateway-api/bookinfo-gateway.yaml
kubectl -n bookinfo get gateways
kubectl -n bookinfo get httproutes -o yaml
```

```
## note the external-ip from this output
## gateway automatically creates a service
kubectl -n bookinfo get svc bookinfo-gateway-istio
```

```
http://<external-ip>/productpage
## or in your browser
```

Reference

- <https://istio.io/latest/docs/ambient/getting-started/deploy-sample-app/>

Uebung: Header-basiertes Routing (Gateway API HTTPRoute)

Prerequisites

- Bookinfo im Ambient-Mode aufgesetzt ([04-install-demo-app.md](#))
- Namespace `bookinfo` hat einen **Waypoint-Proxy** (`istio.io/use-waypoint=waypoint`)

Hintergrund: Warum hier ein Waypoint nötig ist

- Das HTTPRoute unten hängt (wie im Sidecar-Setup auch) direkt am Service (`parentRefs: kind: Service` , sog. "Mesh-Routing" / GAMMA) - kein eigenes Gateway nötig für diesen internen Traffic
- Im Sidecar-Modus wertet der Envoy-Sidecar jedes Pods das HTTPRoute aus
- Im Ambient-Modus kann `ztunnel` das NICHT - es arbeitet nur auf Layer 4 (TCP, mTLS) und kennt keine HTTP-Header. Header-basiertes Routing braucht deshalb zwingend den **Waypoint-Proxy** (Layer 7) im Namespace

Schritt 1: Vorbereitung: Status review-pods

- Status: alle Pods sind unter einem Service erreichbar

```
## Es gibt 3 verschieden review-pods (v1, v2, v3)
kubectl -n bookinfo get pods --show-labels | grep review
```

```
## Ein Service zeigt auf alle pods (Alle versionen der Review - Pods)
kubectl -n bookinfo get svc | grep reviews
```

Schritt 2: Vorher (ohne request routing)

- Es werden alle Pods angezeigt, die das Label: `app:reviews` haben
- D.h. jedesmal wenn ich die Seite öffne, wird eine andere Version angezeigt (v1, v2 oder v3) - es werden ganz normal die Services von Kubernetes verwendet
- Service (selector: `app:reviews`)

```
kubectl -n bookinfo get svc reviews -o yaml
kubectl -n bookinfo get pods -l app=reviews --show-labels
```

```
## Gateway wurde in der Übung vorher angelegt
## du findest so die IP des gateways raus
kubectl -n bookinfo get svc bookinfo-gateway-istio
```

```
GATEWAY_URL=<external-ip-aus-der-vorigen-Ausgabe-eintragen>
```

```
## Im Browser mehrmals ausführen
## Im Block mit den Reviews wechselt die Version
$GATEWAY_URL/productpage
```

Schritt 3: Übung (jetzt request - routing)

Voraussetzung:

- Bookinfo-App läuft bereits im Namespace `bookinfo` (Ambient + Waypoint)
- Service Reviews existiert
- Es gibt 3 verschieden Pods an Reviews (v1, v2 und v3)

0. Vorbereitung

```
mkdir -p ~/manifests/requests-ambient
cd ~/manifests/requests-ambient
```

1. HTTPRoute: Alle Requests → reviews-v1

```
cat <<EOF > ~/manifests/requests-ambient/httproute-reviews-v1.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF

kubectl apply -f httproute-reviews-v1.yaml
kubectl -n bookinfo get httproute reviews
```

```
## Anzeige im Browser - es ist immer die v1 (keine Sterne)
http://$GATEWAY_URL/productpage
```

2. HTTPRoute anpassen: User jason → reviews-v2, Rest → reviews-v1

```
cat <<EOF > ~/manifests/requests-ambient/httproute-reviews-jason-v2.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - matches:
    - headers:
      - name: end-user
        value: jason
    backendRefs:
    - name: reviews-v2
      port: 9080
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF

kubectl apply -f httproute-reviews-jason-v2.yaml
kubectl -n bookinfo get httproute reviews -o yaml
```

3. Testen im Browser

```
echo "$GATEWAY_URL"

## 1. Im Browser: $GATEWAY_URL/productpage aufrufen (nicht eingeloggt oder anderer User)
##   → Reviews ohne Sterne (v1)

## 2. Im Browser: als User "jason" einloggen
##   → Reviews mit Sternen (v2)
```

4. Testen direkt gegen den Reviews-Service (ohne Browser-Login)

- Der Header `end-user` wird sonst von `productpage` anhand des Login-Cookies gesetzt - für einen schnellen Test genügt ein Client-Pod im Mesh

```
kubectl -n bookinfo apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n bookinfo wait --for=condition=ready pod -l app=sleep --timeout=60s
```

```
## ohne Header -> reviews-v1 (keine "rating" im JSON)
kubectl -n bookinfo exec deploy/sleep -- curl -sS http://reviews:9080/reviews/0

## mit Header end-user: jason -> reviews-v2 (mit "rating"/Sternen)
kubectl -n bookinfo exec deploy/sleep -- curl -sS -H "end-user: jason" http://reviews:9080/reviews/0
```

5. Aufräumen

```
kubectl delete -f httproute-reviews-v1.yaml --ignore-not-found
kubectl delete -f httproute-reviews-jason-v2.yaml --ignore-not-found
```

Reference:

- <https://istio.io/latest/docs/examples/bookinfo/#define-the-service-versions>
- <https://istio.io/latest/docs/ambient/usage/l7-features/>

Uebung: Traffic-Shifting (Gateway API HTTPRoute)

- Schrittweise Umleitung von Netzwerk-Traffic zwischen zwei Service-Versionen
- Voraussetzung wie bei [Request Routing](#): Bookinfo im Ambient-Mode + Waypoint-Proxy im Namespace `bookinfo` (Layer 7 für HTTPRoute-Gewichtung)

0. Vorbereitung

```
mkdir -p ~/manifests/traffic-shifting-ambient
cd ~/manifests/traffic-shifting-ambient
```

1. 100% Traffic -> reviews.v1

```
cat <<'EOF' > ~/manifests/traffic-shifting-ambient/route-reviews-v1.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v1
      port: 9080
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-v1.yaml
kubectl get httproute -n bookinfo reviews -o yaml | less
```

2. Testen

```
## Seite Öffnen
http://<deine-ip>/productpage

## Egal wie oft du die Seite lädst, es bleibt immer v1
```

Direkt gegen den Service (schneller, ohne Browser):

```
kubectl -n bookinfo apply -f ~/istio/samples/sleep/sleep.yaml
kubectl -n bookinfo wait --for=condition=ready pod -l app=sleep --timeout=60s

for i in $(seq 1 5); do
  kubectl -n bookinfo exec deploy/sleep -- curl -sS http://reviews:9080/reviews/0 | grep -o "podname: "[a-z0-9-]*"'
done
```

3. 50% (v1) /50% (v3) Traffic

```
cat <<'EOF' > ~/manifests/traffic-shifting-ambient/route-reviews-50-50.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
```

```
parentRefs:
- group: ""
  kind: Service
  name: reviews
  port: 9080
rules:
- backendRefs:
  - name: reviews-v1
    port: 9080
    weight: 50
  - name: reviews-v3
    port: 9080
    weight: 50
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-50-50.yaml
kubectl get httproute -n bookinfo reviews -o yaml | head -n 40
```

4. Testen

```
## Seite öffnen
http://<deine-ip>/productpage
```

```
## Abwechselnd bei mehrmals laden v1 (keine Sterne) und v3 (sterne)
```

```
for i in $(seq 1 10); do
  kubectl -n bookinfo exec deploy/sleep -- curl -sS http://reviews:9080/reviews/0 | grep -o "podname: "[a-z0-9-]*"
done
```

Erwartetes Ergebnis: gemischt v1/v3, über 10 Requests ungefähr 50/50 verteilt.

5. 100% auf v3

```
cat <<'EOF' > ~/manifests/traffic-shifting-ambient/route-reviews-v3.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: reviews
  namespace: bookinfo
spec:
  parentRefs:
  - group: ""
    kind: Service
    name: reviews
    port: 9080
  rules:
  - backendRefs:
    - name: reviews-v3
      port: 9080
EOF
```

```
kubectl apply -n bookinfo -f route-reviews-v3.yaml
kubectl get httproute -n bookinfo reviews -o yaml | head -n 50
```

```
## Seite öffnen
http://<deine-ip>/productpage

## bei mehrmals laden immer v3
```

6. Aufräumen

```
kubectl delete -n bookinfo httproute reviews --ignore-not-found
```

Reference:

- <https://istio.io/latest/docs/tasks/traffic-management/traffic-shifting/>
- <https://istio.io/latest/docs/ambient/usage/17-features/>

Debugging mit debug/run pod (Ambient: ztunnel + Waypoint)

Why like this ?

- Kein `istio-proxy` -Sidecar mehr im Pod (anders als im Sidecar-Modus) - ein Debug-Container läuft aber trotzdem im gleichen Pod-Netzwerk-Namespace und sieht damit denselben (durch `ztunnel` umgeleiteten) Traffic
- Zusätzlich gibt es Ambient-spezifische Debug-Kommandos, um zu prüfen, ob/wie ein Workload vom Mesh erfasst wird

Variante 1: Debug-Container zum Debuggen

- Debug Container in productpage - pod starten, um Verbindung zu pod -> Review zu debuggen

```
kubectl -n bookinfo get pods | grep productpage
## diesen entsprechend hier verwenden
kubectl -n bookinfo debug productpage-v1-54bb874995-rr7cv -it --image=busybox
```

```
## in der bash
wget -O - http://reviews:9080/reviews/1
```

```
exit
```

Erwartetes Ergebnis: Die Antwort kommt trotz `READY 1/1` (kein Sidecar) korrekt zurück - `ztunnel` leitet den Traffic transparent über das Istio-CNI-Plugin um.

V2 - Eigener Pod - Podtester

```
kubectl -n bookinfo run --rm -it podtester --image=busybox --overrides='{ "spec": { "serviceAccount": "bookinfo-productpage" } }'
```

V3 - Ambient-Aufnahme prüfen (istioctl ztunnel-config)

- Zeigt an, welcher `ztunnel` -Pod (auf welchem Node) einen Workload erfasst hat und ob ein Waypoint zwischengeschaltet ist

```
istioctl ztunnel-config workload -n istio-system | grep bookinfo
```

- Spalte `WAYPOINT`: `waypoint` = Layer 7 aktiv (HTTP-Routing/Policies möglich), `None` = nur Layer 4 (mTLS, aber kein HTTP-Verständnis)
- Spalte `PROTOCOL`: `HBONE` = über den Ambient-Tunnel geroutet, `TCP` = nicht vom Mesh erfasst (z.B. das Gateway selbst)

V4 - Ztunnel-Logs eines konkreten Nodes ansehen

```
NODE=$(kubectl -n bookinfo get pod -l app=productpage -o jsonpath='{.items[0].spec.nodeName}')
ZPOD=$(kubectl -n istio-system get pod -l app=ztunnel --field-selector spec.nodeName=$NODE -o jsonpath='{.items[0].metadata.name}')
echo "node=$NODE ztunnel=$ZPOD"

istioctl ztunnel-config log $ZPOD.istio-system
```

V5 - Waypoint-Proxy wie einen normalen Envoy debuggen

- Der Waypoint ist ein ganz normaler Envoy-Proxy - die klassischen `istioctl proxy-config` -Kommandos funktionieren genauso

```
WAYPOINT_POD=$(kubectl -n bookinfo get pod -l gateway.networking.k8s.io/gateway-name=waypoint -o jsonpath='{.items[0].metadata.name}')
istioctl proxy-config routes $WAYPOINT_POD.bookinfo
```

Reference

- <https://istio.io/latest/docs/ambient/usage/observability/>

GitOps mit Flux

ArgoCD vs. Flux CD im Ueberblick

Hintergrund

GitOps bedeutet: Der gewünschte Zustand des Clusters liegt versioniert in Git. Ein Controller im Cluster vergleicht laufend Soll (Git) und Ist (Cluster) und gleicht Abweichungen automatisch ab (Reconciliation).

```
Git-Repo (Soll)  <---- pull ----> GitOps-Controller im Cluster  ----> Cluster (Ist)
```

Vorteile:

- Nachvollziehbarkeit: Jede Aenderung ist ein Commit (Audit-Trail)
- Rollback = `git revert`
- Kein `kubectl apply` von Entwickler-Rechnern noetig (Pull- statt Push-Prinzip)
- Drift-Erkennung: manuelle Aenderungen im Cluster werden erkannt (und je nach Konfiguration zurueckgesetzt)

Die beiden grossen Player

Kriterium	ArgoCD	Flux CD
Projekt-Status	CNCF Graduated	CNCF Graduated
Web-UI	Ja, sehr ausgereift (Sync-Status, Diff, Rollback per Klick)	Nein (nur CLI; UIs von Drittanbietern, z.B. Weave GitOps)
CLI	<code>argocd</code>	<code>flux</code>
Kern-Konzept	Application (CRD) zeigt auf Repo/Pfad	GitRepository + Kustomization / HelmRelease (CRDs)

Helm-Support	Ja (rendert Charts zu Manifests)	Ja (HelmRelease mit echtem helm install/upgrade)
Multi-Cluster	Ja, zentrale Instanz kann viele Cluster bedienen	Ja, ueblicherweise 1 Flux pro Cluster
Multi-Tenancy	Projects, RBAC, SSO in der UI	ueber Kubernetes-RBAC und Namespaces
Image-Update-Automation	Separates Projekt (argocd-image-updater)	Eingebaut (Image Automation Controller)
Bootstrapping	Manuell (<code>kubectl apply</code> des Install-Manifests) oder argocd-autopilot	<code>flux bootstrap</code> (imperativer CLI-Befehl) oder Flux Operator + FluxInstance (deklarativ, empfohlen - siehe Installation)
Selbst-Update (Tool + Version)	Kein eingebauter Mechanismus - ArgoCD wird klassisch per Manifest/Helm aktualisiert. Es gibt einen argocd-operator (argoproj-labs, v.a. Basis fuer Red Hats OpenShift GitOps): vereinfacht die Installation ueber eine ArgoCD-CRD, aber <code>spec.version</code> ist ein fest gepinnter Image-Tag (kein Semver-Range-Autotracking wie bei Flux), und Operator-Selbst-Update gibt es nur ueber einen OLM-Auto-Update-Channel - OLM ist primaer ein OpenShift-Mechanismus, auf vanilla Kubernetes (wie unseren Clustern) zusätzlicher Aufwand	Flux Operator haelt sowohl die Flux-Version (<code>FluxInstance.spec.distribution.version: "2.x"</code>) als auch sich selbst (per eigener HelmRelease) automatisch aktuell - kein wiederkehrender <code>flux bootstrap</code> oder helm upgrade-Lauf noetig, siehe Installation
Typische Zielgruppe	Teams, die eine UI fuer Devs/Ops wollen	Plattform-Teams, die alles deklarativ/headless wollen

Wann was?

Ist das reine Geschmackssache? **Nein** - beide loesen das GitOps-Grundproblem gleich gut, aber es gibt reale technische Unterschiede, die die Wahl in konkreten Situationen vorwegnehmen. Nur ein Teil der Entscheidung ist tatsaechlich Praeferenz.

Fuer ArgoCD spricht:

- Eine Web-UI fuer Sync-Status, Diff und Rollback wird gebraucht (Devs ohne CLI-Affinitaet, Trainings/Demos, schnelles visuelles Troubleshooting)
- Eine zentrale Instanz soll viele Teams/Cluster mit RBAC/SSO bedienen (App Projects) - ein Ops-Team behaelt die UI-Uebersicht ueber alles
- PR-basierte Review-Workflows, bei denen der Diff am Ende auch visuell bestaetigt werden soll
- Kubernetes-/GitOps-Neulinge im Team - die UI senkt die Einstiegshuerde gegenueber "nur CLI + `kubectl get kustomization`" erheblich

Fuer Flux CD spricht - und mit Flux Operator noch mehr:

- Komplette deklarativ/headless gewuenscht, keine zusätzliche UI-Komponente im Cluster
- Die Plattform soll sich selbst warten: Der Flux Operator haelt sowohl die Flux-Version als auch sich selbst automatisch aktuell, ohne dass jemand `flux bootstrap` erneut anstossen oder ein manuelles `helm upgrade` fahren muss (siehe [Installation](#)). Das zaehlt sich besonders aus, wenn es **viele gleichartige Cluster** gibt (wie hier: ein Cluster pro Teilnehmer) - zentrale, manuelle Wartung pro Cluster skaliert dann nicht
- Helm-Releases sollen "echt" mit Helm-Lifecycle laufen (nicht nur gerenderte Manifeste wie bei ArgoCD)
- Automatische Image-Updates ohne Zusatzprojekt gewuenscht

Tatsaechlich Geschmackssache/Kontext:

- Multi-Tenancy: beide loesen das (ArgoCD ueber Projects+RBAC in der UI, Flux ueber Kubernetes-Bordmittel/Namespaces) - was passender ist, haengt vom bereits vorhandenen RBAC-Modell im Team ab
- Vorhandenes Know-how/Toolchain im Team (schon Erfahrung mit dem einen oder anderen Tool vorhanden)

Fazit

Der Flux Operator veraendert vor allem die **Betriebs-Story** von Flux (Selbstwartung von Tool und Version), nicht das GitOps-Grundprinzip selbst. Wer eine UI braucht oder zentral viele Cluster/Teams bedienen muss, greift weiterhin eher zu ArgoCD. Wer moeglichst wenig manuellen Betriebsaufwand fuer das GitOps-Tool selbst haben will - kein wiederkehrendes `flux bootstrap`, kein manuelles Upgrade des Tools - fuer den macht der Flux Operator Flux CD gegenueber ArgoCD noch einmal deutlicher attraktiver als vorher.

Weiter geht es praktisch

- [Flux Ueberblick - Controller, CRDs und Ablauf](#)
- [Flux Installation mit dem Flux Operator](#)
- [Was ist ArgoCD?](#)
- [Hands-on: Deployment mit ArgoCD](#)

Flux Ueberblick - Controller, CRDs und Ablauf

Hintergrund

Flux ist neben ArgoCD das zweite grosse GitOps-Tool fuer Kubernetes (Vergleich siehe [argocd-vs-flux.md](#)). Flux arbeitet "pull-based":

1. Quelle beobachten (Git/OCI/Helm-Repo/S3-Bucket)
2. Artefakt bauen (z.B. tar.gz mit Repo-Snapshot oder Helm-Chart-Artifact)
3. Zielzustand ableiten (Kustomize oder Helm)
4. Ist-Zustand im Cluster angleichen (apply/upgrade)
5. Wiederholen (in Intervallen), inkl. Status/Events

Flux "macht nichts einmalig", sondern reconciled immer wieder, bis Ist = Soll.

Die Flux-Controller und ihre Aufgaben

Flux besteht aus mehreren Controllern (Deployments), die jeweils bestimmte CRDs beobachten und reconciled ausfuehren:

source-controller (Quellen + Artefakte)

- Holt Inhalte aus Git, OCI, HelmRepositories oder Buckets
- Erzeugt versionierte Artefakte und stellt sie fuer andere Controller bereit
- CRDs: `GitRepository` , `OCIRepository` , `HelmRepository` , `Bucket` , `HelmChart`

helm-controller (Helm Releases)

- Installiert/Upgraded/Uninstallt Helm Releases anhand von `HelmRelease`
- Nutzt Chart-Artefakte aus dem source-controller als Input
- CRD: `HelmRelease`

kustomize-controller (YAML / Kustomize)

- Rendert und applied Kubernetes-Ressourcen aus einem Source-Artefakt
- Unterstuetzt Kustomize (Overlays, Patches, Images)
- CRD: `Kustomization`

notification-controller (Events/Alerts/Webhooks)

- Sendet Benachrichtigungen ueber Zustandsaenderungen (Slack/Webhook/Teams)
- Kann Webhooks empfangen (GitHub/GitLab) und dadurch sofort reconcilen
- CRDs: `Provider` , `Alert` , `Receiver`

image-reflector-controller und image-automation-controller

- Scannen Container-Registries nach neuen Image-Tags (`ImageRepository` , `ImagePolicy`) und schreiben Updates automatisiert zurueck ins Git-Repo (`ImageUpdateAutomation`)
- Sind Teil des Standard-Helm-Charts, werden in dieser Uebung aber nicht konfiguriert

Ablauf am Beispiel Helm Chart

Ziel: Ein Helm Chart (z.B. traefik) deklarativ ausrollen.

Beteiligte Objekte:

1. `HelmRepository` - Chart-Repo als Quelle
2. `HelmRelease` - Release-Definition

A) source-controller:

1. `HelmRepository` wird reconciled: source-controller laedt `index.yaml` des Chart-Repos, speichert Status/Revision
2. Fuer eine `HelmRelease` wird intern ein `HelmChart` -Artifact erzeugt

B) helm-controller: 3. helm-controller reconciled die `HelmRelease` : holt das Chart-Artefakt, rendert Templates mit `values` , fuehrt intern ein `helm upgrade --install` aus 4. Status wird im `HelmRelease` -Objekt aktualisiert (Conditions, letzte erfolgreiche Revision)

C) Wiederholung: 5. Neue Chart-Version im Repo oder Werte-Aenderung -> naechster Reconcile 6. Flux sorgt dafuer, dass der Cluster-Zustand wieder dem gewuenschten Zustand entspricht

Die folgenden Uebungen bauen genau diesen Ablauf Schritt fuer Schritt auf: Installation, `HelmRepository` , `HelmRelease` , `OCIRepository` und ein eigenes Chart aus einem Git-Repo.

Flux Installation und GitOps-Sync mit dem Flux Operator

Hintergrund

Frueher haette man Flux imperativ per `flux bootstrap gitlab ...` installiert: Ein einmaliger CLI-Befehl generiert die Controller-Manifeste, committed sie ins Git-Repo, installiert die Controller und richtet ein `GitRepository` + eine `Kustomization` als Git-Sync ein. Inzwischen wird stattdessen der Flux Operator **empfohlen** (siehe auch [ArgoCD vs. Flux CD](#)):

In dieser Uebung macht das stattdessen der **Flux Operator** (von ControlPlane, <https://fluxoperator.dev>): Ihr installiert per Helm nur einen kleinen Operator. Der Operator liest daraus eine `FluxInstance` Custom Resource - deklarativ, per `kubectl apply` /Git statt per CLI-Befehl - und rollt darauf basierend die eigentlichen Flux-Controller samt Git-Sync aus.

Das bringt zwei Automatik-Effekte, die `flux bootstrap` nicht hat:

1. **Flux selbst aktualisiert sich automatisch:** `spec.distribution.version: "2.x"` in der `FluxInstance` heisst "immer die neueste 2.x-Version" - der Operator rollt neue Flux-Patches/Minor-Releases selbststaendig aus, ohne dass ihr `flux bootstrap` erneut ausfuehren muesst.
2. **Der Operator aktualisiert sich selbst:** Sobald der Git-Sync steht, committen wir eine `HelmRelease` , die den Flux-Operator-Chart selbst per Semver-Range trackt. Ab dann uebernimmt Flux die eigene Operator-Version - ein neuer Chart-Release wird automatisch ausgerollt, ganz ohne erneuten `helm upgrade` .

Komponente	Version
Flux CLI	2.9.5 (Stand 10.09.2026)
Flux Operator (Helm Chart)	0.59.0 (Stand 09.09.2026)
Flux Distribution (ueber FluxInstance)	2.9.5 (Stand 09.09.2026)

Voraussetzungen

- Eigenes Kubernetes-Cluster (jeder Teilnehmer hat sein eigenes)
- `kubectl` und `helm` konfiguriert
- Eigener GitLab.com-Account `training.tn<deine-nr>` (vom Trainer angelegt)

Schritt 1: Flux CLI installieren

Wird hier nicht fuer die Installation gebraucht, aber fuer Reconcile-/Status- Befehle in den naechsten Uebungen:

```
curl -s https://fluxcd.io/install.sh | sudo bash
```

```
flux version --client
```

Schritt 2: Personal Access Token erstellen

Auf gitlab.com unter https://gitlab.com/-/user_settings/personal_access_tokens (als `training.tn<deine-nr>` eingeloggt):

- Scope: `api` (**legacy - token**)
- Name z.B. `flux-sync`

Token kopieren und als Umgebungsvariable setzen:

```
export GITLAB_TOKEN=<dein-personal-access-token>
```

Schritt 3: GitLab-Repo einrichten und lokal klonen

Neues, leeres Projekt anlegen: https://gitlab.com/projects/new#blank_project, Name z.B. `flux-<dein-kuerzel>`.

```
cd
git clone https://gitlab.com/training.tn<deine-nr>/flux-<dein-kuerzel>.git
cd flux-<dein-kuerzel>
```

```
git config user.email "training@example.com"
git config user.name "training.tn<deine-nr>"
```

Unterschied zu `flux bootstrap`: Dort generiert der CLI-Befehl die Manifeste automatisch und committed sie ins Repo. Hier legt ihr die Manifeste selbst an und committed sie - genau der Workflow, den ihr auch in den naechsten Uebungen (`HelmRepository`, `HelmRelease`, ...) verwendet.

Schritt 4: Flux Operator per Helm installieren

Der Operator kommt als Helm Chart aus einer OCI-Registry - kein zusaeztliches Repo hinzufuegen noetig:

```
helm install flux-operator oci://ghcr.io/controlplaneio-fluxcd/charts/flux-operator \
-n flux-system --create-namespace --wait --timeout 3m
```

```
kubectl get pods -n flux-system
```

Erwartete Ausgabe (nur der Operator, noch keine Flux-Controller):

NAME	READY	STATUS	RESTARTS	AGE
flux-operator-xxxxxxxx-xxxxx	1/1	Running	0	20s

Schritt 5: Pull-Secret fuer den Git-Sync anlegen

Damit der Operator euer GitLab-Repo lesen kann, braucht er ein Secret mit Benutzername + Token:

```
kubectl create secret generic flux-system \
--namespace=flux-system \
--from-literal=username=training.tn<deine-nr> \
--from-literal=password=$GITLAB_TOKEN
```

Schritt 6: FluxInstance mit Git-Sync anlegen

Die `FluxInstance` beschreibt, welche Flux-Version, welche Controller UND welches Git-Repo als Sync-Quelle gewuenscht sind:

```
cd
mkdir -p flux-<dein-kuerzel>/clusters/production/flux-system
cd flux-<dein-kuerzel>/clusters/production/flux-system
```

```
## vi fluxinstance.yml
apiVersion: fluxcd.controlplane.io/v1
kind: FluxInstance
metadata:
  name: flux
  namespace: flux-system
spec:
  distribution:
    version: "2.x"
    registry: "ghcr.io/fluxcd"
  components:
    - source-controller
    - kustomize-controller
    - helm-controller
    - notification-controller
  cluster:
    type: kubernetes
    multitenant: false
```

```

    networkPolicy: true
    domain: "cluster.local"
  sync:
    kind: GitRepository
    url: "https://gitlab.com/training.tn<deine-nr>/flux-<dein-kuerzel>.git"
    ref: "refs/heads/main"
    path: "clusters/production"
    pullSecret: "flux-system"

```

Diese allererste Anwendung muss per `kubectl apply` passieren - es laeuft ja noch kein Flux, das einen Git-Commit einlesen koennte:

```
kubectl apply -f fluxinstance.yml
```

Danach committen wir dieselbe Datei in den Pfad, den die `FluxInstance` gerade als Sync-Quelle eingerichtet hat - ab jetzt verwaltet Flux sich damit selbst weiter, jede kuenftige Aenderung an der `FluxInstance` laeuft ueber `git commit + git push`:

```

cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added FluxInstance with git sync"
git push

```

Schritt 7: Operator-Selbst-Update einrichten

Jetzt richten wir den zweiten Automatik-Effekt ein: Der Flux-Operator-Chart selbst wird ab sofort per `HelmRelease` von Flux verwaltet, mit einer Semver-Range, die neue Chart-Releases automatisch uebernimmt.

```
cd flux-<dein-kuerzel>/clusters/production/flux-system
```

```

## vi flux-operator-source.yml
apiVersion: source.toolkit.fluxcd.io/v1
kind: OCIRepository
metadata:
  name: flux-operator
  namespace: flux-system
spec:
  interval: 30m
  url: oci://ghcr.io/controlplaneio-fluxcd/charts/flux-operator
  ref:
    semver: ">=0.59.0"

```

```

## vi flux-operator-release.yml
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: flux-operator
  namespace: flux-system
spec:
  interval: 30m
  releaseName: flux-operator
  chartRef:
    kind: OCIRepository
    name: flux-operator
    namespace: flux-system

```

Wichtig: Der Release-Name (`releaseName: flux-operator`) ist bewusst identisch mit der Helm-Installation aus Schritt 4. Flux erkennt die bestehende Helm-Release-Storage und uebernimmt sie nahtlos - ohne Neuinstallation.

```

cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added self-update HelmRelease for flux-operator"
git push

```

```
flux reconcile kustomization flux-system --with-source
```

Schritt 8: Installation verifizieren

```
kubectl get pods -n flux-system
```

Erwartete Ausgabe (Operator + 4 Flux-Controller):

NAME	READY	STATUS	RESTARTS	AGE
flux-operator-xxxxxxxx-xxxxx	1/1	Running	0	3m
helm-controller-xxxxxxxx-xxxxx	1/1	Running	0	90s
kustomize-controller-xxxxxxxx-xxxxx	1/1	Running	0	90s

notification-controller-xxxxxxxx-xxxxx	1/1	Running	0	90s
source-controller-xxxxxxxx-xxxxx	1/1	Running	0	90s

Status der `FluxInstance` und des Git-Syncs:

```
kubectl get fluxinstance -n flux-system
flux get sources git
flux get kustomizations
```

Erwartete Ausgabe der `FluxInstance` :

NAME	AGE	READY	STATUS	REVISION
flux	3m	True	Reconciliation finished in 21s	v2.9.5@sha256:...

Status des Operator-Selbst-Updates:

```
kubectl get ocirepository,helmrelease -n flux-system flux-operator
```

Was wurde eingerichtet?

1. Der Flux Operator (Schritt 4) plus die 4 Flux-Controller, die er anhand der `FluxInstance` ausgerollt hat
2. Ein `GitRepository` - und `Kustomization` -Objekt `flux-system` , das auf euer eigenes GitLab-Repo zeigt (von der `FluxInstance` per `spec.sync` erzeugt - dieselben Namen wie bei `flux bootstrap`)
3. Eine `OCIRepository` + `HelmRelease` , die den Flux-Operator-Chart selbst per Semver-Range trackt und bei neuen Releases automatisch upgraded

Im naechsten Schritt legt ihr ein `HelmRepository` an - nicht per `kubectl apply` , sondern per Commit in dieses Repo (genau wie in Schritt 6/7 gerade schon gemacht).

Vergleich zu `flux bootstrap`

	<code>flux bootstrap</code>	Flux Operator + <code>FluxInstance</code>
Installation	Imperativer CLI-Befehl	Deklarative Custom Resource
Manifeste ins Repo committen	Automatisch durch den CLI-Befehl	Selbst angelegt und committed
Flux-Version aktuell halten	<code>flux bootstrap</code> erneut ausfuehren	Automatisch (<code>version: "2.x"</code>)
Operator/Werkzeug selbst aktuell halten	Entfaellt (kein separater Operator)	Automatisch (eigene <code>HelmRelease</code>)
Voraussetzung	Flux CLI + Git-Token	Helm + Git-Token

HelmRepository - Helm Chart Repositories verwalten

Hintergrund

`HelmRepository` ist die Flux-CRD, die ein Helm-Chart-Repository als Quelle definiert. Der source-controller ueberwacht diese Ressource und laedt periodisch den Repository-Index (`index.yaml`).

Eigenschaft	Beschreibung
API Group	<code>source.toolkit.fluxcd.io/v1</code>
Controller	<code>source-controller</code>
Funktion	Helm-Chart-Repository-Index bereitstellen
Update	Periodisch (<code>interval</code>)

Voraussetzungen

- Flux Operator installiert, Git-Sync eingerichtet (siehe [02-installation.md](#))
- Euer Repo `flux-<dein-kuerzel>` lokal geklont

Schritt 1: Vorbereitung

```
cd
cd flux-<dein-kuerzel>
mkdir -p clusters/production/infrastructure
cd clusters/production/infrastructure
```

Schritt 2: HelmRepository fuer Traefik erstellen

```
## vi 01-traefik-repo.yml
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: traefik
  namespace: flux-system
spec:
  interval: 10m
  url: https://traefik.github.io/charts
```

Nicht `kubect1 apply` - stattdessen committen und pushen:

```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added HelmRepository traefik"
git push
```

Schritt 3: Reconciliation abwarten

Flux zieht Aenderungen automatisch (Standard-Intervall der Kustomization). Fuer die Uebung koennt ihr das manuell anstossen:

```
flux reconcile kustomization flux-system --with-source

kubect1 get helmrepository -n flux-system
```

Erwartete Ausgabe:

NAME	URL	AGE	READY	STATUS
traefik	https://traefik.github.io/charts	15s	True	stored artifact: revision 'sha256:...'

Erklaerung:

Feld	Wert	Bedeutung
interval	10m	Alle 10 Minuten Index neu laden
url	https://...	Helm-Chart-Repository-URL
READY	True	Repository-Index erfolgreich geladen

Was passiert im Hintergrund?

- 1. `git push` bringt die Aenderung ins GitLab-Repo
- 2. Der source-controller im Cluster erkennt (per `GitRepository flux-system`) den neuen Commit
- 3. Der kustomize-controller wendet die neuen Manifeste aus `clusters/production/` an (das erstellt hier das `HelmRepository` -Objekt)
- 4. Der source-controller laedt daraufhin `index.yaml` vom Traefik-Repo

Naechster Schritt

Im naechsten Schritt ([04-helmrelease.md](#)) nutzt ihr dieses `HelmRepository` , um tatsaechlich ein Helm Chart mit `HelmRelease` auszurollen.

Aufräumen

Nicht ausfuehren, falls ihr direkt mit der naechsten Uebung weitermacht - die `HelmRelease` braucht das `HelmRepository` als Quelle.

```
rm clusters/production/infrastructure/01-traefik-repo.yml
git add -A
git commit -m "Removed HelmRepository traefik"
git push
```

HelmRelease - Helm Charts deklarativ ausrollen

Hintergrund

`HelmRelease` ist die zentrale Flux-CRD zum Ausrollen von Helm Charts. Der helm-controller reconciled diese Ressource und fuehrt intern `helm upgrade --install` aus.

Eigenschaft	Beschreibung
API Group	helm.toolkit.fluxcd.io/v2
Controller	helm-controller
Funktion	Helm Release deklarativ verwalten
Reconciliation	Automatische Upgrades bei Aenderungen

Voraussetzungen

- `HelmRepository` `traefik` existiert (siehe [03-helmrepository.md](#))

Schritt 1: Namespace anlegen

Der Namespace fuer das `HelmRelease` -Objekt muss existieren, bevor Flux das Objekt darin anlegen kann - das ist reines Kubernetes-Verhalten und gilt unabhaengig von GitOps:

```
kubect1 create namespace ingress
```

Schritt 2: HelmRelease fuer Traefik erstellen

```
cd
cd flux-<dein-kuerzel>/clusters/production/infrastructure
```

```
## vi 02-traefik-release.yml
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: traefik
  namespace: ingress
spec:
  interval: 5m
  chart:
    spec:
      chart: traefik
      sourceRef:
        kind: HelmRepository
        name: traefik
        namespace: flux-system
  values:
    replicas: 2
```

```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added HelmRelease traefik"
git push
```

Schritt 3: Status pruefen

```
flux reconcile kustomization flux-system --with-source
```

```
kubectl get helmrelease -n ingress
```

Erwarteter Fehler:

NAME	AGE	READY	STATUS
traefik	30s	False	Helm install failed for release ingress/traefik with chart traefik@41.4.0: ...

```
kubectl -n ingress describe helmrelease traefik
```

```
Warning InstallFailed ... helm-controller Helm install failed for release ingress/traefik with chart traefik@41.4.0: values
don't meet the specifications of the schema(s) in the following chart(s):
traefik:
- at '': additional properties 'replicas' not allowed
```

Das Traefik-Chart erwartet `deployment.replicas`, nicht `replicas` auf oberster Ebene. Um das richtige Feld zu finden, hilft ein Blick auf artifacthub.io (Values-Schema des Charts).

Schritt 4: Values korrigieren

```
## vi 02-traefik-release.yml
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: traefik
  namespace: ingress
spec:
  interval: 5m
  chart:
    spec:
      chart: traefik
      sourceRef:
        kind: HelmRepository
        name: traefik
        namespace: flux-system
  values:
    deployment:
      replicas: 2
```

```
git add -A
git commit -m "Fixed values schema for traefik"
git push
```

Schritt 5: Erfolg pruefen

```
flux reconcile kustomization flux-system --with-source
```

```
kubectl get helmrelease -n ingress
kubectl -n ingress get pods
helm -n ingress list
```

Erwartete Ausgabe:

```
NAME      AGE   READY   STATUS
traefik   63s   True    Helm install succeeded for release ingress/traefik.v1 with chart traefik@41.4.0

NAME      READY   STATUS    RESTARTS   AGE
traefik-... 1/1     Running   0          27s
traefik-... 1/1     Running   0          27s
```

Erklärung:

Feld	Wert	Bedeutung
interval	5m	Pruefe alle 5 Minuten auf Drift/Updates
chart	traefik	Chart-Name aus dem Repository
sourceRef	traefik	Referenz auf das HelmRepository
values	...	Ueberschreibt die Chart-Default-Values

Details des HelmRelease anzeigen

```
kubectl get helmrelease traefik -n ingress -o yaml | grep -A 10 status
```

Wichtige Felder: `lastAttemptedRevision` (letzte versuchte Version), `conditions` (Status der Reconciliation).

Naechster Schritt

Im naechsten Schritt ([05-oci-helm-chart.md](#)) rollt ihr ein Helm Chart aus einer OCI-Registry aus.

Aufraeumen

```
rm clusters/production/infrastructure/02-traefik-release.yml
git add -A
git commit -m "Removed HelmRelease traefik"
git push
kubectl delete namespace ingress
```

OCI-Helm-Chart verwenden

Hintergrund

Helm Charts koennen auch direkt aus einer OCI-Registry (z.B. Docker Hub, GHCR) bezogen werden - ohne klassisches HTTP-Chart-Repository. Flux bildet das mit der CRD `OCIRepository` ab.

Eigenschaft	Beschreibung
API Group	<code>source.toolkit.fluxcd.io/v1</code>
Controller	<code>source-controller</code>
Funktion	OCI-Artifact (Helm Chart) als Quelle bereitstellen

Voraussetzungen

- Flux Operator installiert, Git-Sync eingerichtet (siehe [02-installation.md](#))

Schritt 1: OCIRepository einrichten

```
cd
cd flux-<dein-kuerzel>/clusters/production/infrastructure

## vi 03-mariadb-ocirepo.yml
apiVersion: source.toolkit.fluxcd.io/v1
kind: OCIRepository
metadata:
  name: cloudpirates-mariadb
  namespace: flux-system
spec:
  interval: 10m
  url: oci://registry-1.docker.io/cloudpirates/mariadb
  ref:
    version: "0.14.1"
```



```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added OCIRepository cloudpirates-mariadb"
git push
```

Schritt 2: Funktioniert nicht - warum?

```
flux reconcile kustomization flux-system --with-source
```

```
kubectl -n flux-system get ocirepositories
```

Erwarteter Fehler:

```
Error from server (BadRequest): OCIRepository in version "v1" cannot be handled as a OCIRepository: strict decoding error: unknown field "spec.ref.version"
```

Das Feld heisst nicht `version`, sondern `tag`. Nachschauen, welche Felder unter `ref` erlaubt sind:

```
kubectl explain OCIRepository.spec.ref
```

Ausgabe zeigt u.a. `tag`, `semver`, `digest` - aber kein `version`.

Schritt 3: Feld korrigieren

```
## vi 03-mariadb-ocirepo.yml
apiVersion: source.toolkit.fluxcd.io/v1
kind: OCIRepository
metadata:
  name: cloudpirates-mariadb
  namespace: flux-system
spec:
  interval: 10m
  url: oci://registry-1.docker.io/cloudpirates/mariadb
  ref:
    tag: "0.14.1"
```

```
git add -A
git commit -m "Fixed OCIRepository ref field"
git push
```

```
flux reconcile kustomization flux-system --with-source
```

```
kubectl -n flux-system get ocirepositories
```

Erwartete Ausgabe:

NAME	URL	READY	STATUS
cloudpirates-mariadb	oci://registry-1.docker.io/cloudpirates/mariadb	True	stored artifact for digest '0.14.1@sha256:...'

Schritt 4: HelmRelease anlegen

Bei einem `OCIRepository` wird `chartRef` statt `chart.spec` verwendet, weil die Chart-Version bereits im `OCIRepository` festgelegt ist.

```
## vi 04-mariadb-release.yml
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: mariadb
  namespace: default
spec:
  interval: 10m
  chartRef:
    kind: OCIRepository
    name: cloudpirates-mariadb
    namespace: flux-system
  values:
    persistence:
      enabled: false
```

Hinweis: `persistence.enabled: false` ist hier bewusst gesetzt - ohne `StorageClass` im Cluster wuerde der Pod sonst mit einer unbound `PersistentVolumeClaim` in `Pending` haengen bleiben. Fuer eine echte `Persistence`-Uebung braeuchte es eine `StorageClass` (z.B. per NFS-CSI-Treiber)

- das ist fuer diese Uebung bewusst ausgeklammert.

```
git add -A
git commit -m "Added HelmRelease mariadb"
git push
```

Schritt 5: War die Installation erfolgreich?

```
flux reconcile kustomization flux-system --with-source
```

```
kubectl get helmrelease -n default
kubectl get pods -n default
helm -n default status mariadb
```

Erwartete Ausgabe:

NAME	AGE	READY	STATUS	
mariadb	90s	True	Helm install succeeded for release default/mariadb.v1 with chart mariadb@0.14.1+...	
NAME	READY	STATUS	RESTARTS	AGE
mariadb-0	1/1	Running	0	44s

Naechster Schritt

Im naechsten Schritt ([06-eigenes-helmchart.md](#)) rollt ihr ein eigenes Helm Chart aus einem eigenen Git-Repository aus.

Aufraeumen

```
rm clusters/production/infrastructure/03-mariadb-ocirepo.yml
rm clusters/production/infrastructure/04-mariadb-release.yml
git add -A
git commit -m "Removed mariadb OCIRepository and HelmRelease"
git push
```

Eigenes Helm Chart aus Git-Repository ausrollen

Hintergrund

`GitRepository` ist die Flux-CRD fuer Git als Quelle - damit koennt ihr auch ein eigenes, selbst geschriebenes Helm Chart per Flux ausrollen, ohne es vorher in eine Chart-Registry zu veroeffentlichen.

Eigenschaft	Beschreibung
API Group	<code>source.toolkit.fluxcd.io/v1</code>
Controller	<code>source-controller</code>
Funktion	Git-Repository als Quelle bereitstellen

Voraussetzungen

- Flux Operator installiert, Git-Sync eingerichtet (siehe [02-installation.md](#))

Schritt 1: Chart erstellen

```
cd
mkdir helm-chart-test
cd helm-chart-test
helm create final-chart
```

Schritt 2: Neues, leeres GitLab-Repo anlegen

Auf `gitlab.com` als `training.tn<deine-nr>` :neues Projekt `final-chart-<dein-kuerzel>` anlegen, **Sichtbarkeit: Public** (der source-controller braucht sonst Zugangsdaten fuer dieses zweite Repo, was wir uns fuer die Uebung sparen).

Schritt 3: Chart lokal committen und pushen

```
cd final-chart
git init
git config user.email "training@example.com"
git config user.name "training.tn<deine-nr>"
git remote add origin https://gitlab.com/training.tn<deine-nr>/final-chart-<dein-kuerzel>.git
git add -A
git commit -m "Chart hochschicken"
git branch -M main
git push -u origin main
```

Bei der ersten Push-Authentifizierung: Username `training.tn<deine-nr>` , Passwort der Personal Access Token aus [02-installation.md](#).

Schritt 4: GitRepository im Flux-Repo anlegen

```
cd
cd flux-<dein-kuerzel>
mkdir -p clusters/production/apps
cd clusters/production/apps
```

```
## vi 01-final-chart-gitrepo.yml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: final-chart-source
  namespace: flux-system
spec:
  interval: 1m
  url: https://gitlab.com/training.tn<deine-nr>/final-chart-<dein-kuerzel>.git
  ref:
    branch: main
```

```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added GitRepository for own chart"
git push
```

Schritt 5: Ueberpruefen

```
flux reconcile kustomization flux-system --with-source
```

```
flux get sources git
```

Erwartete Ausgabe (Auszug):

NAME	REVISION	READY
final-chart-source	main@shal:...	True

Schritt 6: HelmRelease einpflegen

```
cd flux-<dein-kuerzel>/clusters/production/apps
```

```
## vi 02-final-chart-release.yml
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: final-chart
  namespace: flux-system
spec:
  interval: 1m
  targetNamespace: final-chart-demo
  install:
    createNamespace: true
  chart:
    spec:
      chart: ./
      sourceRef:
        kind: GitRepository
        name: final-chart-source
        namespace: flux-system
  values:
    replicaCount: 2
```

```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added HelmRelease for own chart"
git push
```

Erklärung: chart.spec.chart: ./ **verweist auf das Wurzelverzeichnis des GitRepository** - das Chart liegt hier direkt im Repo-Root. Liegt das Chart in einem Unterordner (z.B. charts/final-chart), muss der Pfad entsprechend angepasst werden.

Schritt 7: Ueberpruefen

```
flux reconcile kustomization flux-system --with-source
```

```
flux get source git
flux get helmrelease -A
```

```
helm list -A
kubectl -n final-chart-demo get pods
```

Erwartete Ausgabe:

```
NAME          AGE   READY   STATUS
final-chart   ...   True    Helm install succeeded for release final-chart-demo/final-chart-demo-final-chart.v1 with chart final-chart@0.1.0

NAME                                READY   STATUS    RESTARTS   AGE
final-chart-demo-final-chart-...    1/1     Running   0           28s
final-chart-demo-final-chart-...    1/1     Running   0           28s
```

Zusammenfassung der gesamten Uebungsreihe

CRD	Zweck
HelmRepository	HTTP-Chart-Repository als Quelle
HelmRelease (mit chart.spec)	Helm Release aus HelmRepository
OCIRepository	Helm Chart aus OCI-Registry als Quelle
HelmRelease (mit chartRef)	Helm Release aus OCIRepository
GitRepository	Eigenes/fremdes Git-Repo als Quelle
HelmRelease (mit chart.spec + GitRepository)	Helm Release aus einem Chart-Pfad im Git-Repo

Der gesamte Workflow lief dabei ausschliesslich ueber `git commit + git push` in euer Flux-Repo - kein einziges `kubectl apply` fuer die eigentlichen GitOps-Objekte. Genau das ist der Kern von GitOps: Git ist die "Source of Truth", Flux gleicht den Cluster automatisch daran an.

Aufräumen

```
cd flux-<dein-kuerzel>
rm clusters/production/apps/01-final-chart-gitrepo.yml
rm clusters/production/apps/02-final-chart-release.yml
git add -A
git commit -m "Removed own chart GitRepository and HelmRelease"
git push
```

Das GitLab-Repo `final-chart-<dein-kuerzel>` koennt ihr danach in den Projekt-Einstellungen loeschen.

Uebung: Flux-Operator Web-UI mit Ingress und HTTPS absichern

Hintergrund

Der Flux Operator bringt seit Version 0.59 eine eingebaute Web-UI mit ("Flux Status Page") - sie ist per Default aktiv (`web.enabled: true`), laeuft im selben Pod wie der Operator und ist ueber den Service `flux-operator` auf Port `9080` (Name `http-web`) erreichbar. Ohne Ingress ist das nur clusterintern nutzbar.

Wir machen die UI von aussen erreichbar - genau wie bei [Prometheus/Grafana](#): Traefik als Ingress-Controller, TLS von Letsencrypt, Zugriff per basic-auth.

Der Punkt dabei: Fuer die Zertifikats-Ausstellung braucht es keinen neuen `ClusterIssuer` - das Objekt aus der Prometheus/Grafana-Uebung (`letsencrypt-prod`) ist nicht an einen Host gebunden, sondern gilt clusterweit fuer jeden Ingress mit `ingressClassName: traefik` . Derselbe "Handler" validiert also auch dieses Zertifikat per HTTP01-Challenge.

Nebeneffekt: Was die UI anzeigt, ist keine eigene Datensammlung, sondern die `FluxReport` -Custom-Resource des Operators - dieselbe, die man auch per `kubectl` abfragen kann. Die UI ist im Kern also nur eine Visualisierung von Schritt 5 dieser Uebung.

Voraussetzungen

- Flux Operator + Git-Sync laufen ([02-installation.md](#)), inklusive Schritt 7 (Operator verwaltet sich selbst per `HelmRelease`)
- Traefik laeuft ueber die `HelmRelease` aus [04-helmrelease.md](#)
- `cert-manager` + `ClusterIssuer` `letsencrypt-prod` aus der Prometheus/Grafana-Uebung sind noch vorhanden:

```
kubectl get clusterissuer letsencrypt-prod
```

Falls nicht mehr da (z.B. schon aufgeraemt): dort Schritt 5+6 nachholen, bevor es hier weitergeht.

- Wildcard-DNS `*.<du>.do.t3isp.de` aus derselben Uebung zeigt schon auf die Traefik-IP - deckt automatisch auch `flux.<du>.do.t3isp.de` ab, kein neuer DNS-Eintrag noetig
- `htpasswd` ist installiert (`apt install apache2-utils` , siehe Prometheus/Grafana-Uebung)

Schritt 1: basic-auth-Secret anlegen

Das Secret legen wir bewusst per `kubectl` an, nicht per Git-Commit - Passwoerter gehoeren nicht im Klartext ins Git-Repo, auch nicht in ein `privates`.

```
htpasswd -c auth admin # Wunsch-Passwort eingeben
kubectl create secret generic flux-web-basic-auth --from-file=users=auth -n flux-system
```

Schritt 2: Middleware fuer Traefik committen

Anders als das Secret enthaelt die `Middleware` keine Geheimnisse, nur eine Referenz auf den Secret-Namen - die kann ganz normal ueber Git laufen, wie alles andere in diesem Kapitel.

```
cd
cd flux-<dein-kuerzel>/clusters/production/flux-system
```

```
## vi flux-web-middleware.yml
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata:
  name: flux-web-auth
  namespace: flux-system
spec:
  basicAuth:
    secret: flux-web-basic-auth
```

```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added basic-auth middleware for flux-operator web UI"
git push
```

```
flux reconcile kustomization flux-system --with-source
```

Schritt 3: Ingress fuer die Web-UI ueber das bestehende HelmRelease aktivieren

Wir tragen die Ingress-Konfiguration im `values`-Feld genau der `HelmRelease` ein, die den Flux-Operator-Chart schon selbst verwaltet (aus [02-installation.md](#), Schritt 7) - bisher hatte sie noch kein `values`-Feld.

```
cd flux-<dein-kuerzel>/clusters/production/flux-system
```

```
## vi flux-operator-release.yml
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: flux-operator
  namespace: flux-system
spec:
  interval: 30m
  releaseName: flux-operator
  chartRef:
    kind: OCIRepository
    name: flux-operator
    namespace: flux-system
  values:
    web:
      ingress:
        enabled: true
        className: traefik
        annotations:
          cert-manager.io/cluster-issuer: letsencrypt-prod
          traefik.ingress.kubernetes.io/router.middlewares: flux-system-flux-web-auth@kubernetescrd
        hosts:
          - host: flux.<du>.do.t3isp.de
            paths:
              - path: /
                pathType: Prefix
      tls:
        - hosts:
            - flux.<du>.do.t3isp.de
          secretName: flux-web-tls
```

Achtung, Stolperstein: Das Feld heisst hier `className`, nicht `ingressClassName` wie bei jedem anderen Chart in diesem Training (Prometheus, Grafana, Alertmanager, Traefik selbst). Jedes Helm Chart definiert sein eigenes values-Schema - es gibt dafuer keine Kubernetes-weite Konvention. Im Zweifel im Chart selbst nachsehen: [fluxoperator.dev/docs/charts/flux-operator](#) bzw. [artifacthub.io](#).

```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Added ingress for flux-operator web UI"
git push
```

```
flux reconcile kustomization flux-system --with-source
```

Schritt 4: Status pruefen

```
kubectl -n flux-system get helmrelease flux-operator
kubectl -n flux-system get ingress
kubectl -n flux-system get certificate flux-web-tls
```

Der `Ingress` steht sofort. Das Zertifikat bleibt aber haengen:

NAME	READY	SECRET	AGE
flux-web-tls	False	flux-web-tls	3m

Schritt 5: Erwarteter Fehler - Challenge haengt fest

```
kubectl -n flux-system get challenges
kubectl -n flux-system describe challenge <name-aus-get-challenges>
```

Erwarteter Auszug:

```
Status:
  Reason: Waiting for HTTP-01 challenge propagation: failed to perform
           self check GET request '.../.well-known/acme-challenge/...':
           context deadline exceeded (Client.Timeout exceeded while
           awaiting headers)
  State: pending
```

Ursache: Der Flux Operator legt (weil die FluxInstance aus [02-installation.md](#) `spec.cluster.networkPolicy: true` setzt) automatisch mehrere `NetworkPolicy`-Objekte im Namespace `flux-system` an - unter anderem `allow-egress`, die per `podSelector: {}` fuer JEDEN Pod im Namespace gilt und eingehenden Traffic nur noch von Pods **im selben Namespace** erlaubt.

Genau das trifft den temporaeren `cm-acme-http-solver-...`-Pod, den cert-manager fuer die Challenge in `flux-system` anlegt: Traefik sitzt im Namespace `traefik` und darf ihn deshalb nicht mehr erreichen - anders als bei Prometheus/Alertmanager, wo bisher keine `NetworkPolicy` im Weg stand.

```
kubectl -n flux-system get networkpolicy
```

NAME	POD-SELECTOR	AGE
allow-egress	<none>	...
allow-scraping	<none>	...
allow-webhooks	app=notification-controller	...
flux-operator-web	app.kubernetes.io/instance=flux-operator,app.kubernetes.io/name=flux-operator	...

`flux-operator-web` erlaubt die Web-UI selbst zwar schon von ueberall auf Port `9080` - aber der ACME-Solver-Pod ist ein voellig anderer Pod ohne passende Labels und faellt deshalb unter die restriktive `allow-egress`-Regel.

Schritt 6: Fix - gezielte NetworkPolicy fuer den ACME-Solver

Statt die vorhandenen Policies aufzuweichen, erlauben wir gezielt nur den Solver-Pods (erkennbar am Label `acme.cert-manager.io/http01-solver`) Traffic auf ihrem Port von ueberall:

```
cd flux-<dein-kuerzel>/clusters/production/flux-system
```

```
## vi 03-allow-acme-http01-solver.yml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-acme-http01-solver
  namespace: flux-system
spec:
  podSelector:
    matchLabels:
      acme.cert-manager.io/http01-solver: "true"
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector: {}
      ports:
        - protocol: TCP
          port: 8089
```

```
cd
cd flux-<dein-kuerzel>
git add -A
git commit -m "Allow ACME HTTP01 solver ingress from other namespaces"
git push
```

```
flux reconcile kustomization flux-system --with-source
```

```
kubectl -n flux-system get certificate flux-web-tls
```

Nach spaetestens 1-2 Minuten:

NAME	READY	SECRET	AGE
flux-web-tls	True	flux-web-tls	...

Schritt 7: Reports-Feature kennenlernen (FluxReport)

Der Operator legt automatisch genau eine FluxReport -Ressource namens flux an und aktualisiert sie alle 5 Minuten - sie fasst den kompletten Zustand der Flux-Installation zusammen und ist die Datenquelle der Web-UI, die wir gleich im Browser oeffnen.

Abschnitt (spec.)	Inhalt
cluster	Kubernetes-Version, Plattform, Node-Anzahl
distribution	Flux-Version, Installationsstatus
components	Status je Flux-Controller (helm-controller, source-controller, ...)
operator	Version des Flux Operators selbst
reconcilers	Statistik je Ressourcentyp: wie viele failing/running/suspended
sync	Kustomization-ID, Quelle, ausgerollte Revision, Sync-Status

```
kubectl get fluxreport -n flux-system
kubectl -n flux-system get fluxreport flux -o yaml
```

Manuelles Neu-Erstellen (statt auf die naechsten 5 Minuten zu warten):

```
kubectl -n flux-system annotate --overwrite fluxreport/flux \
  reconcile.fluxcd.io/requestedAt="$(date +%s)"
```

Schritt 8: Von aussen testen

Gleiche curl-Pruefung wie bei Prometheus/Alertmanager - derselbe Handler, dieselbe Erwartung:

```
curl -s -o /dev/null -w "%{http_code}\n" https://flux.<du>.do.t3isp.de/
## 401 (ohne Auth - gut so!)

curl -s -o /dev/null -w "%{http_code}\n" -u admin:<dein-passwort> https://flux.<du>.do.t3isp.de/
## 200 (mit Auth)
```

Im Browser: https://flux.<du>.do.t3isp.de -> Login-Popup (basic-auth), danach das Flux-Status-Dashboard - dieselben Infos wie eben im FluxReport , nur grafisch aufbereitet: FluxInstance , GitRepository und alle Kustomization / HelmRelease -Objekte in Echtzeit.

 Flux-Operator Web-UI: Status-Dashboard mit Cluster Info, Cluster Sync und Flux-Komponenten

Aufraeumen

```
cd flux-<dein-kuerzel>/clusters/production/flux-system
rm flux-web-middlewre.yml 03-allow-acme-http01-solver.yml
## in flux-operator-release.yml das values:-Feld (web:) wieder entfernen
git add -A
git commit -m "Removed ingress, basic-auth middleware and ACME solver policy for flux-operator web UI"
git push

flux reconcile kustomization flux-system --with-source
kubectl delete secret flux-web-basic-auth flux-web-tls -n flux-system
```

Das Certificate-Objekt selbst entfernt cert-manager automatisch mit (per ownerReference an den Ingress gebunden) - nur das TLS-Secret bleibt stehen und muss von Hand weg.

Referenzen

- <https://fluxoperator.dev/docs/web-ui/ingress/>
- <https://fluxoperator.dev/docs/charts/flux-operator/>
- <https://artifacthub.io/packages/helm/flux-operator/flux-operator>

Abschluss

Autoscaling für fpm-php (Gedankenexperimente, nicht sinnvol)